

ETL SQL VALIDATION PLAYBOOK

140+ Practical SQL Validation Scenarios

ETL • Big Data • Data Warehouse • Data Quality • BI

A practical SQL reference for validating data across the complete lifecycle: Source → Ingestion → Staging → Transformation → Target → Dimensions/Facts → Reconciliation → CDC → E2E → Metadata → BI → Big Data → Performance → Workflow.

Purpose

Use SQL not only to query data, but to prove data completeness, accuracy, consistency, integrity and business correctness throughout the ETL pipeline.

Prepared as a professional reference from the supplied ETL SQL validation document.

How to Use This Playbook

This playbook reorganizes and professionally presents the supplied SQL validation reference. It preserves the source document's validation areas, purposes and SQL examples rather than replacing them with generic test-case descriptions.

- Start with the validation stage that matches the defect risk you are investigating.
- Read the Purpose statement before executing a query; it explains what the validation is intended to prove.
- Adapt database-specific syntax to the platform used by the project. The source reference includes examples for multiple SQL engines.
- Use the queries as reusable patterns and parameterize table names, keys, dates, thresholds and control values for automation.
- For production automation, capture PASS/FAIL results, exceptions and run metadata so SQL validation becomes an auditable quality gate.

Validation Coverage

#	Validation Area	Coverage
01	Source	Connectivity, availability, counts, keys, duplicates, NULLs, data types, formats, integrity
02	Extraction / Ingestion	Load type, CDC, watermarks, reconciliation, truncation, encoding, compression, rejects
03	Staging	Schema, counts, source-stage matching, NULL propagation, duplicates, partitions, idempotency
04	Transformation	Mappings, derived columns, business rules, lookups, cleansing, aggregation, granularity
05	Target	Target creation, reconciliation, column matching, types, precision, constraints, surrogate keys
06	Dimensions	SCD Type 1/2/3, effective dates, current flags, history, late arriving and junk dimensions
07	Facts	Grain, foreign keys, joins, measures, additive/semi-additive logic, snapshots, duplicates
08	Reconciliation	Counts, control totals, hash totals, aggregation, day-over-day and business reconciliation
09	Incremental / CDC	Insert, update, delete, reprocessing, duplicate prevention, restartability
10	Regression	Previous-data consistency, history, backward compatibility, rerun comparison, drift
11	Data Quality	Completeness, accuracy, consistency, timeliness, uniqueness, validity
12	E2E	Source-stage, stage-warehouse, warehouse-report, golden records, orchestration, deployment regression
13	Metadata & Lineage	Column lineage, catalog completeness, audit trail, schema metadata, run metadata, classification
14	Reporting / BI	Report matching, filters, aggregations, drill-down and dashboard refresh
15	Big Data	HDFS, blocks, partitions, bucketing, Hive tables, ORC/Parquet, schema-on-read, Spark, skew, small files
16	Performance	Load time, query response, partition performance, parallelism, resource utilization, recovery
17	Workflow	Dependencies, sequence, failure handling, restart, alerts, SLA

Core mindset: A green ETL job does not automatically mean green data. The validation objective is to prove that data remains correct at every stage of its journey.

ETL Source Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Source\_Validations\_Python.md`

1. Source Connectivity Validation

Purpose: Verify the source database/system is reachable before ETL starts.

```
-- Oracle / SQL Server / PostgreSQL
SELECT 1 AS connection_check;
-- If this returns a result, connection is alive.
```

2. Table / File Availability Check

Purpose: Confirm the expected source table or file exists.

```
-- SQL Server
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'dbo'
  AND TABLE_NAME = 'ORDERS';
-- Oracle
SELECT TABLE_NAME
FROM ALL_TABLES
WHERE OWNER = 'SOURCE_SCHEMA'
  AND TABLE_NAME = 'ORDERS';
-- PostgreSQL
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_name = 'orders';
```

3. Record Count Validation

Purpose: Compare source vs. target record counts to detect data loss.

```
-- Source count
SELECT COUNT(*) AS source_count FROM source_db.dbo.ORDERS;
-- Target count
SELECT COUNT(*) AS target_count FROM target_db.dbo.ORDERS;
-- Compare (run after load)
SELECT
  s.source_count,
  t.target_count,
  (s.source_count - t.target_count) AS difference,
  CASE WHEN s.source_count = t.target_count THEN 'PASS' ELSE 'FAIL' END AS status
FROM
  (SELECT COUNT(*) AS source_count FROM source_db.dbo.ORDERS) s,
  (SELECT COUNT(*) AS target_count FROM target_db.dbo.ORDERS) t;
```

4. Primary Key Uniqueness

Purpose: Ensure no duplicate primary key values exist in the source.

```
-- Find duplicate PKs
SELECT ORDER_ID, COUNT(*) AS duplicate_count
FROM source_db.dbo.ORDERS
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;
-- Summary check
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT ORDER_ID) AS unique_pks,
  COUNT(*) - COUNT(DISTINCT ORDER_ID) AS duplicate_pk_count,
  CASE WHEN COUNT(*) = COUNT(DISTINCT ORDER_ID) THEN 'PASS' ELSE 'FAIL' END AS status
FROM source_db.dbo.ORDERS;
```

5. Duplicate Record Check

Purpose: Detect fully or partially duplicate rows in source data.

```
-- Full row duplicates
SELECT *, COUNT(*) AS dup_count
FROM source_db.dbo.ORDERS
GROUP BY ORDER_ID, CUSTOMER_ID, ORDER_DATE, AMOUNT
HAVING COUNT(*) > 1;
-- Partial duplicates (business key level)
SELECT CUSTOMER_ID, ORDER_DATE, COUNT(*) AS dup_count
FROM source_db.dbo.ORDERS
GROUP BY CUSTOMER_ID, ORDER_DATE
HAVING COUNT(*) > 1
ORDER BY dup_count DESC;
```

6. NULL / NOT NULL Validation

Purpose: Ensure mandatory columns do not contain NULL values.

```
-- Check NULLs across mandatory columns
SELECT
    SUM(CASE WHEN ORDER_ID IS NULL THEN 1 ELSE 0 END) AS order_id_nulls,
    SUM(CASE WHEN CUSTOMER_ID IS NULL THEN 1 ELSE 0 END) AS customer_id_nulls,
    SUM(CASE WHEN ORDER_DATE IS NULL THEN 1 ELSE 0 END) AS order_date_nulls,
    SUM(CASE WHEN AMOUNT IS NULL THEN 1 ELSE 0 END) AS amount_nulls,
    COUNT(*) AS total_rows
FROM source_db.dbo.ORDERS;
```

7. Data Type Validation

Purpose: Confirm that column data types match the expected schema.

```
-- SQL Server / PostgreSQL - check actual column types
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    NUMERIC_PRECISION,
    NUMERIC_SCALE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'ORDERS'
    AND TABLE_SCHEMA = 'dbo'
ORDER BY ORDINAL_POSITION;
```

8. Length & Precision Checks

Purpose: Ensure string lengths and numeric precision are within allowed bounds.

```
-- Check string columns exceeding max length
SELECT ORDER_ID, CUSTOMER_NAME, LEN(CUSTOMER_NAME) AS name_length
FROM source_db.dbo.ORDERS
WHERE LEN(CUSTOMER_NAME) > 100; -- max allowed length = 100
-- Check numeric precision
SELECT ORDER_ID, AMOUNT
FROM source_db.dbo.ORDERS
WHERE AMOUNT > 999999.99 -- max value check
    OR ROUND(AMOUNT, 2) <> AMOUNT; -- more than 2 decimal places
```

9. Date Format Validation

Purpose: Ensure date columns contain valid, parseable dates in expected format.

```
-- SQL Server: find unparseable dates stored as strings
SELECT ORDER_ID, ORDER_DATE_STR
FROM source_db.dbo.ORDERS
WHERE TRY_CONVERT(DATE, ORDER_DATE_STR, 101) IS NULL -- 101 = MM/DD/YYYY
    AND ORDER_DATE_STR IS NOT NULL;
-- Check for future dates or implausible values
SELECT ORDER_ID, ORDER_DATE
```

```
FROM source_db.dbo.ORDERS
WHERE ORDER_DATE > GETDATE()
   OR ORDER_DATE < '2000-01-01';
```

10. Invalid / Special Character Check

Purpose: Detect unwanted special characters, control characters, or injection risks.

```
-- Detect common special/invalid characters
SELECT ORDER_ID, CUSTOMER_NAME
FROM source_db.dbo.ORDERS
WHERE CUSTOMER_NAME LIKE '%[^a-zA-Z0-9 .,&'']%' -- SQL Server pattern
   OR CUSTOMER_NAME LIKE '%[<>!@#$$%^*]%;';
-- Detect non-printable/control characters (SQL Server)
SELECT ORDER_ID, CUSTOMER_NAME
FROM source_db.dbo.ORDERS
WHERE CUSTOMER_NAME LIKE '%' + CHAR(0) + '%'
   OR CUSTOMER_NAME LIKE '%' + CHAR(9) + '%' -- tab
   OR CUSTOMER_NAME LIKE '%' + CHAR(13) + '%' -- carriage return
   OR CUSTOMER_NAME LIKE '%' + CHAR(10) + '%'; -- line feed
```

11. Referential Integrity (FK Checks)

Purpose: Ensure foreign key values in child table exist in the parent table.

```
-- Orders referencing non-existent customers
SELECT o.ORDER_ID, o.CUSTOMER_ID
FROM source_db.dbo.ORDERS o
LEFT JOIN source_db.dbo.CUSTOMERS c ON o.CUSTOMER_ID = c.CUSTOMER_ID
WHERE c.CUSTOMER_ID IS NULL;
-- Summary
SELECT
    COUNT(*) AS orphan_records,
    CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS status
FROM source_db.dbo.ORDERS o
LEFT JOIN source_db.dbo.CUSTOMERS c ON o.CUSTOMER_ID = c.CUSTOMER_ID
WHERE c.CUSTOMER_ID IS NULL;
```

12. Source Data Completeness

Purpose: Validate that all expected partitions/date ranges/segments are present.

```
-- Check completeness by date (no missing days in last 30 days)
WITH date_series AS (
    SELECT CAST(DATEADD(DAY, -n, GETDATE()) AS DATE) AS expected_date
    FROM (SELECT TOP 30 ROW_NUMBER() OVER (ORDER BY object_id) - 1 AS n
          FROM sys.objects) nums
)
SELECT ds.expected_date, COUNT(o.ORDER_ID) AS record_count
FROM date_series ds
LEFT JOIN source_db.dbo.ORDERS o ON CAST(o.ORDER_DATE AS DATE) = ds.expected_date
GROUP BY ds.expected_date
HAVING COUNT(o.ORDER_ID) = 0
ORDER BY ds.expected_date;
-- Check completeness by region/segment
SELECT REGION, COUNT(*) AS cnt
FROM source_db.dbo.ORDERS
GROUP BY REGION
ORDER BY REGION;
```

13. File Naming Convention Check

Purpose: Validate that incoming files follow the required naming pattern.

```
-- If file metadata is tracked in a control table
SELECT file_name, received_datetime,
CASE
    WHEN file_name LIKE 'ORDERS\[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]\.csv' ESCAPE '\'
    THEN 'PASS'
```

```

        ELSE 'FAIL'
      END AS naming_status
FROM etl_control.dbo.FILE_AUDIT
WHERE received_datetime >= CAST(GETDATE() AS DATE);

```

14. File Delimiter & Schema Validation

Purpose: Ensure CSV/flat files use the correct delimiter and expected column headers.

```

-- If loaded into staging with a schema registry
SELECT column_name, data_type, ordinal_position
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'ORDERS_STAGING'
  AND TABLE_SCHEMA = 'staging'
ORDER BY ORDINAL_POSITION;
-- Compare against expected schema in a control table
SELECT
  e.column_name AS expected_col,
  a.COLUMN_NAME AS actual_col,
  CASE WHEN a.COLUMN_NAME IS NULL THEN 'MISSING' ELSE 'OK' END AS status
FROM etl_control.dbo.EXPECTED_SCHEMA e
LEFT JOIN INFORMATION_SCHEMA.COLUMNS a
  ON e.column_name = a.COLUMN_NAME
  AND a.TABLE_NAME = 'ORDERS_STAGING';

```

15. File Arrival Time (SLA) Validation

Purpose: Verify that source files arrive before the agreed SLA deadline.

```

-- Check file arrival vs SLA from a file audit/control table
SELECT
  file_name,
  expected_arrival_time,
  actual_arrival_time,
  DATEDIFF(MINUTE, expected_arrival_time, actual_arrival_time) AS delay_minutes,
  CASE
    WHEN actual_arrival_time IS NULL THEN 'MISSING'
    WHEN actual_arrival_time <= expected_arrival_time THEN 'ON TIME'
    ELSE 'SLA BREACH'
  END AS sla_status
FROM etl_control.dbo.FILE_SLA_TRACKER
WHERE CAST(expected_arrival_time AS DATE) = CAST(GETDATE() AS DATE);

```

ETL Extraction Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Extraction\_Validations\_Python.md`

1. Full Load vs Incremental Load Validation

Purpose: Verify that full loads capture all source rows, and incremental loads capture only the new/changed rows since the last run.

```
-- — FULL LOAD —————
-- Compare total source vs staging row counts
SELECT
    (SELECT COUNT(*) FROM source_db.dbo.ORDERS)          AS source_count,
    (SELECT COUNT(*) FROM staging_db.dbo.ORDERS_STAGE)   AS stage_count,
    (SELECT COUNT(*) FROM source_db.dbo.ORDERS) -
    (SELECT COUNT(*) FROM staging_db.dbo.ORDERS_STAGE) AS difference,
    CASE
        WHEN (SELECT COUNT(*) FROM source_db.dbo.ORDERS) =
             (SELECT COUNT(*) FROM staging_db.dbo.ORDERS_STAGE)
        THEN 'PASS' ELSE 'FAIL'
    END AS full_load_status;
-- — INCREMENTAL LOAD —————
-- Rows modified since last watermark (stored in control table)
DECLARE @last_run DATETIME = (
    SELECT MAX(last_extracted_ts)
    FROM etl_control.dbo.ETL_RUN_LOG
    WHERE table_name = 'ORDERS'
);
-- Expected incremental count from source
SELECT COUNT(*) AS expected_incremental
FROM source_db.dbo.ORDERS
WHERE LAST_MODIFIED_DT > @last_run;
-- Actual count landed in staging
SELECT COUNT(*) AS actual_incremental
FROM staging_db.dbo.ORDERS_STAGE
WHERE EXTRACT_BATCH_ID = (
    SELECT MAX(EXTRACT_BATCH_ID)
    FROM staging_db.dbo.ORDERS_STAGE
);
```

2. CDC (Insert / Update / Delete) Validation

Purpose: Validate that Change Data Capture correctly captures and classifies INSERT, UPDATE, and DELETE operations.

```
-- — CDC event counts by operation type —————
SELECT
    operation,          -- 'I' = Insert, 'U' = Update, 'D' = Delete
    COUNT(*) AS event_count
FROM cdc_db.dbo.CDC_ORDERS_CHANGES
WHERE commit_time >= CAST(GETDATE() AS DATE)
GROUP BY operation;
-- Validate INSERTs: new PKs in CDC must not exist in target before load
SELECT c.ORDER_ID
FROM cdc_db.dbo.CDC_ORDERS_CHANGES c
LEFT JOIN target_db.dbo.ORDERS t ON c.ORDER_ID = t.ORDER_ID
WHERE c.operation = 'I'
    AND t.ORDER_ID IS NOT NULL; -- should return 0 rows
-- Validate DELETES: deleted PKs must not exist in target after load
SELECT c.ORDER_ID
FROM cdc_db.dbo.CDC_ORDERS_CHANGES c
INNER JOIN target_db.dbo.ORDERS t ON c.ORDER_ID = t.ORDER_ID
WHERE c.operation = 'D'; -- should return 0 rows
-- Validate UPDATES: check that changed column values were applied
SELECT
    c.ORDER_ID,
    c.AMOUNT AS cdc_amount,
```



```

    t.AMOUNT      AS target_amount,
    CASE WHEN c.AMOUNT = t.AMOUNT THEN 'MATCH' ELSE 'MISMATCH' END AS status
FROM cdc_db.dbo.CDC_ORDERS_CHANGES c
INNER JOIN target_db.dbo.ORDERS t ON c.ORDER_ID = t.ORDER_ID
WHERE c.operation = 'U';

```

3. Watermark / High-Water-Mark Validation

Purpose: Ensure the watermark (last extracted timestamp or sequence) is correctly read, applied, and updated after each extraction run.

```

-- -- Read current watermark -----
SELECT table_name, last_extracted_ts, last_batch_id, updated_by
FROM etl_control.dbo.ETL_WATERMARK
WHERE table_name = 'ORDERS';
-- -- Verify no rows are missed below the watermark -----
DECLARE @hwm DATETIME = (
    SELECT last_extracted_ts
    FROM etl_control.dbo.ETL_WATERMARK
    WHERE table_name = 'ORDERS'
);
-- Rows in source older than watermark but NOT in target (missed rows)
SELECT s.ORDER_ID, s.LAST_MODIFIED_DT
FROM source_db.dbo.ORDERS s
LEFT JOIN target_db.dbo.ORDERS t ON s.ORDER_ID = t.ORDER_ID
WHERE s.LAST_MODIFIED_DT <= @hwm
    AND t.ORDER_ID IS NULL;    -- should return 0 rows
-- Rows ABOVE the watermark should not be in target yet
SELECT t.ORDER_ID, s.LAST_MODIFIED_DT
FROM source_db.dbo.ORDERS s
INNER JOIN target_db.dbo.ORDERS t ON s.ORDER_ID = t.ORDER_ID
WHERE s.LAST_MODIFIED_DT > @hwm;    -- should return 0 rows

```

4. Source-to-Stage Count Reconciliation

Purpose: Row-by-row reconciliation between source and staging to catch any extraction gaps per partition, batch, or date.

```

-- Count reconciliation by date partition
SELECT
    CAST(s.ORDER_DATE AS DATE)      AS partition_date,
    COUNT(s.ORDER_ID)              AS source_count,
    COUNT(st.ORDER_ID)             AS stage_count,
    COUNT(s.ORDER_ID) -
    COUNT(st.ORDER_ID)             AS gap,
    CASE WHEN COUNT(s.ORDER_ID) =
        COUNT(st.ORDER_ID)
    THEN 'PASS' ELSE 'FAIL'
    END                            AS status
FROM source_db.dbo.ORDERS s
LEFT JOIN staging_db.dbo.ORDERS_STAGE st
    ON s.ORDER_ID = st.ORDER_ID
GROUP BY CAST(s.ORDER_DATE AS DATE)
HAVING COUNT(s.ORDER_ID) <> COUNT(st.ORDER_ID)
ORDER BY partition_date;
-- Column-level checksum reconciliation
SELECT
    SUM(CAST(AMOUNT AS BIGINT)) AS source_amount_sum,
    COUNT(ORDER_ID)            AS source_count
FROM source_db.dbo.ORDERS
WHERE CAST(ORDER_DATE AS DATE) = CAST(GETDATE() AS DATE);

```

5. Column-Level Extraction Validation

Purpose: Verify that every expected column is present in the staged data and that values match the source row-for-row on a sample or full basis.

```

-- Column presence check vs expected schema
SELECT
    e.column_name,

```

```

CASE WHEN a.COLUMN_NAME IS NULL THEN 'MISSING' ELSE 'PRESENT' END AS status
FROM etl_control.dbo.EXPECTED_COLUMNS e
LEFT JOIN INFORMATION_SCHEMA.COLUMNS a
  ON e.column_name = a.COLUMN_NAME
  AND a.TABLE_NAME = 'ORDERS_STAGE'
  AND a.TABLE_SCHEMA = 'staging';
-- Row-level column value comparison (sample 1000 rows)
SELECT
  s.ORDER_ID,
  s.AMOUNT          AS src_amount,
  st.AMOUNT          AS stg_amount,
  s.CUSTOMER_ID     AS src_cust,
  st.CUSTOMER_ID    AS stg_cust,
  CASE WHEN s.AMOUNT = st.AMOUNT
        AND s.CUSTOMER_ID = st.CUSTOMER_ID
        THEN 'MATCH' ELSE 'MISMATCH' END AS row_status
FROM (SELECT TOP 1000 * FROM source_db.dbo.ORDERS ORDER BY NEWID()) s
INNER JOIN staging_db.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
WHERE s.AMOUNT <> st.AMOUNT
     OR s.CUSTOMER_ID <> st.CUSTOMER_ID;

```

6. Data Truncation Checks

Purpose: Detect values silently truncated during extraction due to column length mismatches between source and staging.

```

-- Compare max actual lengths in source vs defined stage column lengths
SELECT
  c.COLUMN_NAME,
  c.CHARACTER_MAXIMUM_LENGTH AS stage_max_length,
  src.actual_max_length,
  CASE
    WHEN src.actual_max_length > c.CHARACTER_MAXIMUM_LENGTH
    THEN 'TRUNCATION RISK'
    ELSE 'OK'
  END AS status
FROM INFORMATION_SCHEMA.COLUMNS c
CROSS JOIN (
  SELECT
    MAX(LEN(CUSTOMER_NAME)) AS actual_max_length
  FROM source_db.dbo.ORDERS
) src
WHERE c.TABLE_NAME = 'ORDERS_STAGE'
     AND c.COLUMN_NAME = 'CUSTOMER_NAME';
-- Find rows where stage value is shorter than source (actual truncation)
SELECT s.ORDER_ID,
  s.CUSTOMER_NAME      AS src_value,
  st.CUSTOMER_NAME      AS stg_value,
  LEN(s.CUSTOMER_NAME)  AS src_len,
  LEN(st.CUSTOMER_NAME) AS stg_len
FROM source_db.dbo.ORDERS s
INNER JOIN staging_db.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
WHERE LEN(st.CUSTOMER_NAME) < LEN(s.CUSTOMER_NAME);

```

7. Encoding Validation (UTF-8, ASCII, etc.)

Purpose: Ensure source files or data use the expected character encoding and that non-conforming characters are detected.

```

-- SQL Server: detect non-ASCII characters in a VARCHAR column
-- (stored as '?' if encoding mismatch occurred during load)
SELECT ORDER_ID, CUSTOMER_NAME
FROM staging_db.dbo.ORDERS_STAGE
WHERE CUSTOMER_NAME LIKE '%?%'          -- replacement character for bad encoding
     OR CUSTOMER_NAME COLLATE Latin1_General_BIN
     <> CONVERT(VARCHAR, CONVERT(VARBINARY, CUSTOMER_NAME));
-- PostgreSQL: find rows with characters outside ASCII range
SELECT ORDER_ID, CUSTOMER_NAME
FROM orders_stage
WHERE CUSTOMER_NAME ~ '^[^x00-\x7F]';

```

8. Compression Validation (gzip, snappy, etc.)

Purpose: Confirm that compressed files are valid, uncorrupted, and match the expected compression format.

```
-- Track compression metadata in a file audit/control table
SELECT
    file_name,
    compression_type,
    compressed_size_bytes,
    uncompressed_size_bytes,
    compression_ratio,
    is_valid,
    CASE WHEN is_valid = 1 THEN 'PASS' ELSE 'FAIL - CORRUPT' END AS status
FROM etl_control.dbo.FILE_COMPRESSION_AUDIT
WHERE load_date = CAST(GETDATE() AS DATE)
ORDER BY file_name;
```

9. Late Arriving Data Handling

Purpose: Detect records that arrive after their expected processing window and ensure they are correctly routed to the right partition or batch.

```
-- Identify late-arriving records (arrived after their event date partition)
SELECT
    ORDER_ID,
    ORDER_DATE,
    LOAD_TIMESTAMP,
    DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) AS hours_late,
    CASE
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) > 24 THEN 'LATE > 1 DAY'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) > 1 THEN 'LATE > 1 HR'
        ELSE 'ON TIME'
    END AS arrival_status
FROM staging_db.dbo.ORDERS_STAGE
WHERE CAST(LOAD_TIMESTAMP AS DATE) > CAST(ORDER_DATE AS DATE)
ORDER BY hours_late DESC;

-- Count of late records per day
SELECT
    CAST(ORDER_DATE AS DATE) AS event_date,
    COUNT(*) AS late_record_count,
    AVG(DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP)) AS avg_delay_hours
FROM staging_db.dbo.ORDERS_STAGE
WHERE LOAD_TIMESTAMP > DATEADD(DAY, 1, ORDER_DATE)
GROUP BY CAST(ORDER_DATE AS DATE)
ORDER BY event_date DESC;
```

10. Reject / Error Record Capture

Purpose: Ensure that records rejected during extraction are captured in an error/reject table with sufficient context for investigation.

```
-- Review reject records from today's extraction
SELECT
    reject_id,
    source_table,
    batch_id,
    raw_record,
    error_code,
    error_message,
    rejected_at
FROM etl_control.dbo.EXTRACT_REJECTS
WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)
ORDER BY rejected_at DESC;

-- Summary by error type
SELECT
    error_code,
    COUNT(*) AS reject_count,
    MIN(rejected_at) AS first_seen,
    MAX(rejected_at) AS last_seen
```

```

FROM etl_control.dbo.EXTRACT_REJECTS
WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)
GROUP BY error_code
ORDER BY reject_count DESC;
-- Confirm all rejected records are logged (none silently dropped)
SELECT
    (SELECT COUNT(*) FROM source_db.dbo.ORDERS
     WHERE CAST(CREATED_DT AS DATE) = CAST(GETDATE() AS DATE))
    -
    (SELECT COUNT(*) FROM staging_db.dbo.ORDERS_STAGE
     WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE() AS DATE))
    -
    (SELECT COUNT(*) FROM etl_control.dbo.EXTRACT_REJECTS
     WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)
     AND source_table = 'ORDERS')
AS unaccounted_records; -- should be 0

```

11. Bad Record Threshold Validation

Purpose: Fail the ETL pipeline if the percentage of bad/rejected records exceeds a defined acceptable threshold.

```

-- Calculate bad record percentage for current batch
WITH batch_summary AS (
    SELECT
        b.batch_id,
        b.total_source_count,
        COUNT(r.reject_id) AS reject_count
    FROM etl_control.dbo.ETL_BATCH_LOG b
    LEFT JOIN etl_control.dbo.EXTRACT_REJECTS r ON b.batch_id = r.batch_id
    WHERE b.batch_id = 'BATCH_20241201_001'
    GROUP BY b.batch_id, b.total_source_count
)
SELECT
    batch_id,
    total_source_count,
    reject_count,
    ROUND(100.0 * reject_count / NULLIF(total_source_count, 0), 4) AS reject_pct,
    CASE
        WHEN 100.0 * reject_count / NULLIF(total_source_count, 0) <= 1.0 THEN 'PASS'
        WHEN 100.0 * reject_count / NULLIF(total_source_count, 0) <= 5.0 THEN 'WARN'
        ELSE 'FAIL - ABORT ETL'
    END AS threshold_status
FROM batch_summary;

```

Staging Layer Validation Checks

ETL Staging Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Staging\_Validations\_Python.md`

1. Stage Table Creation Validation

Purpose: Confirm that all required staging tables were created successfully

before loading begins. Catches DDL failures early.

```
-- SQL Server: verify all expected stage tables exist
SELECT
    expected.table_name,
    CASE WHEN t.TABLE_NAME IS NULL THEN 'MISSING' ELSE 'EXISTS' END AS status
FROM (
    VALUES
        ('ORDERS_STAGE'),
        ('CUSTOMERS_STAGE'),
        ('PRODUCTS_STAGE'),
        ('INVOICES_STAGE')
) AS expected(table_name)
LEFT JOIN INFORMATION_SCHEMA.TABLES t
    ON t.TABLE_NAME = expected.table_name
   AND t.TABLE_SCHEMA = 'staging'
   AND t.TABLE_TYPE = 'BASE TABLE';
-- Verify table was created today (useful for ephemeral stage tables)
SELECT
    TABLE_NAME,
    CREATE_DATE,
    CASE
        WHEN CAST(CREATE_DATE AS DATE) = CAST(GETDATE() AS DATE)
        THEN 'CREATED TODAY'
        ELSE 'STALE / OLD TABLE'
    END AS creation_status
FROM sys.tables t
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name = 'staging'
   AND t.name IN ('ORDERS_STAGE', 'CUSTOMERS_STAGE')
ORDER BY CREATE_DATE DESC;
```

2. Stage Schema Validation

Purpose: Verify that every staging table has the correct columns, data

types, nullability, and ordering as defined in the canonical schema registry.

```
-- Compare actual stage schema vs expected (stored in a schema registry table)
SELECT
    e.column_name                AS expected_column,
    e.data_type                  AS expected_type,
    e.is_nullable                 AS expected_nullable,
    e.ordinal_position            AS expected_position,
    a.DATA_TYPE                  AS actual_type,
    a.IS_NULLABLE                 AS actual_nullable,
    a.ORDINAL_POSITION           AS actual_position,
    CASE
        WHEN a.COLUMN_NAME IS NULL                THEN 'COLUMN MISSING'
        WHEN a.DATA_TYPE <> e.data_type             THEN 'TYPE MISMATCH'
        WHEN a.IS_NULLABLE <> e.is_nullable         THEN 'NULLABILITY MISMATCH'
        WHEN a.ORDINAL_POSITION <> e.ordinal_position THEN 'ORDER MISMATCH'
        ELSE 'OK'
    END AS validation_status
FROM etl_control.dbo.SCHEMA_REGISTRY e
LEFT JOIN INFORMATION_SCHEMA.COLUMNS a
    ON a.TABLE_NAME = e.table_name
   AND a.TABLE_SCHEMA = 'staging'
   AND a.COLUMN_NAME = e.column_name
```

```

WHERE e.table_name = 'ORDERS_STAGE'
ORDER BY e.ordinal_position;
-- Quick: find any extra columns in stage not in registry
SELECT a.COLUMN_NAME AS extra_column
FROM INFORMATION_SCHEMA.COLUMNS a
LEFT JOIN etl_control.dbo.SCHEMA_REGISTRY e
  ON e.column_name = a.COLUMN_NAME
  AND e.table_name = a.TABLE_NAME
WHERE a.TABLE_NAME = 'ORDERS_STAGE'
  AND a.TABLE_SCHEMA = 'staging'
  AND e.column_name IS NULL;

```

3. Stage Record Count Check

Purpose: Confirm staging row counts are within expected bounds — not zero,

not more than the source, and consistent with historical averages.

```

-- Basic count check
SELECT
  COUNT(*) AS stage_count,
  CASE WHEN COUNT(*) = 0 THEN 'FAIL - EMPTY TABLE'
        ELSE 'PASS'
  END AS empty_check
FROM staging.dbo.ORDERS_STAGE;
-- Compare against historical average (stored in run log)
WITH history AS (
  SELECT
    AVG(record_count) AS avg_count,
    STDEV(record_count) AS stddev_count
  FROM etl_control.dbo.ETL_RUN_LOG
  WHERE table_name = 'ORDERS_STAGE'
    AND run_status = 'SUCCESS'
    AND run_date >= DATEADD(DAY, -30, GETDATE())
)
SELECT
  (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE) AS today_count,
  h.avg_count,
  h.stddev_count,
  ABS((SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE) - h.avg_count)
  / NULLIF(h.stddev_count, 0) AS z_score,
  CASE
    WHEN ABS((SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE) - h.avg_count)
    / NULLIF(h.stddev_count, 0) > 3
    THEN 'ANOMALY - REVIEW'
    ELSE 'WITHIN NORMAL RANGE'
  END AS anomaly_flag
FROM history h;

```

4. Source vs Stage Data Match

Purpose: Confirm that data values in staging exactly mirror the source —

no silent transformations, corruptions, or missing rows.

```

-- Row-level hash comparison (SQL Server)
SELECT
  s.ORDER_ID,
  HASHBYTES('SHA2_256',
    CONCAT(CAST(s.ORDER_ID AS VARCHAR),
      CAST(s.CUSTOMER_ID AS VARCHAR),
      CAST(s.AMOUNT AS VARCHAR),
      ISNULL(s.STATUS, ''),
      CAST(s.ORDER_DATE AS VARCHAR))
  ) AS src_hash,
  HASHBYTES('SHA2_256',
    CONCAT(CAST(st.ORDER_ID AS VARCHAR),
      CAST(st.CUSTOMER_ID AS VARCHAR),
      CAST(st.AMOUNT AS VARCHAR),
      ISNULL(st.STATUS, ''))
  ) AS tgt_hash
FROM staging.dbo.ORDERS_STAGE s
FULL JOIN staging.dbo.ORDERS_STAGE st
  ON s.ORDER_ID = st.ORDER_ID
  AND s.CUSTOMER_ID = st.CUSTOMER_ID
  AND s.AMOUNT = st.AMOUNT
  AND s.ORDER_DATE = st.ORDER_DATE
  AND s.STATUS = st.STATUS

```

```

        CAST(st.ORDER_DATE AS VARCHAR))
    )
    AS stg_hash,
CASE
    WHEN HASHBYTES('SHA2_256',
        CONCAT(CAST(s.ORDER_ID AS VARCHAR),
            CAST(s.CUSTOMER_ID AS VARCHAR),
            CAST(s.AMOUNT AS VARCHAR),
            ISNULL(s.STATUS, ''),
            CAST(s.ORDER_DATE AS VARCHAR)))
    = HASHBYTES('SHA2_256',
        CONCAT(CAST(st.ORDER_ID AS VARCHAR),
            CAST(st.CUSTOMER_ID AS VARCHAR),
            CAST(st.AMOUNT AS VARCHAR),
            ISNULL(st.STATUS, ''),
            CAST(st.ORDER_DATE AS VARCHAR)))
    THEN 'MATCH' ELSE 'MISMATCH'
END AS hash_status
FROM source_db.dbo.ORDERS s
INNER JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
WHERE HASHBYTES('SHA2_256',
    CONCAT(CAST(s.ORDER_ID AS VARCHAR), CAST(s.CUSTOMER_ID AS VARCHAR),
        CAST(s.AMOUNT AS VARCHAR), ISNULL(s.STATUS, ''),
        CAST(s.ORDER_DATE AS VARCHAR)))
<> HASHBYTES('SHA2_256',
    CONCAT(CAST(st.ORDER_ID AS VARCHAR), CAST(st.CUSTOMER_ID AS VARCHAR),
        CAST(st.AMOUNT AS VARCHAR), ISNULL(st.STATUS, ''),
        CAST(st.ORDER_DATE AS VARCHAR)));
-- Aggregate checksum comparison (fast alternative for large tables)
SELECT
    SUM(CAST(AMOUNT AS DECIMAL(18,2))) AS amount_sum,
    COUNT(DISTINCT CUSTOMER_ID) AS unique_customers,
    MIN(ORDER_DATE) AS min_date,
    MAX(ORDER_DATE) AS max_date
FROM source_db.dbo.ORDERS;
-- Run the same for staging and compare manually or in a UNION

```

5. Null Propagation Check

Purpose: Ensure that NULL values from the source are faithfully preserved

in staging — not replaced by defaults, empty strings, or zeros unintentionally.

```

-- Compare NULL counts per column between source and stage
SELECT
    'ORDER_ID' AS column_name,
    SUM(CASE WHEN s.ORDER_ID IS NULL THEN 1 ELSE 0 END) AS src_nulls,
    SUM(CASE WHEN st.ORDER_ID IS NULL THEN 1 ELSE 0 END) AS stg_nulls
FROM source_db.dbo.ORDERS s
INNER JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
UNION ALL
SELECT
    'CUSTOMER_ID',
    SUM(CASE WHEN s.CUSTOMER_ID IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN st.CUSTOMER_ID IS NULL THEN 1 ELSE 0 END)
FROM source_db.dbo.ORDERS s
INNER JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
UNION ALL
SELECT
    'AMOUNT',
    SUM(CASE WHEN s.AMOUNT IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN st.AMOUNT IS NULL THEN 1 ELSE 0 END)
FROM source_db.dbo.ORDERS s
INNER JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID;
-- Detect NULLs silently replaced by empty string in stage
SELECT s.ORDER_ID, s.NOTES AS src_notes, st.NOTES AS stg_notes
FROM source_db.dbo.ORDERS s
INNER JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.ORDER_ID
WHERE s.NOTES IS NULL AND st.NOTES = ''; -- NULL became empty string

```

6. Duplicate Records in Stage

Purpose: Ensure staging contains no duplicate rows — a common symptom of

double-loading, failed truncate-and-reload, or CDC misfire.

```
-- Full duplicate rows in stage
SELECT
    ORDER_ID, CUSTOMER_ID, ORDER_DATE, AMOUNT,
    COUNT(*) AS dup_count
FROM staging.dbo.ORDERS_STAGE
GROUP BY ORDER_ID, CUSTOMER_ID, ORDER_DATE, AMOUNT
HAVING COUNT(*) > 1
ORDER BY dup_count DESC;
-- PK-level duplicates (most critical)
SELECT
    ORDER_ID,
    COUNT(*) AS occurrences
FROM staging.dbo.ORDERS_STAGE
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;
-- Summary: total duplicate rows vs clean rows
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN dup.ORDER_ID IS NOT NULL THEN 1
            ELSE 0 END) AS duplicate_rows,
    COUNT(*) - SUM(CASE WHEN dup.ORDER_ID IS NOT NULL
                       THEN 1 ELSE 0 END) AS clean_rows
FROM staging.dbo.ORDERS_STAGE s
LEFT JOIN (
    SELECT ORDER_ID
    FROM staging.dbo.ORDERS_STAGE
    GROUP BY ORDER_ID
    HAVING COUNT(*) > 1
) dup ON s.ORDER_ID = dup.ORDER_ID;
```

7. Partition Creation Validation

Purpose: For partitioned stage tables (by date, region, etc.) verify that

the correct partitions exist and contain data after each load.

```
-- SQL Server: verify partition row distribution
SELECT
    p.partition_number,
    prv.value AS partition_boundary,
    p.rows AS row_count,
    CASE WHEN p.rows = 0 THEN 'EMPTY'
        ELSE 'POPULATED'
    END AS partition_status
FROM sys.partitions p
INNER JOIN sys.tables t ON p.object_id = t.object_id
INNER JOIN sys.indexes i ON p.object_id = i.object_id
                        AND p.index_id = i.index_id
LEFT JOIN sys.partition_range_values prv
    ON p.partition_number = prv.boundary_id + 1
WHERE t.name = 'ORDERS_STAGE'
    AND i.type <= 1 -- heap or clustered index
ORDER BY p.partition_number;
-- Hive / Spark SQL: verify partitions on a partitioned table
SHOW PARTITIONS staging.orders_stage;
-- Validate today's partition specifically
SELECT
    COUNT(*) AS partition_row_count,
    MIN(ORDER_DATE) AS min_date,
    MAX(ORDER_DATE) AS max_date
FROM staging.orders_stage
WHERE order_date = CURRENT_DATE;
-- Detect empty partitions within expected range
SELECT
```



```

    partition_dt,
    row_count,
    CASE WHEN row_count = 0 THEN 'EMPTY PARTITION' ELSE 'OK' END AS status
FROM (
    SELECT
        CAST(ORDER_DATE AS DATE) AS partition_dt,
        COUNT(*) AS row_count
    FROM staging.dbo.ORDERS_STAGE
    WHERE ORDER_DATE >= DATEADD(DAY, -7, GETDATE())
    GROUP BY CAST(ORDER_DATE AS DATE)
) part_summary
ORDER BY partition_dt;

```

8. File Count & File Size Validation

Purpose: Verify the correct number of files landed in the staging/landing

zone and that each file meets minimum and maximum size thresholds.

```

-- File audit from control/metadata table
SELECT
    file_name,
    file_size_bytes,
    row_count,
    load_timestamp,
    CASE
        WHEN file_size_bytes = 0 THEN 'FAIL - EMPTY FILE'
        WHEN file_size_bytes < 1024 THEN 'WARN - VERY SMALL'
        WHEN file_size_bytes > 10 * 1024 * 1024 * 1024 THEN 'WARN - VERY LARGE'
        ELSE 'OK'
    END AS size_status
FROM etl_control.dbo.FILE_AUDIT
WHERE CAST(load_timestamp AS DATE) = CAST(GETDATE() AS DATE)
ORDER BY file_name;
-- Check expected file count for today's batch
SELECT
    COUNT(*) AS file_count,
    SUM(file_size_bytes) AS total_size_bytes,
    MIN(file_size_bytes) AS min_file_size,
    MAX(file_size_bytes) AS max_file_size,
    CASE WHEN COUNT(*) = 5 THEN 'PASS'
        ELSE 'FAIL - Expected 5 files'
    END AS count_status
FROM etl_control.dbo.FILE_AUDIT
WHERE CAST(load_timestamp AS DATE) = CAST(GETDATE() AS DATE)
    AND source_system = 'ORDERS';

```

9. Landing Zone Cleanup Confirmation

Purpose: After successful staging, confirm that processed files are removed

(or archived) from the landing zone to prevent reprocessing on the next run.

```

-- Check landing zone file status in audit table
SELECT
    file_name,
    landing_path,
    processed_flag,
    archived_path,
    deleted_at,
    CASE
        WHEN processed_flag = 1 AND deleted_at IS NOT NULL THEN 'DELETED'
        WHEN processed_flag = 1 AND archived_path IS NOT NULL THEN 'ARCHIVED'
        WHEN processed_flag = 1 THEN 'PROCESSED - NOT CLEANED'
        ELSE 'UNPROCESSED'
    END AS cleanup_status
FROM etl_control.dbo.LANDING_ZONE_AUDIT
WHERE CAST(load_date AS DATE) = CAST(GETDATE() AS DATE)
ORDER BY file_name;
-- Files still sitting in landing zone past their SLA (should be gone)

```

```

SELECT file_name, landing_path, load_date
FROM etl_control.dbo.LANDING_ZONE_AUDIT
WHERE processed_flag = 1
      AND deleted_at      IS NULL
      AND archived_path   IS NULL
      AND load_date       < DATEADD(HOUR, -2, GETDATE()); -- SLA = 2 hours post-processing

```

10. Rerun Idempotency Check

Purpose: Guarantee that re-running the staging load produces the same

result as the first run — no double inserts, no cumulative drift.

Critical for fault-tolerant, self-healing ETL pipelines.

```

-- Capture row count before rerun
DECLARE @count_before INT = (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE);
-- Simulate rerun: truncate + reload (idempotent pattern)
TRUNCATE TABLE staging.dbo.ORDERS_STAGE;
-- ... reload logic here ...
-- Count after rerun should equal count before
DECLARE @count_after INT = (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE);
SELECT
    @count_before AS count_before_rerun,
    @count_after  AS count_after_rerun,
    @count_after - @count_before AS difference,
    CASE WHEN @count_before = @count_after THEN 'IDEMPOTENT - PASS'
          ELSE 'NOT IDEMPOTENT - FAIL'
    END AS idempotency_status;
-- Verify no duplicate batch entries in run log
SELECT batch_id, COUNT(*) AS run_count
FROM etl_control.dbo.ETL_RUN_LOG
WHERE table_name = 'ORDERS_STAGE'
      AND run_date = CAST(GETDATE() AS DATE)
GROUP BY batch_id
HAVING COUNT(*) > 1;
-- Checksum comparison: pre-run vs post-rerun
SELECT
    SUM(CAST(AMOUNT AS BIGINT)) AS amount_checksum,
    COUNT(*)                  AS row_count,
    COUNT(DISTINCT ORDER_ID)  AS unique_orders
FROM staging.dbo.ORDERS_STAGE;
-- Run before and after rerun; both should be identical

```

Transformation Testing Checks (Core ETL)

ETL Transformation Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Transformation\_Validations\_Python.md`

⚠ This is where 90% of ETL bugs live — test every rule, every branch, every edge case.

1. Column Mapping Validation

Purpose: Confirm every source column lands in the correct target column —

right name, right position, no swapped or dropped mappings.

```
-- Compare source → target column mappings via a mapping registry
SELECT
    m.source_column,
    m.target_column,
    m.transformation_rule,
    src_col.DATA_TYPE          AS source_dtype,
    tgt_col.DATA_TYPE          AS target_dtype,
    CASE
        WHEN tgt_col.COLUMN_NAME IS NULL      THEN 'TARGET COLUMN MISSING'
        WHEN src_col.COLUMN_NAME IS NULL      THEN 'SOURCE COLUMN MISSING'
        WHEN src_col.DATA_TYPE <> tgt_col.DATA_TYPE
            AND m.transformation_rule = 'DIRECT' THEN 'TYPE MISMATCH - NO CAST'
        ELSE 'OK'
    END AS mapping_status
FROM etl_control.dbo.COLUMN_MAPPING_REGISTRY m
LEFT JOIN INFORMATION_SCHEMA.COLUMNS src_col
    ON src_col.TABLE_NAME = m.source_table
    AND src_col.COLUMN_NAME = m.source_column
LEFT JOIN INFORMATION_SCHEMA.COLUMNS tgt_col
    ON tgt_col.TABLE_NAME = m.target_table
    AND tgt_col.COLUMN_NAME = m.target_column
WHERE m.pipeline_name = 'ORDERS_ETL'
ORDER BY m.ordinal_position;
-- Verify actual data moved to the right column (spot check 100 rows)
SELECT TOP 100
    s.ORDER_ID,
    s.CUST_ID          AS source_customer_id,
    t.CUSTOMER_ID      AS target_customer_id,  -- mapped from CUST_ID → CUSTOMER_ID
    s.ORD_DT           AS source_order_date,
    t.ORDER_DATE       AS target_order_date,  -- mapped from ORD_DT → ORDER_DATE
    CASE
        WHEN s.CUST_ID = t.CUSTOMER_ID
            AND s.ORD_DT = t.ORDER_DATE
        THEN 'MATCH' ELSE 'MISMATCH'
    END AS mapping_check
FROM source_db.dbo.ORDERS s
JOIN target_db.dbo.ORDERS_DWH t ON s.ORDER_ID = t.ORDER_ID;
```

2. Derived Column Calculation

Purpose: Verify that calculated/derived columns (e.g., NET_AMOUNT, AGE,

PROFIT_MARGIN) produce the mathematically correct result.

```
-- Validate NET_AMOUNT = GROSS_AMOUNT - DISCOUNT
SELECT
    ORDER_ID,
    GROSS_AMOUNT,
    DISCOUNT,
    NET_AMOUNT          AS stored_net,
    GROSS_AMOUNT - DISCOUNT AS expected_net,
    ABS(NET_AMOUNT - (GROSS_AMOUNT - DISCOUNT)) AS difference,
    CASE
        WHEN ABS(NET_AMOUNT - (GROSS_AMOUNT - DISCOUNT)) < 0.01
        THEN 'PASS' ELSE 'FAIL'
```

```

    END AS calc_status
FROM target_db.dbo.ORDERS_DWH
WHERE ABS(NET_AMOUNT - (GROSS_AMOUNT - DISCOUNT)) >= 0.01;
-- Validate AGE = DATEDIFF(YEAR, BIRTH_DATE, GETDATE())
SELECT
    CUSTOMER_ID,
    BIRTH_DATE,
    AGE                                AS stored_age,
    DATEDIFF(YEAR, BIRTH_DATE, GETDATE()) AS expected_age,
    CASE WHEN AGE = DATEDIFF(YEAR, BIRTH_DATE, GETDATE())
        THEN 'PASS' ELSE 'FAIL'
    END AS age_status
FROM target_db.dbo.CUSTOMERS_DWH
WHERE AGE <> DATEDIFF(YEAR, BIRTH_DATE, GETDATE());
-- Validate PROFIT_MARGIN % = (REVENUE - COST) / REVENUE * 100
SELECT
    PRODUCT_ID,
    REVENUE, COST, PROFIT_MARGIN        AS stored_margin,
    ROUND((REVENUE - COST) / NULLIF(REVENUE, 0) * 100, 2) AS expected_margin,
    CASE
        WHEN ABS(PROFIT_MARGIN
            - ROUND((REVENUE - COST) / NULLIF(REVENUE, 0) * 100, 2)) < 0.01
        THEN 'PASS' ELSE 'FAIL'
    END AS margin_status
FROM target_db.dbo.PRODUCTS_DWH
WHERE ABS(PROFIT_MARGIN
    - ROUND((REVENUE - COST) / NULLIF(REVENUE, 0) * 100, 2)) >= 0.01;

```

3. Conditional Logic Validation

Purpose: Test every branch of IF/CASE/WHEN logic — including boundary

conditions, NULL branches, and catch-all ELSE paths.

```

-- Validate ORDER_STATUS_LABEL derived via CASE statement
-- Rule: AMOUNT >= 1000 → 'HIGH', 500-999 → 'MEDIUM', < 500 → 'LOW'
SELECT
    ORDER_ID,
    AMOUNT,
    ORDER_STATUS_LABEL                AS stored_label,
    CASE
        WHEN AMOUNT >= 1000 THEN 'HIGH'
        WHEN AMOUNT >= 500  THEN 'MEDIUM'
        ELSE                  'LOW'
    END                                AS expected_label,
    CASE
        WHEN ORDER_STATUS_LABEL =
            CASE WHEN AMOUNT >= 1000 THEN 'HIGH'
                WHEN AMOUNT >= 500  THEN 'MEDIUM'
                ELSE 'LOW' END
        THEN 'PASS' ELSE 'FAIL'
    END AS logic_check
FROM target_db.dbo.ORDERS_DWH
WHERE ORDER_STATUS_LABEL <>
    CASE WHEN AMOUNT >= 1000 THEN 'HIGH'
        WHEN AMOUNT >= 500  THEN 'MEDIUM'
        ELSE 'LOW' END;
-- Test boundary values specifically
SELECT
    AMOUNT,
    ORDER_STATUS_LABEL,
    CASE WHEN AMOUNT IN (499.99, 500, 500.01, 999.99, 1000, 1000.01)
        THEN 'BOUNDARY ROW' END AS boundary_flag
FROM target_db.dbo.ORDERS_DWH
WHERE AMOUNT IN (499.99, 500, 500.01, 999.99, 1000, 1000.01);

```

4. Default Value Logic

Purpose: Confirm that NULL or missing source values are replaced by the

correct business default — and that non-NULL values are NOT overwritten.

```
-- Rule: COUNTRY defaults to 'US' when NULL in source
SELECT
    t.CUSTOMER_ID,
    s.COUNTRY          AS source_country,
    t.COUNTRY          AS target_country,
    CASE
        WHEN s.COUNTRY IS NULL AND t.COUNTRY = 'US' THEN 'DEFAULT APPLIED - PASS'
        WHEN s.COUNTRY IS NULL AND t.COUNTRY <> 'US' THEN 'WRONG DEFAULT - FAIL'
        WHEN s.COUNTRY IS NOT NULL
            AND t.COUNTRY = s.COUNTRY THEN 'VALUE PRESERVED - PASS'
        WHEN s.COUNTRY IS NOT NULL
            AND t.COUNTRY <> s.COUNTRY THEN 'VALUE OVERWRITTEN - FAIL'
        ELSE 'UNKNOWN'
    END AS default_logic_status
FROM source_db.dbo.CUSTOMERS s
JOIN target_db.dbo.CUSTOMERS_DWH t ON s.CUSTOMER_ID = t.CUSTOMER_ID;
-- Summary: count each outcome
SELECT default_logic_status, COUNT(*) AS cnt
FROM (
    SELECT
        CASE
            WHEN s.COUNTRY IS NULL AND t.COUNTRY = 'US' THEN 'DEFAULT APPLIED - PASS'
            WHEN s.COUNTRY IS NULL AND t.COUNTRY <> 'US' THEN 'WRONG DEFAULT - FAIL'
            WHEN s.COUNTRY IS NOT NULL
                AND t.COUNTRY = s.COUNTRY THEN 'VALUE PRESERVED - PASS'
            ELSE 'VALUE OVERWRITTEN - FAIL'
        END AS default_logic_status
    FROM source_db.dbo.CUSTOMERS s
    JOIN target_db.dbo.CUSTOMERS_DWH t ON s.CUSTOMER_ID = t.CUSTOMER_ID
) x
GROUP BY default_logic_status;
```

5. Lookup Transformation Validation

Purpose: Confirm that lookup joins return the correct enriched value and

that unmatched rows are handled correctly (default, reject, or NULL).

```
-- Validate PRODUCT_CATEGORY was correctly looked up from PRODUCT_DIM
SELECT
    o.ORDER_ID,
    o.PRODUCT_ID,
    o.PRODUCT_CATEGORY          AS stored_category,
    p.CATEGORY                  AS lookup_category,
    CASE
        WHEN p.PRODUCT_ID IS NULL
            AND o.PRODUCT_CATEGORY = 'UNKNOWN' THEN 'UNMATCHED - DEFAULT OK'
        WHEN p.PRODUCT_ID IS NULL
            AND o.PRODUCT_CATEGORY <> 'UNKNOWN' THEN 'UNMATCHED - WRONG DEFAULT'
        WHEN o.PRODUCT_CATEGORY = p.CATEGORY THEN 'MATCH - PASS'
        ELSE 'MISMATCH - FAIL'
    END AS lookup_status
FROM target_db.dbo.ORDERS_DWH o
LEFT JOIN reference_db.dbo.PRODUCT_DIM p
    ON o.PRODUCT_ID = p.PRODUCT_ID
WHERE o.PRODUCT_CATEGORY <>
    COALESCE(p.CATEGORY, 'UNKNOWN');
-- Count unmatched lookup keys (orphaned FKs)
SELECT COUNT(*) AS unmatched_count
FROM target_db.dbo.ORDERS_DWH o
LEFT JOIN reference_db.dbo.PRODUCT_DIM p ON o.PRODUCT_ID = p.PRODUCT_ID
WHERE p.PRODUCT_ID IS NULL;
```

6. Reference Data Mapping

Purpose: Validate that code/value translations from reference tables

(e.g., STATUS_CODE 'A' → 'ACTIVE') are applied correctly and completely.

```
-- Validate STATUS_CODE → STATUS_DESC mapping
SELECT
    t.ORDER_ID,
    t.STATUS_CODE,
    t.STATUS_DESC                AS stored_desc,
    r.STATUS_DESC                AS reference_desc,
    CASE
        WHEN r.STATUS_CODE IS NULL THEN 'CODE NOT IN REFERENCE - FAIL'
        WHEN t.STATUS_DESC = r.STATUS_DESC THEN 'PASS'
        ELSE 'WRONG MAPPING - FAIL'
    END AS ref_mapping_status
FROM target_db.dbo.ORDERS_DWH t
LEFT JOIN reference_db.dbo.STATUS_REF r ON t.STATUS_CODE = r.STATUS_CODE
WHERE t.STATUS_DESC <> ISNULL(r.STATUS_DESC, '__MISSING__');
-- Find any codes used in data but missing from reference table
SELECT DISTINCT STATUS_CODE
FROM target_db.dbo.ORDERS_DWH
WHERE STATUS_CODE NOT IN (
    SELECT STATUS_CODE FROM reference_db.dbo.STATUS_REF
);
```

7. CASE Statement Validation

Purpose: Exhaustively test all CASE branches — including NULL input,

overlapping conditions, and the catch-all ELSE path.

```
-- Validate TIER classification CASE logic
-- Rule: SPEND > 10000 = 'PLATINUM', > 5000 = 'GOLD', > 1000 = 'SILVER', else 'BRONZE'
SELECT
    CUSTOMER_ID,
    TOTAL_SPEND,
    CUSTOMER_TIER                AS stored_tier,
    CASE
        WHEN TOTAL_SPEND > 10000 THEN 'PLATINUM'
        WHEN TOTAL_SPEND > 5000 THEN 'GOLD'
        WHEN TOTAL_SPEND > 1000 THEN 'SILVER'
        ELSE 'BRONZE'
    END
    AS expected_tier,
    CASE
        WHEN CUSTOMER_TIER = CASE
            WHEN TOTAL_SPEND > 10000 THEN 'PLATINUM'
            WHEN TOTAL_SPEND > 5000 THEN 'GOLD'
            WHEN TOTAL_SPEND > 1000 THEN 'SILVER'
            ELSE 'BRONZE' END
        THEN 'PASS' ELSE 'FAIL'
    END AS case_status
FROM target_db.dbo.CUSTOMERS_DWH
WHERE CUSTOMER_TIER <> CASE
    WHEN TOTAL_SPEND > 10000 THEN 'PLATINUM'
    WHEN TOTAL_SPEND > 5000 THEN 'GOLD'
    WHEN TOTAL_SPEND > 1000 THEN 'SILVER'
    ELSE 'BRONZE' END;
-- Verify distribution across all CASE outcomes (no branch starved)
SELECT CUSTOMER_TIER, COUNT(*) AS cnt,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct
FROM target_db.dbo.CUSTOMERS_DWH
GROUP BY CUSTOMER_TIER
ORDER BY cnt DESC;
```

8. De-duplication Logic

Purpose: Confirm the de-duplication strategy (keep first, keep last, keep

by rank) correctly eliminates duplicates and retains the right survivor record.

```
-- Verify no duplicates remain in target after de-dup
SELECT ORDER_ID, COUNT(*) AS cnt
FROM target_db.dbo.ORDERS_DWH
GROUP BY ORDER_ID
```

```

HAVING COUNT(*) > 1;
-- Validate correct survivor was kept (keep latest by MODIFIED_DT)
WITH ranked AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY ORDER_ID
            ORDER BY MODIFIED_DT DESC
        ) AS rn
    FROM source_db.dbo.ORDERS
)
SELECT
    r.ORDER_ID,
    r.MODIFIED_DT AS expected_survivor_dt,
    t.MODIFIED_DT AS target_survivor_dt,
    CASE
        WHEN r.MODIFIED_DT = t.MODIFIED_DT THEN 'CORRECT SURVIVOR'
        ELSE 'WRONG SURVIVOR - FAIL'
    END AS dedup_status
FROM ranked r
JOIN target_db.dbo.ORDERS_DWH t ON r.ORDER_ID = t.ORDER_ID
WHERE r.rn = 1
    AND r.MODIFIED_DT <> t.MODIFIED_DT;

```

9. Standardization Rules

Purpose: Verify that text values are standardized consistently —

case normalization, phone/date formats, gender codes, country codes, etc.

```

-- Validate EMAIL is lowercase
SELECT CUSTOMER_ID, EMAIL
FROM target_db.dbo.CUSTOMERS_DWH
WHERE EMAIL <> LOWER(EMAIL);
-- Validate PHONE is in E.164 format (+1XXXXXXXX)
SELECT CUSTOMER_ID, PHONE
FROM target_db.dbo.CUSTOMERS_DWH
WHERE PHONE NOT LIKE '+1[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'
    AND PHONE IS NOT NULL;
-- Validate GENDER is standardized to 'M' / 'F' / 'U'
SELECT CUSTOMER_ID, GENDER
FROM target_db.dbo.CUSTOMERS_DWH
WHERE GENDER NOT IN ('M', 'F', 'U');
-- Validate COUNTRY_CODE is ISO 3166-1 alpha-2 (2 uppercase chars)
SELECT CUSTOMER_ID, COUNTRY_CODE
FROM target_db.dbo.CUSTOMERS_DWH
WHERE COUNTRY_CODE NOT LIKE '[A-Z][A-Z]'
    OR LEN(COUNTRY_CODE) <> 2;

```

10. Data Cleansing Rules

Purpose: Verify cleansing transformations — removal of test/dummy records,

invalid values, out-of-range data, and known garbage patterns.

```

-- Rule 1: Test accounts removed (email contains 'test' or 'dummy')
SELECT COUNT(*) AS test_records_remaining
FROM target_db.dbo.CUSTOMERS_DWH
WHERE LOWER(EMAIL) LIKE '%test%'
    OR LOWER(EMAIL) LIKE '%dummy%'
    OR LOWER(EMAIL) LIKE '%example.com'; -- should be 0
-- Rule 2: Invalid AGE removed (< 0 or > 120)
SELECT COUNT(*) AS invalid_age_count
FROM target_db.dbo.CUSTOMERS_DWH
WHERE AGE < 0 OR AGE > 120; -- should be 0
-- Rule 3: Negative AMOUNT removed or flagged
SELECT COUNT(*) AS negative_amount_count
FROM target_db.dbo.ORDERS_DWH
WHERE AMOUNT < 0; -- should be 0 after cleansing
-- Rule 4: Placeholder values removed
SELECT COUNT(*) AS placeholder_count

```

```
FROM target_db.dbo.CUSTOMERS_DWH
WHERE LOWER(FULL_NAME) IN ('n/a', 'na', 'null', 'none', 'unknown', 'tbd', 'test');
```

11. Trimming & Formatting Checks

Purpose: Confirm that leading/trailing whitespace, extra spaces, tabs,

newlines, and formatting artifacts have been removed from text columns.

```
-- Detect leading/trailing whitespace
SELECT CUSTOMER_ID, FULL_NAME,
       LEN(FULL_NAME) - LEN(LTRIM(RTRIM(FULL_NAME))) AS whitespace_chars
FROM target_db.dbo.CUSTOMERS_DWH
WHERE FULL_NAME <> LTRIM(RTRIM(FULL_NAME));
-- Detect double spaces inside values
SELECT CUSTOMER_ID, FULL_NAME
FROM target_db.dbo.CUSTOMERS_DWH
WHERE FULL_NAME LIKE '% %'; -- two consecutive spaces
-- Detect tab or newline characters
SELECT CUSTOMER_ID, NOTES
FROM target_db.dbo.ORDERS_DWH
WHERE NOTES LIKE '%' + CHAR(9) + '%' -- tab
      OR NOTES LIKE '%' + CHAR(10) + '%' -- newline
      OR NOTES LIKE '%' + CHAR(13) + '%'; -- carriage return
-- Validate date formatted as YYYY-MM-DD string (if stored as VARCHAR)
SELECT ORDER_ID, ORDER_DATE_STR
FROM target_db.dbo.ORDERS_DWH
WHERE ORDER_DATE_STR NOT LIKE '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]';
```

12. Null Handling Logic

Purpose: Validate every deliberate NULL-handling transformation —

COALESCE, NULLIF, ISNULL — and confirm the correct action was taken

(fill, keep, or convert to NULL).

```
-- Validate COALESCE(DISCOUNT, 0) applied: no NULLs remain in DISCOUNT
SELECT COUNT(*) AS null_discount_count
FROM target_db.dbo.ORDERS_DWH
WHERE DISCOUNT IS NULL; -- should be 0 after COALESCE(DISCOUNT, 0)
-- Validate NULLIF(AMOUNT, 0) applied: zero amounts converted to NULL
SELECT COUNT(*) AS zero_amount_remaining
FROM target_db.dbo.ORDERS_DWH
WHERE AMOUNT = 0; -- should be 0 if NULLIF was applied
-- Validate NULL fill was correct: NULLs in source → 0 in target
SELECT
    s.ORDER_ID,
    s.DISCOUNT AS source_discount, -- NULL
    t.DISCOUNT AS target_discount, -- should be 0
    CASE
        WHEN s.DISCOUNT IS NULL AND t.DISCOUNT = 0 THEN 'COALESCE OK'
        WHEN s.DISCOUNT IS NULL AND t.DISCOUNT <> 0 THEN 'WRONG FILL VALUE'
        WHEN s.DISCOUNT IS NOT NULL
            AND t.DISCOUNT = s.DISCOUNT THEN 'VALUE PRESERVED'
        ELSE 'VALUE CHANGED - INVESTIGATE'
    END AS null_handling_status
FROM source_db.dbo.ORDERS s
JOIN target_db.dbo.ORDERS_DWH t ON s.ORDER_ID = t.ORDER_ID;
```

13. SUM / COUNT / AVG Validation

Purpose: Verify aggregate metrics in the transformed output match

recalculations from the source — the most common ETL data trust issue.

```
-- Validate aggregates: source vs target
SELECT
    'SUM_AMOUNT' AS metric,
    SUM(s.AMOUNT) AS source_value,
    SUM(t.AMOUNT) AS target_value,
```



```

    ABS(SUM(s.AMOUNT) - SUM(t.AMOUNT)) AS abs_diff,
    CASE WHEN ABS(SUM(s.AMOUNT) - SUM(t.AMOUNT)) < 0.01
         THEN 'PASS' ELSE 'FAIL' END AS status
FROM source_db.dbo.ORDERS s, target_db.dbo.ORDERS_DWH t
UNION ALL
SELECT
    'COUNT_ORDERS',
    COUNT(s.ORDER_ID), COUNT(t.ORDER_ID),
    ABS(COUNT(s.ORDER_ID) - COUNT(t.ORDER_ID)),
    CASE WHEN COUNT(s.ORDER_ID) = COUNT(t.ORDER_ID)
         THEN 'PASS' ELSE 'FAIL' END
FROM source_db.dbo.ORDERS s, target_db.dbo.ORDERS_DWH t
UNION ALL
SELECT
    'AVG_AMOUNT',
    AVG(CAST(s.AMOUNT AS FLOAT)),
    AVG(CAST(t.AMOUNT AS FLOAT)),
    ABS(AVG(CAST(s.AMOUNT AS FLOAT)) - AVG(CAST(t.AMOUNT AS FLOAT))),
    CASE WHEN ABS(AVG(CAST(s.AMOUNT AS FLOAT)) -
                  AVG(CAST(t.AMOUNT AS FLOAT))) < 0.01
         THEN 'PASS' ELSE 'FAIL' END
FROM source_db.dbo.ORDERS s, target_db.dbo.ORDERS_DWH t;

```

14. Group By Validation

Purpose: Confirm that **GROUP BY** aggregations are applied at the correct

grain — right grouping keys, correct aggregated values per group.

```

-- Validate daily revenue totals: source grouped vs target grouped
SELECT
    s.order_date,
    s.daily_sum           AS source_daily_sum,
    t.daily_sum           AS target_daily_sum,
    ABS(s.daily_sum - t.daily_sum) AS diff,
    CASE WHEN ABS(s.daily_sum - t.daily_sum) < 0.01
         THEN 'PASS' ELSE 'FAIL' END AS status
FROM (
    SELECT CAST(ORDER_DATE AS DATE) AS order_date,
           SUM(AMOUNT)              AS daily_sum
    FROM source_db.dbo.ORDERS
    GROUP BY CAST(ORDER_DATE AS DATE)
) s
FULL OUTER JOIN (
    SELECT ORDER_DATE,
           SUM(AMOUNT) AS daily_sum
    FROM target_db.dbo.DAILY_REVENUE_AGG
    GROUP BY ORDER_DATE
) t ON s.order_date = t.ORDER_DATE
WHERE ABS(ISNULL(s.daily_sum,0) - ISNULL(t.daily_sum,0)) >= 0.01
ORDER BY s.order_date;

-- Validate group count: source groups vs target groups
SELECT
    (SELECT COUNT(DISTINCT CAST(ORDER_DATE AS DATE))
     FROM source_db.dbo.ORDERS)          AS source_groups,
    (SELECT COUNT(*)
     FROM target_db.dbo.DAILY_REVENUE_AGG) AS target_groups;

```

15. Granularity Validation

Purpose: Verify the transformed dataset is at the correct level of

granularity — one row per expected key combination, no over- or under-aggregation.

```

-- Expect one row per (ORDER_ID, PRODUCT_ID) — check grain
SELECT
    ORDER_ID, PRODUCT_ID,
    COUNT(*) AS row_count
FROM target_db.dbo.ORDER_LINE_ITEMS
GROUP BY ORDER_ID, PRODUCT_ID

```

```

HAVING COUNT(*) > 1;           -- should return 0 rows
-- Validate total row count matches expected grain
-- (orders × avg_lines_per_order should ≈ order_line_items count)
SELECT
  (SELECT COUNT(DISTINCT ORDER_ID) FROM source_db.dbo.ORDERS)      AS order_count,
  (SELECT COUNT(*)          FROM target_db.dbo.ORDER_LINE_ITEMS) AS line_item_count,
  ROUND(
    CAST((SELECT COUNT(*) FROM target_db.dbo.ORDER_LINE_ITEMS) AS FLOAT)
    / NULLIF((SELECT COUNT(DISTINCT ORDER_ID) FROM source_db.dbo.ORDERS), 0),
    2
  ) AS avg_lines_per_order;
-- Find collapsed rows (under-aggregation detection)
SELECT ORDER_ID, COUNT(DISTINCT PRODUCT_ID) AS products_in_target
FROM target_db.dbo.ORDER_LINE_ITEMS
GROUP BY ORDER_ID
HAVING COUNT(DISTINCT PRODUCT_ID) <
  (SELECT COUNT(*) FROM source_db.dbo.ORDER_LINES
   WHERE ORDER_LINES.ORDER_ID = ORDER_LINE_ITEMS.ORDER_ID);

```

16. Double Aggregation Issue Detection

Purpose: Detect the classic ETL bug where data is aggregated at the source

AND again at the target, producing inflated metrics (e.g., counts 2× or 4× too large).

```

-- Compare raw source total vs aggregated target total
-- If target total >> source total → double aggregation likely
SELECT
  (SELECT SUM(AMOUNT) FROM source_db.dbo.ORDERS)      AS raw_source_sum,
  (SELECT SUM(DAILY_AMOUNT) FROM target_db.dbo.DAILY_REVENUE_AGG) AS agg_target_sum,
  (SELECT SUM(DAILY_AMOUNT) FROM target_db.dbo.DAILY_REVENUE_AGG)
  / NULLIF((SELECT SUM(AMOUNT) FROM source_db.dbo.ORDERS), 0) AS ratio,
  CASE
    WHEN ABS(
      (SELECT SUM(DAILY_AMOUNT) FROM target_db.dbo.DAILY_REVENUE_AGG)
      - (SELECT SUM(AMOUNT) FROM source_db.dbo.ORDERS)
    ) < 0.01 THEN 'PASS'
    WHEN (SELECT SUM(DAILY_AMOUNT) FROM target_db.dbo.DAILY_REVENUE_AGG)
      > (SELECT SUM(AMOUNT) FROM source_db.dbo.ORDERS) * 1.01
      THEN 'OVER-AGGREGATION'
    ELSE THEN 'UNDER-AGGREGATION'
  END AS double_agg_status;
-- Detect fan-out: if joining bloats row count
-- (classic symptom of missing DISTINCT or wrong JOIN)
SELECT
  COUNT(*)          AS joined_count,
  COUNT(DISTINCT ORDER_ID) AS distinct_orders,
  COUNT(*) / NULLIF(COUNT(DISTINCT ORDER_ID), 0) AS fan_out_ratio
FROM (
  SELECT o.ORDER_ID
  FROM source_db.dbo.ORDERS o
  JOIN source_db.dbo.ORDER_LINES ol ON o.ORDER_ID = ol.ORDER_ID
) fan;
-- fan_out_ratio > 1 is expected only if that JOIN is intentional

```

Target (Warehouse / Hive) Validation Checks

ETL Target Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Target\_Validations\_Python.md`

 Final gate — if data passes here, it's analytics-ready and trusted by consumers.

1. Target Table Creation Validation

Purpose: Confirm all target warehouse/Hive tables (fact, dimension, aggregate)

were created with the correct structure, storage format, partitioning, and clustering before any data lands.

```
-- — SQL Server / Synapse —————
-- Verify all expected target tables exist
SELECT
    e.table_name                                AS expected_table,
    CASE WHEN t.TABLE_NAME IS NULL THEN 'MISSING'
         ELSE 'EXISTS'
    END                                          AS existence_status,
    t.TABLE_TYPE
FROM (VALUES
    ('FACT_ORDERS'),
    ('DIM_CUSTOMERS'),
    ('DIM_PRODUCTS'),
    ('DIM_DATE'),
    ('AGG_DAILY_REVENUE')
) AS e(table_name)
LEFT JOIN INFORMATION_SCHEMA.TABLES t
    ON t.TABLE_NAME = e.table_name
   AND t.TABLE_SCHEMA = 'dwh';
-- Verify table has at least one index (warehouse readiness)
SELECT
    t.name            AS table_name,
    i.name            AS index_name,
    i.type_desc       AS index_type,
    i.is_primary_key,
    i.is_unique
FROM sys.tables t
JOIN sys.indexes i ON t.object_id = i.object_id
WHERE SCHEMA_NAME(t.schema_id) = 'dwh'
   AND i.type > 0          -- exclude heap
ORDER BY t.name, i.type_desc;
-- — Hive / SparkSQL —————
-- Verify table exists and is partitioned correctly
DESCRIBE FORMATTED dwh.fact_orders;
-- Show all tables in warehouse database
SHOW TABLES IN dwh;
-- Verify partition columns
SELECT table_name, partition_column, partition_type
FROM INFORMATION_SCHEMA.PARTITIONS
WHERE table_schema = 'dwh'
   AND table_name = 'fact_orders';
```

2. Source → Target Count Reconciliation

Purpose: End-to-end count reconciliation across the full pipeline —

source → staging → target — with per-partition and overall totals.

```
-- Full pipeline row count reconciliation
SELECT
    'source'            AS layer,
    COUNT(*)            AS record_count
FROM source_db.dbo.ORDERS
UNION ALL
SELECT 'staging',      COUNT(*) FROM staging.dbo.ORDERS_STAGE
```

```

UNION ALL
SELECT 'target',          COUNT(*) FROM dwh.dbo.FACT_ORDERS;
-- Gap analysis: where did rows drop?
WITH counts AS (
    SELECT
        (SELECT COUNT(*) FROM source_db.dbo.ORDERS)      AS src,
        (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE) AS stg,
        (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDERS)      AS tgt
    )
SELECT
    src,
    stg,
    tgt,
    src - stg  AS source_to_stage_gap,
    stg - tgt  AS stage_to_target_gap,
    src - tgt  AS end_to_end_gap,
    ROUND(100.0 * tgt / NULLIF(src, 0), 4) AS pct_loaded,
    CASE
        WHEN src = tgt THEN 'PASS - PERFECT MATCH'
        WHEN src = tgt + (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDERS_REJECTS)
            THEN 'PASS - REJECTS ACCOUNTED'
        ELSE
            'FAIL - UNACCOUNTED GAP'
    END AS reconciliation_status
FROM counts;
-- Partition-level reconciliation (by date)
SELECT
    CAST(s.ORDER_DATE AS DATE) AS partition_dt,
    COUNT(s.ORDER_ID)          AS source_count,
    COUNT(t.ORDER_KEY)         AS target_count,
    COUNT(s.ORDER_ID) -
    COUNT(t.ORDER_KEY)         AS gap,
    CASE WHEN COUNT(s.ORDER_ID) = COUNT(t.ORDER_KEY)
        THEN 'PASS' ELSE 'FAIL' END AS status
FROM source_db.dbo.ORDERS s
LEFT JOIN dwh.dbo.FACT_ORDERS t
    ON s.ORDER_ID = t.SOURCE_ORDER_ID
GROUP BY CAST(s.ORDER_DATE AS DATE)
HAVING COUNT(s.ORDER_ID) <> COUNT(t.ORDER_KEY)
ORDER BY partition_dt;

```

3. Column-to-Column Data Match

Purpose: Row-level verification that every column value in the target

exactly mirrors its source counterpart — detecting corruption, truncation, or silent transformation side-effects.

```

-- Row-level column comparison (sample 5000 rows)
SELECT
    s.ORDER_ID,
    -- Numeric columns
    CASE WHEN ABS(ISNULL(s.AMOUNT,0) - ISNULL(t.AMOUNT,0)) > 0.01
        THEN 'AMOUNT_MISMATCH' END AS amount_check,
    -- String columns
    CASE WHEN ISNULL(s.STATUS,'') <> ISNULL(t.STATUS,'')
        THEN 'STATUS_MISMATCH' END AS status_check,
    -- Date columns
    CASE WHEN CAST(s.ORDER_DATE AS DATE) <> CAST(t.ORDER_DATE AS DATE)
        THEN 'DATE_MISMATCH' END AS date_check
FROM (SELECT TOP 5000 * FROM source_db.dbo.ORDERS ORDER BY NEWID()) s
INNER JOIN dwh.dbo.FACT_ORDERS t
    ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE
    ABS(ISNULL(s.AMOUNT,0) - ISNULL(t.AMOUNT,0)) > 0.01
    OR ISNULL(s.STATUS,'') <> ISNULL(t.STATUS,'')
    OR CAST(s.ORDER_DATE AS DATE) <> CAST(t.ORDER_DATE AS DATE);
-- Column-level mismatch count summary
SELECT
    'AMOUNT'          AS column_name,

```

```

SUM(CASE WHEN ABS(ISNULL(s.AMOUNT,0) - ISNULL(t.AMOUNT,0)) > 0.01
      THEN 1 ELSE 0 END) AS mismatch_count,
COUNT(*)                AS total_compared
FROM source_db.dbo.ORDERS s
INNER JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
UNION ALL
SELECT 'STATUS',
      SUM(CASE WHEN ISNULL(s.STATUS,'') <> ISNULL(t.STATUS,'')
            THEN 1 ELSE 0 END),
COUNT(*)
FROM source_db.dbo.ORDERS s
INNER JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID;

```

4. Data Type Compatibility

Purpose: Ensure that data types in the target are compatible with the

source — no implicit truncation, lossy casting, or type mismatches that could corrupt analytics queries.

```

-- Compare source vs target data types from INFORMATION_SCHEMA
SELECT
  s.COLUMN_NAME,
  s.DATA_TYPE           AS source_type,
  s.CHARACTER_MAXIMUM_LENGTH AS source_max_len,
  s.NUMERIC_PRECISION   AS source_precision,
  s.NUMERIC_SCALE       AS source_scale,
  t.DATA_TYPE           AS target_type,
  t.CHARACTER_MAXIMUM_LENGTH AS target_max_len,
  t.NUMERIC_PRECISION   AS target_precision,
  t.NUMERIC_SCALE       AS target_scale,
  CASE
    WHEN t.COLUMN_NAME IS NULL THEN 'COLUMN MISSING IN TARGET'
    WHEN s.DATA_TYPE = t.DATA_TYPE THEN 'EXACT MATCH'
    WHEN s.DATA_TYPE IN ('int','bigint')
      AND t.DATA_TYPE IN ('int','bigint','decimal') THEN 'COMPATIBLE WIDENING'
    WHEN s.DATA_TYPE IN ('varchar','nvarchar')
      AND t.DATA_TYPE IN ('varchar','nvarchar')
      AND t.CHARACTER_MAXIMUM_LENGTH >= s.CHARACTER_MAXIMUM_LENGTH
      THEN 'COMPATIBLE WIDENING'
    WHEN s.DATA_TYPE IN ('varchar','nvarchar')
      AND t.DATA_TYPE IN ('varchar','nvarchar')
      AND t.CHARACTER_MAXIMUM_LENGTH < s.CHARACTER_MAXIMUM_LENGTH
      THEN 'TRUNCATION RISK'
    ELSE 'TYPE MISMATCH - REVIEW'
  END AS compatibility_status
FROM INFORMATION_SCHEMA.COLUMNS s
LEFT JOIN INFORMATION_SCHEMA.COLUMNS t
  ON t.TABLE_NAME = 'FACT_ORDERS'
  AND t.TABLE_SCHEMA = 'dwh'
  AND t.COLUMN_NAME = s.COLUMN_NAME
WHERE s.TABLE_NAME = 'ORDERS'
  AND s.TABLE_SCHEMA = 'dbo'
ORDER BY s.ORDINAL_POSITION;
-- Detect actual cast failures (values that don't fit target type)
SELECT ORDER_ID, AMOUNT
FROM source_db.dbo.ORDERS
WHERE TRY_CAST(AMOUNT AS DECIMAL(10,2)) IS NULL
  AND AMOUNT IS NOT NULL; -- values that would fail on insert

```

5. Precision & Scale Validation

Purpose: Validate that numeric columns in the target have sufficient

precision and scale to hold source values without rounding or overflow.

```

-- Verify target DECIMAL precision/scale is adequate for source data
SELECT
  ORDER_ID,
  AMOUNT,

```

```

CAST(AMOUNT AS DECIMAL(10,2)) AS cast_result,
CASE
    WHEN ABS(AMOUNT - CAST(AMOUNT AS DECIMAL(10,2))) > 0.001
    THEN 'PRECISION LOSS'
    ELSE 'OK'
END AS precision_check
FROM source_db.dbo.ORDERS
WHERE ABS(AMOUNT - CAST(AMOUNT AS DECIMAL(10,2))) > 0.001;
-- Find max digits needed for INTEGER part (to set correct precision)
SELECT
    MAX(LEN(CAST(FLOOR(ABS(AMOUNT)) AS VARCHAR))) AS max_integer_digits,
    MAX(LEN(
        SUBSTRING(CAST(AMOUNT AS VARCHAR),
            CHARINDEX('.', CAST(AMOUNT AS VARCHAR)) + 1, 100)
    )) AS max_decimal_digits
FROM source_db.dbo.ORDERS
WHERE AMOUNT IS NOT NULL;
-- Detect values that exceed target column precision/scale
-- Target: DECIMAL(10,2) → max integer digits = 8, scale = 2
SELECT ORDER_ID, AMOUNT,
CASE
    WHEN FLOOR(ABS(AMOUNT)) >= 100000000 THEN 'INTEGER OVERFLOW'
    WHEN ROUND(AMOUNT,2) <> AMOUNT THEN 'SCALE LOSS'
    ELSE 'OK'
END AS precision_status
FROM source_db.dbo.ORDERS
WHERE FLOOR(ABS(AMOUNT)) >= 100000000
OR ROUND(AMOUNT, 2) <> AMOUNT;

```

6. NULL / NOT NULL Enforcement

Purpose: Verify that NOT NULL constraints defined on the target warehouse

table are honoured — no NULLs slipping into mandatory columns, and NULLable columns still accept NULLs without coercion.

```

-- List all NOT NULL columns in target table
SELECT COLUMN_NAME, IS_NULLABLE, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'FACT_ORDERS'
    AND TABLE_SCHEMA = 'dwh'
    AND IS_NULLABLE = 'NO'
ORDER BY ORDINAL_POSITION;
-- Check for NULL violations in all NOT NULL columns
SELECT
    'ORDER_KEY' AS column_name,
    SUM(CASE WHEN ORDER_KEY IS NULL THEN 1 ELSE 0 END) AS null_count
FROM dwh.dbo.FACT_ORDERS
UNION ALL
SELECT 'SOURCE_ORDER_ID',
    SUM(CASE WHEN SOURCE_ORDER_ID IS NULL THEN 1 ELSE 0 END)
FROM dwh.dbo.FACT_ORDERS
UNION ALL
SELECT 'ORDER_DATE',
    SUM(CASE WHEN ORDER_DATE IS NULL THEN 1 ELSE 0 END)
FROM dwh.dbo.FACT_ORDERS
UNION ALL
SELECT 'CUSTOMER_KEY',
    SUM(CASE WHEN CUSTOMER_KEY IS NULL THEN 1 ELSE 0 END)
FROM dwh.dbo.FACT_ORDERS;
-- Confirm no row with NULL in a NOT NULL column was allowed through
SELECT COUNT(*) AS constraint_violations
FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_KEY IS NULL OR SOURCE_ORDER_ID IS NULL OR ORDER_DATE IS NULL;

```

7. Default Value Validation

Purpose: Confirm that DEFAULT constraints on warehouse columns fire

correctly when no value is supplied, and that explicit non-NULL values are never replaced by defaults.

```
-- Verify DEFAULT constraint definitions on target table
SELECT
    c.COLUMN_NAME,
    c.COLUMN_DEFAULT,
    c.IS_NULLABLE,
    c.DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS c
WHERE c.TABLE_NAME = 'FACT_ORDERS'
    AND c.TABLE_SCHEMA = 'dwh'
    AND c.COLUMN_DEFAULT IS NOT NULL
ORDER BY c.ORDINAL_POSITION;
-- Validate LOAD_TIMESTAMP default (should equal insert date, not NULL)
SELECT
    ORDER_KEY,
    LOAD_TIMESTAMP,
    CAST(GETDATE() AS DATE) AS today,
    CASE
        WHEN LOAD_TIMESTAMP IS NULL THEN 'DEFAULT NOT APPLIED'
        WHEN CAST(LOAD_TIMESTAMP AS DATE)
            < CAST(GETDATE() AS DATE) THEN 'STALE TIMESTAMP'
        ELSE 'PASS'
    END AS load_ts_status
FROM dwh.dbo.FACT_ORDERS
WHERE CAST(INSERT_DATE AS DATE) = CAST(GETDATE() AS DATE)
    AND (LOAD_TIMESTAMP IS NULL
        OR CAST(LOAD_TIMESTAMP AS DATE) < CAST(GETDATE() AS DATE));
-- Validate IS_ACTIVE defaults to 1 when not supplied
SELECT
    COUNT(*) AS rows_without_active_flag
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_ACTIVE IS NULL OR IS_ACTIVE NOT IN (0, 1);
```

8. Duplicate Records in Target

Purpose: Guarantee the warehouse/Hive table contains no duplicate rows

at the business key and surrogate key levels — a hard requirement for accurate analytics and BI joins.

```
-- PK / Surrogate key duplicates in fact table
SELECT ORDER_KEY, COUNT(*) AS dup_count
FROM dwh.dbo.FACT_ORDERS
GROUP BY ORDER_KEY
HAVING COUNT(*) > 1;
-- Business key duplicates (source order_id should be unique in target)
SELECT SOURCE_ORDER_ID, COUNT(*) AS dup_count
FROM dwh.dbo.FACT_ORDERS
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(*) > 1;
-- Full row duplicates
SELECT
    ORDER_KEY, CUSTOMER_KEY, PRODUCT_KEY,
    ORDER_DATE, AMOUNT, COUNT(*) AS full_dup_count
FROM dwh.dbo.FACT_ORDERS
GROUP BY ORDER_KEY, CUSTOMER_KEY, PRODUCT_KEY, ORDER_DATE, AMOUNT
HAVING COUNT(*) > 1;
-- Dimension table SCD duplicate check
-- (only one active record per natural key)
SELECT CUSTOMER_ID, COUNT(*) AS active_count
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 1
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;
```

9. Surrogate Key Generation

Purpose: Validate surrogate keys (SKs) are generated correctly — unique,

sequential or hash-based, non-NULL, no gaps in sequences, and consistent across SCD Type 2 dimensions.

```
-- Validate surrogate key properties
SELECT
    MIN(ORDER_KEY)                AS min_sk,
    MAX(ORDER_KEY)                AS max_sk,
    COUNT(*)                      AS total_rows,
    COUNT(DISTINCT ORDER_KEY)     AS distinct_sks,
    COUNT(*) - COUNT(DISTINCT ORDER_KEY) AS duplicate_sks,
    SUM(CASE WHEN ORDER_KEY IS NULL THEN 1
          ELSE 0 END)             AS null_sks,
    CASE
        WHEN COUNT(*) = COUNT(DISTINCT ORDER_KEY)
          AND SUM(CASE WHEN ORDER_KEY IS NULL THEN 1 ELSE 0 END) = 0
        THEN 'PASS' ELSE 'FAIL'
    END AS sk_validation
FROM dwh.dbo.FACT_ORDERS;
-- Sequence gap detection (integer SKs only)
WITH seq AS (
    SELECT ORDER_KEY,
           LAG(ORDER_KEY) OVER (ORDER BY ORDER_KEY) AS prev_key
    FROM dwh.dbo.FACT_ORDERS
)
SELECT ORDER_KEY, prev_key,
       ORDER_KEY - prev_key AS gap_size
FROM seq
WHERE ORDER_KEY - prev_key > 1;
-- Hash-based SK consistency check
-- (verify same source PK always maps to the same SK)
SELECT SOURCE_ORDER_ID, COUNT(DISTINCT ORDER_KEY) AS sk_count
FROM dwh.dbo.FACT_ORDERS
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(DISTINCT ORDER_KEY) > 1;    -- should return 0 rows
```

10. Fact-to-Dimension Relationship Validation

Purpose: Confirm every foreign key in the fact table resolves to a valid

dimension record — no orphaned facts, no broken star/snowflake schema joins that would silently exclude data from BI queries.

```
-- Validate FACT_ORDERS → DIM_CUSTOMERS join integrity
SELECT
    f.ORDER_KEY,
    f.CUSTOMER_KEY,
    CASE WHEN d.CUSTOMER_KEY IS NULL THEN 'ORPHAN - FAIL'
         ELSE 'VALID'
    END AS customer_fk_status
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.CUSTOMER_KEY IS NULL;    -- should return 0 rows
-- Validate all dimension FKs in one shot
SELECT 'DIM_CUSTOMERS' AS dim_table,
       COUNT(*)        AS orphan_facts
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.CUSTOMER_KEY IS NULL
UNION ALL
SELECT 'DIM_PRODUCTS', COUNT(*)
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_PRODUCTS p ON f.PRODUCT_KEY = p.PRODUCT_KEY
WHERE p.PRODUCT_KEY IS NULL
UNION ALL
SELECT 'DIM_DATE', COUNT(*)
```



```

FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_DATE dt      ON f.DATE_KEY      = dt.DATE_KEY
WHERE dt.DATE_KEY      IS NULL;
-- % of facts with valid dim keys (analytical completeness)
SELECT
    COUNT(*)                                AS total_facts,
    SUM(CASE WHEN d.CUSTOMER_KEY IS NOT NULL
              AND p.PRODUCT_KEY  IS NOT NULL
              AND dt.DATE_KEY    IS NOT NULL
              THEN 1 ELSE 0 END)            AS fully_resolved_facts,
    ROUND(100.0 *
          SUM(CASE WHEN d.CUSTOMER_KEY IS NOT NULL
                    AND p.PRODUCT_KEY  IS NOT NULL
                    AND dt.DATE_KEY    IS NOT NULL
                    THEN 1 ELSE 0 END)
          / NULLIF(COUNT(*), 0), 2)         AS resolution_pct
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
LEFT JOIN dwh.dbo.DIM_PRODUCTS  p ON f.PRODUCT_KEY  = p.PRODUCT_KEY
LEFT JOIN dwh.dbo.DIM_DATE      dt ON f.DATE_KEY    = dt.DATE_KEY;

```

Dimension Table Testing

ETL Dimension Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Dimension\_Validations\_Python.md`

 Dimension quality = reporting accuracy. A broken dimension silently corrupts every metric joined to it.

1. SCD Type 1 Validation (Overwrite)

Purpose: In SCD Type 1, old values are overwritten in-place — no history

is kept. Validate that updates are applied correctly, only one row exists per natural key, and no stale/pre-update values survive in the target.

```
-- — Rule: exactly one row per natural key —————
SELECT
    CUSTOMER_ID,
    COUNT(*) AS row_count
FROM dwh.dbo.DIM_CUSTOMERS
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;                                -- must return 0 rows
-- — Rule: current value matches latest source value —————
SELECT
    s.CUSTOMER_ID,
    s.EMAIL                AS source_email,
    d.EMAIL                AS dim_email,
    s.PHONE                AS source_phone,
    d.PHONE                AS dim_phone,
    CASE
        WHEN s.EMAIL = d.EMAIL
            AND s.PHONE = d.PHONE THEN 'PASS - UP TO DATE'
        ELSE 'FAIL - STALE VALUE'
    END AS scd1_status
FROM source_db.dbo.CUSTOMERS s
JOIN dwh.dbo.DIM_CUSTOMERS d ON s.CUSTOMER_ID = d.CUSTOMER_ID
WHERE s.EMAIL <> d.EMAIL
    OR s.PHONE <> d.PHONE;
-- — Rule: UPDATE_TIMESTAMP reflects the latest change —————
SELECT
    d.CUSTOMER_ID,
    d.EMAIL,
    d.UPDATE_TIMESTAMP,
    s.LAST_MODIFIED_DT     AS source_last_modified,
    CASE
        WHEN d.UPDATE_TIMESTAMP >= s.LAST_MODIFIED_DT THEN 'PASS'
        ELSE 'FAIL - TIMESTAMP NOT REFRESHED'
    END AS ts_status
FROM source_db.dbo.CUSTOMERS s
JOIN dwh.dbo.DIM_CUSTOMERS d ON s.CUSTOMER_ID = d.CUSTOMER_ID
WHERE d.UPDATE_TIMESTAMP < s.LAST_MODIFIED_DT;
-- — Summary —————
SELECT
    (SELECT COUNT(*) FROM dwh.dbo.DIM_CUSTOMERS) AS total_rows,
    (SELECT COUNT(DISTINCT CUSTOMER_ID) FROM dwh.dbo.DIM_CUSTOMERS) AS unique_keys,
    (SELECT COUNT(*) FROM dwh.dbo.DIM_CUSTOMERS d
     JOIN source_db.dbo.CUSTOMERS s ON d.CUSTOMER_ID = s.CUSTOMER_ID
     WHERE d.EMAIL <> s.EMAIL OR d.PHONE <> s.PHONE) AS stale_rows,
    CASE WHEN
        (SELECT COUNT(*) FROM dwh.dbo.DIM_CUSTOMERS) =
        (SELECT COUNT(DISTINCT CUSTOMER_ID) FROM dwh.dbo.DIM_CUSTOMERS)
    THEN 'PASS' ELSE 'FAIL' END AS uniqueness_check;
```

2. SCD Type 2 Validation (Add New Row)

Purpose: In SCD Type 2, every change generates a new row with new

effective dates. Validate that history rows are added (not overwritten),

date ranges are contiguous and non-overlapping, and exactly one row is active per natural key.

```
-- — Rule: exactly one CURRENT row per natural key —————
SELECT
    CUSTOMER_ID,
    COUNT(*) AS current_row_count
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 1
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;           -- must return 0 rows
-- — Rule: no gaps or overlaps in date ranges —————
WITH ordered AS (
    SELECT
        CUSTOMER_ID,
        EFFECTIVE_START_DATE,
        EFFECTIVE_END_DATE,
        LEAD(EFFECTIVE_START_DATE) OVER (
            PARTITION BY CUSTOMER_ID
            ORDER BY EFFECTIVE_START_DATE
        ) AS next_start
    FROM dwh.dbo.DIM_CUSTOMERS
)
SELECT
    CUSTOMER_ID,
    EFFECTIVE_START_DATE,
    EFFECTIVE_END_DATE,
    next_start,
    CASE
        -- Gap: end date is not contiguous with next start
        WHEN DATEADD(DAY, 1, EFFECTIVE_END_DATE) < next_start
        THEN 'GAP DETECTED'
        -- Overlap: end date goes past next start
        WHEN EFFECTIVE_END_DATE >= next_start
        THEN 'OVERLAP DETECTED'
        ELSE 'OK'
    END AS date_continuity_status
FROM ordered
WHERE next_start IS NOT NULL
    AND (DATEADD(DAY, 1, EFFECTIVE_END_DATE) <> next_start
        OR EFFECTIVE_END_DATE >= next_start);
-- — Rule: latest source value matches current (IS_CURRENT=1) row
SELECT
    s.CUSTOMER_ID,
    s.EMAIL AS source_email,
    d.EMAIL AS current_dim_email,
    CASE WHEN s.EMAIL = d.EMAIL THEN 'PASS'
        ELSE 'FAIL - CURRENT ROW STALE' END AS currency_check
FROM source_db.dbo.CUSTOMERS s
JOIN dwh.dbo.DIM_CUSTOMERS d
    ON s.CUSTOMER_ID = d.CUSTOMER_ID
    AND d.IS_CURRENT = 1
WHERE s.EMAIL <> d.EMAIL;
-- — History count: each changed attribute creates a new row —
SELECT
    CUSTOMER_ID,
    COUNT(*) AS version_count,
    MIN(EFFECTIVE_START_DATE) AS first_seen,
    MAX(EFFECTIVE_START_DATE) AS last_changed
FROM dwh.dbo.DIM_CUSTOMERS
GROUP BY CUSTOMER_ID
ORDER BY version_count DESC;
```

3. SCD Type 3 Validation (Previous Value Column)

Purpose: SCD Type 3 tracks exactly one prior value in a dedicated

"previous" column. Validate that the PREV column holds the immediately

preceding value, and that only one level of history is stored.

```
-- — Rule: PREV_CITY = the value that existed before the current CITY
SELECT
    d.CUSTOMER_ID,
    d.CITY                AS current_city,
    d.PREV_CITY           AS stored_prev_city,
    h.CITY                AS actual_prev_city,    -- from change log
    CASE
        WHEN h.CITY IS NULL AND d.PREV_CITY IS NULL THEN 'PASS - NO PREV (FIRST RECORD)'
        WHEN d.PREV_CITY = h.CITY                      THEN 'PASS'
        ELSE                                           'FAIL - WRONG PREV VALUE'
    END AS scd3_status
FROM dwh.dbo.DIM_CUSTOMERS d
LEFT JOIN etl_control.dbo.DIM_CHANGE_LOG h
    ON h.CUSTOMER_ID = d.CUSTOMER_ID
   AND h.COLUMN_NAME = 'CITY'
   AND h.CHANGE_DATE = (
        SELECT MAX(CHANGE_DATE)
        FROM etl_control.dbo.DIM_CHANGE_LOG
        WHERE CUSTOMER_ID = d.CUSTOMER_ID
          AND COLUMN_NAME = 'CITY'
          AND CHANGE_DATE < d.CITY_CHANGE_DATE
    );
-- — Rule: PREV columns are NULL for first-time records ———
SELECT CUSTOMER_ID, CITY, PREV_CITY
FROM dwh.dbo.DIM_CUSTOMERS
WHERE PREV_CITY IS NOT NULL
   AND CUSTOMER_ID NOT IN (
        SELECT CUSTOMER_ID FROM etl_control.dbo.DIM_CHANGE_LOG
        WHERE COLUMN_NAME = 'CITY'
    );
-- — Rule: only ONE level of history (no chaining) ———
-- SCD3 never stores prev-prev; validate no "PREV_PREV" pattern
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'DIM_CUSTOMERS'
   AND TABLE_SCHEMA = 'dwh'
   AND COLUMN_NAME LIKE 'PREV_PREV_%';    -- should return 0 rows
```

4. Effective Start & End Date Checks

Purpose: Validate that **EFFECTIVE_START_DATE** and **EFFECTIVE_END_DATE** are

populated correctly, ordered logically, cover the full timeline, and use the correct sentinel end-date (e.g. 9999-12-31) for active records.

```
-- — Rule: END_DATE >= START_DATE (no inverted ranges) ———
SELECT CUSTOMER_KEY, CUSTOMER_ID,
    EFFECTIVE_START_DATE, EFFECTIVE_END_DATE
FROM dwh.dbo.DIM_CUSTOMERS
WHERE EFFECTIVE_END_DATE < EFFECTIVE_START_DATE;    -- must return 0 rows
-- — Rule: Active rows (IS_CURRENT=1) have sentinel end date —
SELECT CUSTOMER_KEY, CUSTOMER_ID,
    EFFECTIVE_START_DATE, EFFECTIVE_END_DATE, IS_CURRENT
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 1
   AND EFFECTIVE_END_DATE <> '9999-12-31';    -- must return 0 rows
-- — Rule: Expired rows (IS_CURRENT=0) don't use sentinel date —
SELECT CUSTOMER_KEY, CUSTOMER_ID,
    EFFECTIVE_START_DATE, EFFECTIVE_END_DATE
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 0
   AND EFFECTIVE_END_DATE = '9999-12-31';    -- must return 0 rows
-- — Rule: No NULL dates ———
SELECT
    SUM(CASE WHEN EFFECTIVE_START_DATE IS NULL THEN 1 ELSE 0 END) AS null_start,
    SUM(CASE WHEN EFFECTIVE_END_DATE IS NULL THEN 1 ELSE 0 END) AS null_end
FROM dwh.dbo.DIM_CUSTOMERS;    -- both must be 0
-- — Rule: START_DATE cannot be in the future ———
```

```

SELECT CUSTOMER_KEY, CUSTOMER_ID, EFFECTIVE_START_DATE
FROM dwh.dbo.DIM_CUSTOMERS
WHERE EFFECTIVE_START_DATE > CAST(GETDATE() AS DATE);
-- — Timeline coverage per natural key —————
WITH timeline AS (
    SELECT
        CUSTOMER_ID,
        MIN(EFFECTIVE_START_DATE) AS first_effective,
        MAX(EFFECTIVE_END_DATE)   AS last_effective,
        COUNT(*)                  AS version_count
    FROM dwh.dbo.DIM_CUSTOMERS
    GROUP BY CUSTOMER_ID
)
SELECT *,
    DATEDIFF(DAY, first_effective, last_effective) AS covered_days
FROM timeline
ORDER BY version_count DESC;

```

5. Current Flag Validation

Purpose: Verify the IS_CURRENT (or IS_ACTIVE) flag is set correctly —

exactly one active row per natural key, consistent with EFFECTIVE_END_DATE, and no NULL flags.

```

-- — Rule: IS_CURRENT has no NULLs —————
SELECT COUNT(*) AS null_flags
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT IS NULL;          -- must be 0
-- — Rule: IS_CURRENT only contains 0 or 1 (no rogue values) —
SELECT DISTINCT IS_CURRENT
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT NOT IN (0, 1);    -- must return 0 rows
-- — Rule: exactly ONE IS_CURRENT=1 per natural key —————
SELECT CUSTOMER_ID, COUNT(*) AS active_count
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 1
GROUP BY CUSTOMER_ID
HAVING COUNT(*) <> 1;              -- must return 0 rows
-- — Rule: IS_CURRENT=1 ⇔ EFFECTIVE_END_DATE='9999-12-31' ———
SELECT CUSTOMER_KEY, CUSTOMER_ID, IS_CURRENT, EFFECTIVE_END_DATE,
    CASE
        WHEN IS_CURRENT = 1 AND EFFECTIVE_END_DATE = '9999-12-31' THEN 'CONSISTENT'
        WHEN IS_CURRENT = 0 AND EFFECTIVE_END_DATE <> '9999-12-31' THEN 'CONSISTENT'
        ELSE 'INCONSISTENT - FLAG/DATE MISMATCH'
    END AS flag_date_consistency
FROM dwh.dbo.DIM_CUSTOMERS
WHERE NOT (
    (IS_CURRENT = 1 AND EFFECTIVE_END_DATE = '9999-12-31') OR
    (IS_CURRENT = 0 AND EFFECTIVE_END_DATE <> '9999-12-31')
);
-- — Rule: every natural key has at least one IS_CURRENT=1 row —
SELECT CUSTOMER_ID
FROM dwh.dbo.DIM_CUSTOMERS
GROUP BY CUSTOMER_ID
HAVING SUM(CASE WHEN IS_CURRENT = 1 THEN 1 ELSE 0 END) = 0; -- orphaned keys

```

6. History Preservation

Purpose: Confirm that previously loaded historical rows are not modified,

deleted, or overwritten during subsequent ETL runs — a critical guarantee for audit trails, regulatory compliance, and trend analysis.

```

-- Snapshot comparison: rows loaded in previous run must be identical now
-- (Uses a pre-run snapshot table created before each load)
SELECT
    prev.CUSTOMER_KEY,
    prev.CUSTOMER_ID,

```

```

prev.CITY          AS prev_city,
curr.CITY          AS curr_city,
prev.EFFECTIVE_START_DATE,
CASE
    WHEN curr.CUSTOMER_KEY IS NULL      THEN 'ROW DELETED - FAIL'
    WHEN prev.CITY <> curr.CITY          THEN 'HISTORY MODIFIED - FAIL'
    WHEN prev.EFFECTIVE_START_DATE
        <> curr.EFFECTIVE_START_DATE    THEN 'DATE MODIFIED - FAIL'
    ELSE                                'PASS'
END AS history_integrity_status
FROM etl_control.dbo.DIM_CUSTOMERS_SNAPSHOT prev -- snapshot from last run
LEFT JOIN dwh.dbo.DIM_CUSTOMERS curr
    ON prev.CUSTOMER_KEY = curr.CUSTOMER_KEY
WHERE prev.IS_CURRENT = 0 -- only check historical rows
AND (curr.CUSTOMER_KEY IS NULL
    OR prev.CITY <> curr.CITY
    OR prev.EFFECTIVE_START_DATE <> curr.EFFECTIVE_START_DATE);
-- Count history rows: should never decrease between runs
SELECT
    (SELECT COUNT(*) FROM etl_control.dbo.DIM_CUSTOMERS_SNAPSHOT
     WHERE IS_CURRENT = 0) AS prev_history_rows,
    (SELECT COUNT(*) FROM dwh.dbo.DIM_CUSTOMERS
     WHERE IS_CURRENT = 0) AS curr_history_rows,
CASE
    WHEN (SELECT COUNT(*) FROM dwh.dbo.DIM_CUSTOMERS WHERE IS_CURRENT=0) >=
        (SELECT COUNT(*) FROM etl_control.dbo.DIM_CUSTOMERS_SNAPSHOT WHERE IS_CURRENT=0)
    THEN 'PASS - HISTORY GREW OR STABLE'
    ELSE 'FAIL - HISTORY ROWS DECREASED'
END AS history_preservation_status;

```

7. Late Arriving Dimension Handling

Purpose: When a dimension record arrives after fact records that reference

it have already been loaded, validate that the retroactive dimension insert correctly back-fills the fact table, and that orphaned facts are resolved.

```

-- Identify fact rows that still reference unknown/dummy dimension keys
-- (orphaned facts written before the dimension record arrived)
SELECT
    f.ORDER_KEY,
    f.CUSTOMER_KEY,
    d.CUSTOMER_KEY          AS dim_key_resolved,
CASE
    WHEN d.CUSTOMER_KEY IS NOT NULL THEN 'RESOLVED'
    WHEN f.CUSTOMER_KEY = -1        THEN 'STILL UNKNOWN MEMBER'
    ELSE                            'ORPHAN - DIM NOT YET ARRIVED'
END AS late_dim_status
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.CUSTOMER_KEY IS NULL;
-- Validate back-fill: after late dim arrives, facts should be updated
SELECT
    f.ORDER_KEY,
    f.CUSTOMER_KEY,
    f.ORDER_DATE,
    d.EFFECTIVE_START_DATE,
    d.EFFECTIVE_END_DATE,
CASE
    WHEN f.ORDER_DATE BETWEEN d.EFFECTIVE_START_DATE
        AND d.EFFECTIVE_END_DATE THEN 'CORRECT VERSION'
    ELSE 'WRONG DIM VERSION ASSIGNED'
END AS late_dim_version_check
FROM dwh.dbo.FACT_ORDERS f
JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.EFFECTIVE_START_DATE > f.ORDER_DATE; -- dim row effective after fact
-- Summary: how many orphaned facts resolved after late dim load
SELECT
    COUNT(*) AS total_late_resolved,

```

```

    MIN(f.ORDER_DATE) AS earliest_late_fact,
    MAX(f.ORDER_DATE) AS latest_late_fact
FROM dwh.dbo.FACT_ORDERS f
JOIN dwh.dbo.DIM_CUSTOMERS d
    ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
    AND d.EFFECTIVE_START_DATE <= f.ORDER_DATE
    AND d.EFFECTIVE_END_DATE >= f.ORDER_DATE
WHERE f.CUSTOMER_KEY = -1; -- was unknown before back-fill

```

8. Junk Dimension Validation

Purpose: A junk dimension combines low-cardinality flags and indicators

(e.g., IS_PROMO, IS_ONLINE, PAYMENT_TYPE) into a single dimension to reduce fact table width. Validate that all valid combinations exist, no duplicates appear, and the fact table FK correctly maps to the right combination.

```

-- — Rule: all combinations in junk dim are unique —————
SELECT
    IS_PROMO, IS_ONLINE, IS_GIFT, PAYMENT_TYPE,
    COUNT(*) AS combo_count
FROM dwh.dbo.DIM_JUNK_ORDER_FLAGS
GROUP BY IS_PROMO, IS_ONLINE, IS_GIFT, PAYMENT_TYPE
HAVING COUNT(*) > 1; -- must return 0 rows
-- — Rule: fact FK resolves to correct combination —————
SELECT
    f.ORDER_KEY,
    f.ORDER_FLAGS_KEY,
    f.IS_PROMO AS fact_is_promo,
    j.IS_PROMO AS junk_is_promo,
    f.PAYMENT_TYPE AS fact_payment_type,
    j.PAYMENT_TYPE AS junk_payment_type,
    CASE
        WHEN f.IS_PROMO = j.IS_PROMO
            AND f.PAYMENT_TYPE = j.PAYMENT_TYPE THEN 'PASS'
        ELSE 'FAIL - WRONG JUNK COMBINATION'
    END AS junk_match_status
FROM dwh.dbo.FACT_ORDERS f
JOIN dwh.dbo.DIM_JUNK_ORDER_FLAGS j ON f.ORDER_FLAGS_KEY = j.ORDER_FLAGS_KEY
WHERE f.IS_PROMO <> j.IS_PROMO
    OR f.PAYMENT_TYPE <> j.PAYMENT_TYPE;
-- — Rule: validate expected combination coverage —————
-- All 2^3 = 8 combinations for 3 boolean flags must exist
SELECT COUNT(DISTINCT CONCAT(IS_PROMO, '-', IS_ONLINE, '-', IS_GIFT)) AS distinct_combos
FROM dwh.dbo.DIM_JUNK_ORDER_FLAGS;
-- — Rule: no fact FK references an undefined junk key —————
SELECT f.ORDER_KEY, f.ORDER_FLAGS_KEY
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_JUNK_ORDER_FLAGS j ON f.ORDER_FLAGS_KEY = j.ORDER_FLAGS_KEY
WHERE j.ORDER_FLAGS_KEY IS NULL; -- must return 0 rows

```

Fact Table Testing

ETL Fact Table Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Fact\_Table\_Validations\_Python.md`

 Facts drive every KPI, dashboard, and business decision — one bad row silently corrupts them all.

1. Grain Validation

Purpose: The grain is the single most important design decision in a fact

table. It defines what one row represents. Validate that the table is at exactly the declared grain — no finer, no coarser — and that every row is uniquely identifiable at that grain.

```
-- — Declared grain: one row per (ORDER_ID, PRODUCT_ID, SHIP_DATE) —
SELECT
    SOURCE_ORDER_ID,
    PRODUCT_KEY,
    SHIP_DATE_KEY,
    COUNT(*) AS row_count
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY SOURCE_ORDER_ID, PRODUCT_KEY, SHIP_DATE_KEY
HAVING COUNT(*) > 1; -- must return 0 rows
-- — Verify total row count is consistent with grain —————
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT CONCAT(
        CAST(SOURCE_ORDER_ID AS VARCHAR), '|',
        CAST(PRODUCT_KEY AS VARCHAR), '|',
        CAST(SHIP_DATE_KEY AS VARCHAR)
    )) AS distinct_grain_combos,
    COUNT(*) -
    COUNT(DISTINCT CONCAT(
        CAST(SOURCE_ORDER_ID AS VARCHAR), '|',
        CAST(PRODUCT_KEY AS VARCHAR), '|',
        CAST(SHIP_DATE_KEY AS VARCHAR)
    )) AS grain_violations,
    CASE
        WHEN COUNT(*) =
            COUNT(DISTINCT CONCAT(
                CAST(SOURCE_ORDER_ID AS VARCHAR), '|',
                CAST(PRODUCT_KEY AS VARCHAR), '|',
                CAST(SHIP_DATE_KEY AS VARCHAR)))
        THEN 'PASS' ELSE 'FAIL'
    END AS grain_status
FROM dwh.dbo.FACT_ORDER_LINES;
-- — Detect accidental over-granularity (fan-out from bad join) —
-- Compare fact row count vs expected from source
SELECT
    (SELECT COUNT(*) FROM source_db.dbo.ORDER_LINES) AS src_count,
    (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDER_LINES) AS fact_count,
    (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDER_LINES)
    / NULLIF(
        (SELECT COUNT(*) FROM source_db.dbo.ORDER_LINES), 0
    ) AS fanout_ratio,
    CASE
        WHEN (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDER_LINES) =
            (SELECT COUNT(*) FROM source_db.dbo.ORDER_LINES) THEN 'PASS'
        WHEN (SELECT COUNT(*) FROM dwh.dbo.FACT_ORDER_LINES) >
            (SELECT COUNT(*) FROM source_db.dbo.ORDER_LINES) THEN 'FAN-OUT - FAIL'
        ELSE 'UNDER-LOADED - FAIL'
    END AS load_status;
-- — Coarse grain check: rows that SHOULD be split but aren't —
-- Example: an order line should never aggregate two products
SELECT ORDER_LINE_KEY,
```



```

COUNT(DISTINCT PRODUCT_KEY) AS products_per_line
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY ORDER_LINE_KEY
HAVING COUNT(DISTINCT PRODUCT_KEY) > 1;  -- must return 0 rows

```

2. Foreign Key Validation

Purpose: Every FK in the fact table must resolve to a valid, existing

dimension row. Orphaned FKs cause silent null-joins in BI tools, making measures vanish from reports without any error message.

```

-- — Validate all dimension FKs in one comprehensive query ———
SELECT
    'DIM_CUSTOMERS'      AS dimension,
    'CUSTOMER_KEY'       AS fk_column,
    COUNT(*)             AS orphan_count
FROM dwh.dbo.FACT_ORDER_LINES f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.CUSTOMER_KEY IS NULL
    AND f.CUSTOMER_KEY <> -1          -- exclude intentional unknown member
UNION ALL SELECT 'DIM_PRODUCTS', 'PRODUCT_KEY', COUNT(*)
FROM dwh.dbo.FACT_ORDER_LINES f
LEFT JOIN dwh.dbo.DIM_PRODUCTS p ON f.PRODUCT_KEY = p.PRODUCT_KEY
WHERE p.PRODUCT_KEY IS NULL AND f.PRODUCT_KEY <> -1
UNION ALL SELECT 'DIM_DATE', 'ORDER_DATE_KEY', COUNT(*)
FROM dwh.dbo.FACT_ORDER_LINES f
LEFT JOIN dwh.dbo.DIM_DATE dt ON f.ORDER_DATE_KEY = dt.DATE_KEY
WHERE dt.DATE_KEY IS NULL AND f.ORDER_DATE_KEY <> -1
UNION ALL SELECT 'DIM_STORES', 'STORE_KEY', COUNT(*)
FROM dwh.dbo.FACT_ORDER_LINES f
LEFT JOIN dwh.dbo.DIM_STORES s ON f.STORE_KEY = s.STORE_KEY
WHERE s.STORE_KEY IS NULL AND f.STORE_KEY <> -1;
-- — FK null/zero/unknown-member breakdown —————
SELECT
    SUM(CASE WHEN CUSTOMER_KEY IS NULL THEN 1 ELSE 0 END) AS null_fk,
    SUM(CASE WHEN CUSTOMER_KEY = 0 THEN 1 ELSE 0 END) AS zero_fk,
    SUM(CASE WHEN CUSTOMER_KEY = -1 THEN 1 ELSE 0 END) AS unknown_member,
    SUM(CASE WHEN CUSTOMER_KEY > 0 THEN 1 ELSE 0 END) AS valid_fk,
    COUNT(*) AS total_rows
FROM dwh.dbo.FACT_ORDER_LINES;
-- — % of facts with all FKs resolved (overall completeness) —
SELECT
    ROUND(100.0 *
        SUM(CASE WHEN d.CUSTOMER_KEY IS NOT NULL
                    AND p.PRODUCT_KEY IS NOT NULL
                    AND dt.DATE_KEY IS NOT NULL
                    THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 4) AS pct_fully_resolved,
    COUNT(*) AS total_rows
FROM dwh.dbo.FACT_ORDER_LINES f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
LEFT JOIN dwh.dbo.DIM_PRODUCTS p ON f.PRODUCT_KEY = p.PRODUCT_KEY
LEFT JOIN dwh.dbo.DIM_DATE dt ON f.ORDER_DATE_KEY = dt.DATE_KEY;

```

3. Fact-to-Dimension Join Accuracy

Purpose: Validate that the FK in the fact table joins to the *correct*

dimension row and version — particularly for SCD Type 2 dimensions where the wrong version produces historically inaccurate attributes in reports.

```

-- — Validate correct SCD2 version joined for each fact ———
SELECT
    f.ORDER_LINE_KEY,
    f.CUSTOMER_KEY,
    f.ORDER_DATE_KEY,
    d.EFFECTIVE_START_DATE,
    d.EFFECTIVE_END_DATE,

```

```

    d.CITY                AS joined_city,
CASE
    WHEN f.ORDER_DATE BETWEEN d.EFFECTIVE_START_DATE
                                AND d.EFFECTIVE_END_DATE THEN 'CORRECT VERSION'
    ELSE                        'WRONG VERSION - FAIL'
END AS version_accuracy
FROM dwh.dbo.FACT_ORDER_LINES f
JOIN dwh.dbo.DIM_CUSTOMERS    d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE f.ORDER_DATE NOT BETWEEN d.EFFECTIVE_START_DATE
                                AND d.EFFECTIVE_END_DATE;      -- must return 0 rows
-- — Validate joined dimension attributes make business sense —
-- e.g. order date must be within the customer's active period
SELECT
    f.ORDER_LINE_KEY,
    f.ORDER_DATE,
    d.CREATE_DATE        AS customer_created_date,
CASE
    WHEN f.ORDER_DATE < d.CREATE_DATE THEN 'FAIL - ORDER BEFORE CUSTOMER EXISTED'
    ELSE 'PASS'
END AS temporal_sanity
FROM dwh.dbo.FACT_ORDER_LINES f
JOIN dwh.dbo.DIM_CUSTOMERS    d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
                                AND d.IS_CURRENT    = 1
WHERE f.ORDER_DATE < d.CREATE_DATE;
-- — Verify products in fact existed at time of order —————
SELECT
    f.ORDER_LINE_KEY,
    f.ORDER_DATE,
    p.PRODUCT_LAUNCH_DATE,
CASE
    WHEN f.ORDER_DATE < p.PRODUCT_LAUNCH_DATE
    THEN 'FAIL - ORDER BEFORE PRODUCT LAUNCH'
    ELSE 'PASS'
END AS product_temporal_sanity
FROM dwh.dbo.FACT_ORDER_LINES f
JOIN dwh.dbo.DIM_PRODUCTS    p ON f.PRODUCT_KEY = p.PRODUCT_KEY
WHERE f.ORDER_DATE < p.PRODUCT_LAUNCH_DATE;
-- — Attribute distribution after join (sanity check) —————
SELECT
    d.COUNTRY,
    COUNT(*)                AS order_line_count,
    SUM(f.SALE_AMOUNT)      AS total_sales
FROM dwh.dbo.FACT_ORDER_LINES f
JOIN dwh.dbo.DIM_CUSTOMERS    d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
GROUP BY d.COUNTRY
ORDER BY total_sales DESC;

```

4. Measure Calculation Accuracy

Purpose: Verify every numeric measure (SALE_AMOUNT, NET_REVENUE,

DISCOUNT_AMOUNT, TAX_AMOUNT) is calculated correctly, is within valid business ranges, and matches independently recalculated values from source.

```

-- — Rule: NET_REVENUE = GROSS_AMOUNT - DISCOUNT - TAX —————
SELECT
    ORDER_LINE_KEY,
    GROSS_AMOUNT,
    DISCOUNT_AMOUNT,
    TAX_AMOUNT,
    NET_REVENUE                AS stored_net,
    GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT AS expected_net,
    ABS(NET_REVENUE -
        (GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT)) AS diff,
CASE
    WHEN ABS(NET_REVENUE -
        (GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT)) < 0.01
    THEN 'PASS' ELSE 'FAIL'
END AS calc_status

```

```

FROM dwh.dbo.FACT_ORDER_LINES
WHERE ABS(NET_REVENUE -
  (GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT)) >= 0.01;
-- — Business range validation —————
SELECT
  SUM(CASE WHEN SALE_AMOUNT < 0 THEN 1 ELSE 0 END) AS negative_sales,
  SUM(CASE WHEN SALE_AMOUNT > 1000000 THEN 1 ELSE 0 END) AS huge_sales,
  SUM(CASE WHEN DISCOUNT_AMOUNT < 0 THEN 1 ELSE 0 END) AS negative_disc,
  SUM(CASE WHEN DISCOUNT_AMOUNT > SALE_AMOUNT THEN 1 ELSE 0 END) AS disc_exceeds_sale,
  SUM(CASE WHEN TAX_RATE NOT BETWEEN 0 AND 1 THEN 1 ELSE 0 END) AS invalid_tax_rate,
  SUM(CASE WHEN QUANTITY <= 0 THEN 1 ELSE 0 END) AS non_positive_qty,
  COUNT(*) AS total_rows
FROM dwh.dbo.FACT_ORDER_LINES;
-- — Reconcile total measures: source vs fact —————
SELECT
  'GROSS_AMOUNT' AS measure,
  SUM(s.GROSS_AMOUNT) AS source_total,
  SUM(f.GROSS_AMOUNT) AS fact_total,
  ABS(SUM(s.GROSS_AMOUNT) - SUM(f.GROSS_AMOUNT)) AS abs_diff,
  CASE WHEN ABS(SUM(s.GROSS_AMOUNT) - SUM(f.GROSS_AMOUNT)) < 1.0
    THEN 'PASS' ELSE 'FAIL' END AS status
FROM source_db.dbo.ORDER_LINES s
JOIN dwh.dbo.FACT_ORDER_LINES f ON s.ORDER_LINE_ID = f.SOURCE_ORDER_LINE_ID
UNION ALL
SELECT 'NET_REVENUE',
  SUM(s.GROSS_AMOUNT - s.DISCOUNT - s.TAX),
  SUM(f.NET_REVENUE),
  ABS(SUM(s.GROSS_AMOUNT - s.DISCOUNT - s.TAX) - SUM(f.NET_REVENUE)),
  CASE WHEN ABS(SUM(s.GROSS_AMOUNT - s.DISCOUNT - s.TAX)
    - SUM(f.NET_REVENUE)) < 1.0 THEN 'PASS' ELSE 'FAIL' END
FROM source_db.dbo.ORDER_LINES s
JOIN dwh.dbo.FACT_ORDER_LINES f ON s.ORDER_LINE_ID = f.SOURCE_ORDER_LINE_ID;

```

5. Additive / Semi-Additive Measures

Purpose: Additive measures (e.g. SALE_AMOUNT) can be summed across all

dimensions. Semi-additive measures (e.g. ACCOUNT_BALANCE, INVENTORY_QTY)

can only be summed across some dimensions — typically not time. Validate

that each measure type produces correct aggregations.

```

-- — ADDITIVE: SUM across all dimensions must equal grand total —
-- SALE_AMOUNT should sum correctly by day, by region, and overall
SELECT
  'BY_DATE' AS dimension,
  SUM(SALE_AMOUNT) AS partial_sum
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY ORDER_DATE_KEY
-- Running total of all BY_DATE sums should equal grand total below
;
SELECT SUM(SALE_AMOUNT) AS grand_total
FROM dwh.dbo.FACT_ORDER_LINES;
-- Verify: SUM of daily sums = grand total (additive property)
SELECT
  SUM(daily_sum) AS sum_of_daily_totals,
  (SELECT SUM(SALE_AMOUNT)
   FROM dwh.dbo.FACT_ORDER_LINES) AS grand_total,
  ABS(SUM(daily_sum) -
    (SELECT SUM(SALE_AMOUNT)
     FROM dwh.dbo.FACT_ORDER_LINES)) AS diff,
  CASE WHEN ABS(SUM(daily_sum) -
    (SELECT SUM(SALE_AMOUNT)
     FROM dwh.dbo.FACT_ORDER_LINES)) < 0.01
    THEN 'ADDITIVE - PASS' ELSE 'NON-ADDITIVE - FAIL' END AS additive_check
FROM (
  SELECT ORDER_DATE_KEY, SUM(SALE_AMOUNT) AS daily_sum
  FROM dwh.dbo.FACT_ORDER_LINES
  GROUP BY ORDER_DATE_KEY
) daily;

```

```
-- — SEMI-ADDITIVE: ACCOUNT_BALANCE – correct aggregation ———
-- Semi-additive: sum across customers OK, but NOT across time
-- Correct: point-in-time balance = balance on latest snapshot date
SELECT
  CUSTOMER_KEY,
  MAX(SNAPSHOT_DATE)      AS latest_snapshot,
  SUM(ACCOUNT_BALANCE)    AS wrong_sum,      -- WRONG for semi-additive!
  -- Correct: use the latest snapshot value
  MAX(CASE WHEN SNAPSHOT_DATE =
    (SELECT MAX(SNAPSHOT_DATE) FROM dwh.dbo.FACT_ACCOUNT_BALANCE)
    THEN ACCOUNT_BALANCE END)      AS correct_latest_balance
FROM dwh.dbo.FACT_ACCOUNT_BALANCE
GROUP BY CUSTOMER_KEY;
-- — SEMI-ADDITIVE: detect wrong SUM-over-time patterns ———
-- Flagging consumers who accidentally SUM account balance over time
SELECT
  CUSTOMER_KEY,
  COUNT(DISTINCT SNAPSHOT_DATE) AS snapshot_count,
  SUM(ACCOUNT_BALANCE)          AS naive_sum,
  AVG(ACCOUNT_BALANCE)          AS period_avg,  -- sometimes correct
  MAX(ACCOUNT_BALANCE)          AS period_max
FROM dwh.dbo.FACT_ACCOUNT_BALANCE
WHERE SNAPSHOT_DATE >= DATEADD(MONTH, -3, GETDATE())
GROUP BY CUSTOMER_KEY
ORDER BY naive_sum DESC;
```

6. Snapshot vs Transaction Fact Validation

Purpose: Transaction fact tables record discrete events (one row per

transaction). Periodic snapshot facts record state at regular intervals.

Accumulating snapshot facts track a process through defined milestones.

Validate that the correct fact type behaviour is exhibited.

```
-- — TRANSACTION FACT: no repeated event keys ———
SELECT
  TRANSACTION_ID,
  COUNT(*) AS occurrences
FROM dwh.dbo.FACT_SALES_TRANSACTIONS
GROUP BY TRANSACTION_ID
HAVING COUNT(*) > 1;      -- must return 0 rows
-- — TRANSACTION FACT: event date = load date (same-day load) —
SELECT COUNT(*) AS late_loaded_txns
FROM dwh.dbo.FACT_SALES_TRANSACTIONS
WHERE DATEDIFF(DAY, TRANSACTION_DATE, LOAD_TIMESTAMP) > 1;
-- — PERIODIC SNAPSHOT: one row per (entity, period) ———
SELECT
  CUSTOMER_KEY,
  SNAPSHOT_DATE,
  COUNT(*) AS snapshot_count
FROM dwh.dbo.FACT_ACCOUNT_SNAPSHOTS
GROUP BY CUSTOMER_KEY, SNAPSHOT_DATE
HAVING COUNT(*) > 1;      -- must return 0 rows
-- — PERIODIC SNAPSHOT: no gaps in snapshot calendar ———
WITH expected_dates AS (
  SELECT CAST(DATEADD(DAY, -n, GETDATE()) AS DATE) AS snap_dt
  FROM (
    SELECT TOP 90 ROW_NUMBER() OVER (ORDER BY object_id)-1 AS n
    FROM sys.objects
  ) nums
)
SELECT
  e.snap_dt,
  COUNT(f.SNAPSHOT_DATE) AS snapshot_count
FROM expected_dates e
LEFT JOIN dwh.dbo.FACT_ACCOUNT_SNAPSHOTS f
  ON CAST(f.SNAPSHOT_DATE AS DATE) = e.snap_dt
GROUP BY e.snap_dt
HAVING COUNT(f.SNAPSHOT_DATE) = 0
```

```

ORDER BY e.snap_dt;                -- missing snapshot dates
-- — ACCUMULATING SNAPSHOT: milestone dates ordered correctly —
SELECT
  ORDER_KEY,
  ORDER_DATE,
  PICK_DATE,
  PACK_DATE,
  SHIP_DATE,
  DELIVER_DATE,
  CASE
    WHEN PICK_DATE < ORDER_DATE THEN 'FAIL - PICKED BEFORE ORDERED'
    WHEN PACK_DATE < PICK_DATE THEN 'FAIL - PACKED BEFORE PICKED'
    WHEN SHIP_DATE < PACK_DATE THEN 'FAIL - SHIPPED BEFORE PACKED'
    WHEN DELIVER_DATE < SHIP_DATE THEN 'FAIL - DELIVERED BEFORE SHIPPED'
    ELSE 'PASS'
  END AS milestone_sequence_check
FROM dwh.dbo.FACT_ORDER_FULFILLMENT
WHERE PICK_DATE < ORDER_DATE
   OR PACK_DATE < PICK_DATE
   OR SHIP_DATE < PACK_DATE
   OR DELIVER_DATE < SHIP_DATE;

```

7. Duplicate Fact Detection

Purpose: Duplicate rows in a fact table silently double-count every

measure — sales appear twice, costs inflate, KPIs mislead. Detect duplicates at surrogate key, business key, and full-row level with severity classification.

```

-- — Surrogate / PK duplicates —————
SELECT ORDER_LINE_KEY, COUNT(*) AS dup_count
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY ORDER_LINE_KEY
HAVING COUNT(*) > 1;                -- must return 0 rows
-- — Business key duplicates —————
SELECT SOURCE_ORDER_LINE_ID, COUNT(*) AS dup_count
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY SOURCE_ORDER_LINE_ID
HAVING COUNT(*) > 1;                -- must return 0 rows
-- — Full-row (all-column) duplicates —————
SELECT
  CUSTOMER_KEY, PRODUCT_KEY, ORDER_DATE_KEY,
  STORE_KEY, SALE_AMOUNT, QUANTITY,
  COUNT(*) AS full_dup_count
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY
  CUSTOMER_KEY, PRODUCT_KEY, ORDER_DATE_KEY,
  STORE_KEY, SALE_AMOUNT, QUANTITY
HAVING COUNT(*) > 1
ORDER BY full_dup_count DESC;
-- — Impact assessment: duplicated measure totals —————
WITH dup_rows AS (
  SELECT ORDER_LINE_KEY
  FROM dwh.dbo.FACT_ORDER_LINES
  GROUP BY ORDER_LINE_KEY
  HAVING COUNT(*) > 1
)
SELECT
  COUNT(*)                AS dup_row_count,
  SUM(f.SALE_AMOUNT)       AS inflated_amount,
  SUM(f.SALE_AMOUNT) / COUNT(*) AS avg_dup_amount,
  (SELECT SUM(SALE_AMOUNT) FROM dwh.dbo.FACT_ORDER_LINES) AS grand_total
FROM dwh.dbo.FACT_ORDER_LINES f
JOIN dup_rows d ON f.ORDER_LINE_KEY = d.ORDER_LINE_KEY;
-- — Detect duplicates introduced by rerun (same batch loaded twice)
SELECT
  LOAD_BATCH_ID,
  COUNT(*)                AS rows_in_batch,
  COUNT(DISTINCT SOURCE_ORDER_LINE_ID) AS unique_sources

```

```
FROM dwh.dbo.FACT_ORDER_LINES  
GROUP BY LOAD_BATCH_ID  
HAVING COUNT(*) <> COUNT(DISTINCT SOURCE_ORDER_LINE_ID);
```

Data Reconciliation Checks

ETL Reconciliation Checks — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Reconciliation\_Checks\_Python.md`

 Reconciliation = the last line of defence before bad data reaches the business.

1. Source vs Target Count Match

Purpose: The most fundamental reconciliation — every row that left the

source must be accounted for in the target (or in rejects). Any gap is an unexplained data loss that must be investigated before sign-off.

```
-- — Full pipeline count reconciliation —————
WITH pipeline_counts AS (
  SELECT 'SOURCE'      AS layer, COUNT(*) AS row_count,
        GETDATE()      AS checked_at
  FROM source_db.dbo.ORDERS
  UNION ALL
  SELECT 'STAGING',    COUNT(*), GETDATE()
  FROM staging.dbo.ORDERS_STAGE
  UNION ALL
  SELECT 'TARGET',     COUNT(*), GETDATE()
  FROM dwh.dbo.FACT_ORDERS
  UNION ALL
  SELECT 'REJECTS',    COUNT(*), GETDATE()
  FROM etl_control.dbo.LOAD_REJECTS
  WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)
)
SELECT
  layer,
  row_count,
  LAG(row_count) OVER (ORDER BY
    CASE layer
      WHEN 'SOURCE' THEN 1
      WHEN 'STAGING' THEN 2
      WHEN 'TARGET' THEN 3
      WHEN 'REJECTS' THEN 4
    END
  )
  AS prev_layer_count,
  row_count - LAG(row_count) OVER (ORDER BY
    CASE layer
      WHEN 'SOURCE' THEN 1
      WHEN 'STAGING' THEN 2
      WHEN 'TARGET' THEN 3
      WHEN 'REJECTS' THEN 4
    END
  )
  AS layer_gap,
  checked_at
FROM pipeline_counts;
-- — End-to-end accountability: source = target + rejects ———
SELECT
  src.row_count      AS source_count,
  tgt.row_count      AS target_count,
  rej.row_count      AS reject_count,
  tgt.row_count + rej.row_count AS accounted_for,
  src.row_count - (tgt.row_count + rej.row_count) AS unaccounted,
  CASE
    WHEN src.row_count = tgt.row_count + rej.row_count
    THEN 'PASS  — INVESTIGATE'
  END
  AS reconciliation_status
FROM
  (SELECT COUNT(*) AS row_count FROM source_db.dbo.ORDERS) src,
  (SELECT COUNT(*) AS row_count FROM dwh.dbo.FACT_ORDERS) tgt,
  (SELECT COUNT(*) AS row_count FROM etl_control.dbo.LOAD_REJECTS
   WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)) rej;
```

```
-- — Partition-level count reconciliation (by load date) ———
SELECT
    CAST(s.ORDER_DATE AS DATE)          AS partition_dt,
    COUNT(s.ORDER_ID)                   AS source_count,
    COUNT(t.ORDER_KEY)                   AS target_count,
    COUNT(s.ORDER_ID) -
    COUNT(t.ORDER_KEY)                   AS gap,
    ROUND(100.0 * COUNT(t.ORDER_KEY)
          / NULLIF(COUNT(s.ORDER_ID), 0), 4) AS load_pct,
    CASE
        WHEN COUNT(s.ORDER_ID) = COUNT(t.ORDER_KEY) THEN 'PASS'
        ELSE 'FAIL'
    END                                  AS status
FROM source_db.dbo.ORDERS s
LEFT JOIN dwh.dbo.FACT_ORDERS t
    ON s.ORDER_ID = t.SOURCE_ORDER_ID
GROUP BY CAST(s.ORDER_DATE AS DATE)
ORDER BY partition_dt DESC;
-- — Historical run log: trend of count gaps over time ———
SELECT
    run_date,
    source_count,
    target_count,
    source_count - target_count AS gap,
    ROUND(100.0 * target_count
          / NULLIF(source_count, 0), 4) AS load_pct,
    CASE WHEN source_count = target_count THEN 'PASS' ELSE 'FAIL' END AS status
FROM etl_control.dbo.RECONCILIATION_LOG
WHERE table_name = 'ORDERS'
ORDER BY run_date DESC;
```

2. Control Totals Validation

Purpose: Control totals are pre-agreed numeric checksums (SUM, COUNT, MIN,

MAX) recorded at extraction time, carried through the pipeline, and verified at load time. Any deviation means transformation or load errors occurred.

```
-- — Record control totals at extraction time ———
INSERT INTO etl_control.dbo.CONTROL_TOTALS
    (pipeline_run_id, layer, table_name, metric_name,
     metric_value, captured_at)
SELECT
    'RUN_20241201_001' AS pipeline_run_id,
    'SOURCE'           AS layer,
    'ORDERS'           AS table_name,
    'ROW_COUNT'        AS metric_name,
    COUNT(*)           AS metric_value,
    GETDATE()          AS captured_at
FROM source_db.dbo.ORDERS
UNION ALL
SELECT 'RUN_20241201_001', 'SOURCE', 'ORDERS', 'SUM_AMOUNT',
    SUM(AMOUNT),          GETDATE() FROM source_db.dbo.ORDERS
UNION ALL
SELECT 'RUN_20241201_001', 'SOURCE', 'ORDERS', 'MAX_AMOUNT',
    MAX(AMOUNT),          GETDATE() FROM source_db.dbo.ORDERS
UNION ALL
SELECT 'RUN_20241201_001', 'SOURCE', 'ORDERS', 'MIN_AMOUNT',
    MIN(AMOUNT),          GETDATE() FROM source_db.dbo.ORDERS
UNION ALL
SELECT 'RUN_20241201_001', 'SOURCE', 'ORDERS', 'DISTINCT_CUSTOMERS',
    COUNT(DISTINCT CUSTOMER_ID), GETDATE() FROM source_db.dbo.ORDERS;
-- — Validate control totals at target ———
SELECT
    c.metric_name,
    c.metric_value          AS source_control_total,
    tgt.computed_value      AS target_computed_value,
    ABS(c.metric_value - tgt.computed_value) AS abs_diff,
    CASE
```



```

        WHEN ABS(c.metric_value - tgt.computed_value) <= 0.01
        THEN 'PASS ✓' ELSE 'FAIL ✗'
    END
    AS control_total_status
FROM etl_control.dbo.CONTROL_TOTALS c
CROSS APPLY (
    SELECT
        CASE c.metric_name
            WHEN 'ROW_COUNT' THEN CAST(COUNT(*) AS DECIMAL(20,4))
            WHEN 'SUM_AMOUNT' THEN SUM(AMOUNT)
            WHEN 'MAX_AMOUNT' THEN MAX(AMOUNT)
            WHEN 'MIN_AMOUNT' THEN MIN(AMOUNT)
            WHEN 'DISTINCT_CUSTOMERS' THEN CAST(COUNT(DISTINCT CUSTOMER_KEY) AS DECIMAL(20,4))
        END AS computed_value
    FROM dwh.dbo.FACT_ORDERS
) tgt
WHERE c.pipeline_run_id = 'RUN_20241201_001'
AND c.layer = 'SOURCE'
ORDER BY c.metric_name;

```

3. Hash Total Validation

Purpose: A hash total is a single fingerprint (MD5/SHA-256) computed over

all rows and key columns. If even one byte changes anywhere in the data, the hash changes — making it the most sensitive reconciliation check available.

```

-- — Compute row-level hash for every record —————
SELECT
    ORDER_ID,
    HASHBYTES('SHA2_256',
        CONCAT(
            CAST(ORDER_ID AS VARCHAR(20)), '|',
            CAST(CUSTOMER_ID AS VARCHAR(20)), '|',
            ISNULL(CAST(AMOUNT AS VARCHAR(30)), 'NULL'), '|',
            ISNULL(STATUS, 'NULL'), '|',
            ISNULL(CAST(ORDER_DATE AS VARCHAR(30)), 'NULL')
        )
    ) AS row_hash
INTO #source_hashes
FROM source_db.dbo.ORDERS;
-- — Compute same hash on target —————
SELECT
    SOURCE_ORDER_ID AS ORDER_ID,
    HASHBYTES('SHA2_256',
        CONCAT(
            CAST(SOURCE_ORDER_ID AS VARCHAR(20)), '|',
            CAST(CUSTOMER_KEY AS VARCHAR(20)), '|',
            ISNULL(CAST(AMOUNT AS VARCHAR(30)), 'NULL'), '|',
            ISNULL(STATUS, 'NULL'), '|',
            ISNULL(CAST(ORDER_DATE AS VARCHAR(30)), 'NULL')
        )
    ) AS row_hash
INTO #target_hashes
FROM dwh.dbo.FACT_ORDERS;
-- — Compare: rows with hash mismatch —————
SELECT
    s.ORDER_ID,
    CASE
        WHEN t.ORDER_ID IS NULL THEN 'MISSING IN TARGET'
        WHEN s.row_hash <> t.row_hash THEN 'DATA MISMATCH'
        ELSE 'MATCH'
    END AS hash_status
FROM #source_hashes s
LEFT JOIN #target_hashes t ON s.ORDER_ID = t.ORDER_ID
WHERE t.ORDER_ID IS NULL
OR s.row_hash <> t.row_hash;
-- — Table-level aggregate hash (single fingerprint per table) —
SELECT
    HASHBYTES('SHA2_256',

```

```

        STRING_AGG(
            CONCAT(
                CAST(ORDER_ID AS VARCHAR), '|',
                CAST(AMOUNT AS VARCHAR), '|',
                STATUS
            ), ','
        ) WITHIN GROUP (ORDER BY ORDER_ID)
    ) AS table_hash
FROM source_db.dbo.ORDERS;
-- Run same on target and compare: identical hash = identical data

```

4. Aggregation Reconciliation

Purpose: Compare aggregated metrics (daily revenue, weekly orders, monthly

active customers) computed independently from source and target. Any aggregation-level divergence points to fan-out, double-counting, or a transformation defect in a specific segment.

```

-- — Daily revenue reconciliation: source vs target —————
SELECT
    s.order_date,
    s.daily_revenue      AS source_daily_revenue,
    t.daily_revenue      AS target_daily_revenue,
    ABS(s.daily_revenue - t.daily_revenue)      AS abs_diff,
    ROUND(100.0 * ABS(s.daily_revenue - t.daily_revenue)
          / NULLIF(s.daily_revenue, 0), 4)      AS pct_diff,
    CASE
        WHEN ABS(s.daily_revenue - t.daily_revenue) <= 0.01 THEN 'PASS' ✓
        WHEN ABS(s.daily_revenue - t.daily_revenue) <= 100  THEN 'WARN' ⚠
        ELSE 'FAIL' ✗
    END
    AS recon_status
FROM (
    SELECT CAST(ORDER_DATE AS DATE) AS order_date,
           SUM(AMOUNT)              AS daily_revenue
    FROM source_db.dbo.ORDERS
    GROUP BY CAST(ORDER_DATE AS DATE)
) s
FULL OUTER JOIN (
    SELECT ORDER_DATE,
           SUM(AMOUNT)              AS daily_revenue
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY ORDER_DATE
) t ON s.order_date = t.ORDER_DATE
ORDER BY s.order_date DESC;
-- — Multi-dimension aggregation reconciliation —————
-- By region and product category
SELECT
    s.REGION,
    s.CATEGORY,
    s.agg_revenue      AS source_revenue,
    t.agg_revenue      AS target_revenue,
    s.agg_count        AS source_count,
    t.agg_count        AS target_count,
    ABS(s.agg_revenue - t.agg_revenue) AS revenue_diff,
    CASE WHEN ABS(s.agg_revenue - t.agg_revenue) < 0.01
        THEN 'PASS' ELSE 'FAIL'
    END AS status
FROM (
    SELECT o.REGION,
           p.CATEGORY,
           SUM(o.AMOUNT) AS agg_revenue,
           COUNT(*)     AS agg_count
    FROM source_db.dbo.ORDERS o
    JOIN source_db.dbo.PRODUCTS p ON o.PRODUCT_ID = p.PRODUCT_ID
    GROUP BY o.REGION, p.CATEGORY
) s
FULL OUTER JOIN (

```

```

SELECT st.REGION,
       dp.CATEGORY,
       SUM(f.AMOUNT) AS agg_revenue,
       COUNT(*) AS agg_count
FROM dwh.dbo.FACT_ORDERS f
JOIN dwh.dbo.DIM_STORES st ON f.STORE_KEY = st.STORE_KEY
JOIN dwh.dbo.DIM_PRODUCTS dp ON f.PRODUCT_KEY = dp.PRODUCT_KEY
GROUP BY st.REGION, dp.CATEGORY
) t ON s.REGION = t.REGION AND s.CATEGORY = t.CATEGORY
WHERE ABS(ISNULL(s.agg_revenue,0) - ISNULL(t.agg_revenue,0)) > 0.01
ORDER BY revenue_diff DESC;
-- — Grand total reconciliation summary —————
SELECT
    SUM(s.AMOUNT) AS source_grand_total,
    SUM(t.AMOUNT) AS target_grand_total,
    ABS(SUM(s.AMOUNT) - SUM(t.AMOUNT)) AS total_diff,
    ROUND(100.0 * ABS(SUM(s.AMOUNT) - SUM(t.AMOUNT))
    / NULLIF(SUM(s.AMOUNT), 0), 6) AS pct_diff,
    CASE WHEN ABS(SUM(s.AMOUNT) - SUM(t.AMOUNT)) < 1.0
    THEN 'PASS' ELSE 'FAIL' END AS grand_total_status
FROM source_db.dbo.ORDERS s
CROSS JOIN dwh.dbo.FACT_ORDERS t;

```

5. Day-over-Day Delta Validation

Purpose: Compare today's metrics against yesterday's (or the prior period)

to detect anomalous spikes, drops, or flatlines. A sudden $\pm 20\%$ swing in row count or revenue is almost always a pipeline problem, not a business event.

```

-- — DoD row count delta —————
WITH daily_counts AS (
    SELECT
        CAST(Load_TIMESTAMP AS DATE) AS load_date,
        COUNT(*) AS row_count
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY CAST(Load_TIMESTAMP AS DATE)
),
delta AS (
    SELECT
        load_date,
        row_count,
        LAG(row_count) OVER (ORDER BY load_date) AS prev_row_count,
        row_count - LAG(row_count)
        OVER (ORDER BY load_date) AS abs_delta,
        ROUND(100.0 * (row_count -
        LAG(row_count) OVER (ORDER BY load_date))
        / NULLIF(LAG(row_count)
        OVER (ORDER BY load_date), 0), 2) AS pct_delta
    FROM daily_counts
)
SELECT
    load_date,
    row_count,
    prev_row_count,
    abs_delta,
    pct_delta,
    CASE
        WHEN prev_row_count IS NULL THEN 'BASELINE'
        WHEN ABS(pct_delta) <= 5 THEN 'NORMAL'
        WHEN ABS(pct_delta) <= 20 THEN 'WARN'
        WHEN row_count = 0 THEN 'CRITICAL — ZERO LOAD'
        WHEN row_count = prev_row_count THEN 'WARN — FLATLINE'
        ELSE 'ANOMALY'
    END AS delta_status
FROM delta
ORDER BY load_date DESC;
-- — DoD revenue delta —————

```

```

WITH daily_revenue AS (
    SELECT
        ORDER_DATE,
        SUM(AMOUNT) AS daily_rev,
        COUNT(*) AS daily_cnt
    FROM dwh.dbo.FACT_ORDERS
    WHERE ORDER_DATE >= DATEADD(DAY, -30, GETDATE())
    GROUP BY ORDER_DATE
)
SELECT
    ORDER_DATE,
    daily_rev,
    LAG(daily_rev) OVER (ORDER BY ORDER_DATE) AS prev_day_rev,
    daily_rev -
    LAG(daily_rev) OVER (ORDER BY ORDER_DATE) AS rev_delta,
    ROUND(100.0 * (daily_rev -
        LAG(daily_rev) OVER (ORDER BY ORDER_DATE))
        / NULLIF(LAG(daily_rev) OVER (ORDER BY ORDER_DATE), 0), 2) AS pct_change,
    CASE
        WHEN ABS(ROUND(100.0 * (daily_rev -
            LAG(daily_rev) OVER (ORDER BY ORDER_DATE))
            / NULLIF(LAG(daily_rev) OVER (ORDER BY ORDER_DATE), 0), 2)) > 20
        THEN 'ANOMALY - REVIEW'
        ELSE 'NORMAL'
    END AS revenue_delta_flag
FROM daily_revenue
ORDER BY ORDER_DATE DESC;
-- — 7-day moving average vs today (smoothed baseline) —————
WITH daily_revenue AS (
    SELECT ORDER_DATE, SUM(AMOUNT) AS daily_rev
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY ORDER_DATE
),
with_ma AS (
    SELECT
        ORDER_DATE,
        daily_rev,
        AVG(daily_rev) OVER (
            ORDER BY ORDER_DATE
            ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING
        ) AS ma_7d,
        STDEV(daily_rev) OVER (
            ORDER BY ORDER_DATE
            ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING
        ) AS std_7d
    FROM daily_revenue
)
SELECT
    ORDER_DATE,
    daily_rev,
    ROUND(ma_7d, 2) AS moving_avg_7d,
    ROUND(std_7d, 2) AS std_dev_7d,
    ROUND((daily_rev - ma_7d)
        / NULLIF(std_7d, 0), 3) AS z_score,
    CASE
        WHEN ABS((daily_rev - ma_7d)
            / NULLIF(std_7d, 0)) > 3 THEN 'OUTLIER ✖'
        WHEN ABS((daily_rev - ma_7d)
            / NULLIF(std_7d, 0)) > 2 THEN 'WARN ⚠'
        ELSE 'NORMAL ✔'
    END AS anomaly_flag
FROM with_ma
ORDER BY ORDER_DATE DESC;

```

6. Business Reconciliation Reports

Purpose: A structured, stakeholder-ready reconciliation report that

aggregates all checks into a single signed-off artefact. Used for period-end close, financial reporting sign-off, and audit evidence.

```
-- — Full business reconciliation report —————
SELECT
    r.run_date,
    r.pipeline_name,
    r.report_period,
    -- Volume metrics
    r.source_count,
    r.target_count,
    r.reject_count,
    r.source_count - r.target_count - r.reject_count AS unaccounted_rows,
    -- Financial metrics
    r.source_total_revenue,
    r.target_total_revenue,
    r.source_total_revenue - r.target_total_revenue AS revenue_diff,
    ROUND(100.0 * ABS(r.source_total_revenue - r.target_total_revenue)
        / NULLIF(r.source_total_revenue, 0), 6) AS revenue_pct_diff,
    -- Control total status
    r.control_total_status,
    -- Hash validation
    r.hash_match_status,
    -- Overall sign-off
    CASE
        WHEN r.source_count = r.target_count + r.reject_count
            AND ABS(r.source_total_revenue
                - r.target_total_revenue) < 1.0
            AND r.control_total_status = 'PASS'
            AND r.hash_match_status = 'MATCH'
        THEN 'APPROVED ✓'
        WHEN ABS(r.source_total_revenue
            - r.target_total_revenue) < 100.0
        THEN 'CONDITIONAL APPROVAL △'
        ELSE 'REJECTED ✗ - DO NOT USE'
    END
    AS business_sign_off
FROM etl_control.dbo.RECONCILIATION_REPORT r
WHERE r.run_date = CAST(GETDATE() AS DATE)
    AND r.pipeline_name = 'ORDERS_ETL';
-- — Exception drill-down: what failed —————
SELECT
    check_name,
    check_category,
    status,
    source_value,
    target_value,
    variance,
    variance_pct,
    severity,
    recommended_action
FROM etl_control.dbo.RECONCILIATION_EXCEPTIONS
WHERE run_date = CAST(GETDATE() AS DATE)
    AND pipeline_name = 'ORDERS_ETL'
    AND status = 'FAIL'
ORDER BY
    CASE severity
        WHEN 'CRITICAL' THEN 1
        WHEN 'HIGH' THEN 2
        WHEN 'MEDIUM' THEN 3
        ELSE 4
    END;
-- — Trend: reconciliation pass rate over last 30 days —————
SELECT
    run_date,
    total_checks,
    passed_checks,
    failed_checks,
    ROUND(100.0 * passed_checks / NULLIF(total_checks, 0), 2) AS pass_rate_pct,
```

```
overall_status  
FROM etl_control.dbo.RECONCILIATION_SUMMARY  
WHERE run_date >= DATEADD(DAY, -30, GETDATE())  
ORDER BY run_date DESC;
```

Incremental Load & CDC Testing

ETL Incremental CDC Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Incremental\_CDC\_Validations\_Python.md`

⚡ Incremental loads run every hour in production. One logic flaw — a missed

1. Insert-Only Load Validation

Purpose: In insert-only pipelines (append mode), no existing rows should

ever be modified or removed. Validate that every new batch adds rows correctly, watermarks advance monotonically, and previously loaded rows are byte-perfect.

```
-- — Validate watermark (HWM) advanced correctly —————
SELECT
    prev_run.hwm_value          AS previous_hwm,
    curr_run.hwm_value          AS current_hwm,
    curr_run.hwm_value -
    prev_run.hwm_value          AS hwm_advance,
    CASE
        WHEN curr_run.hwm_value > prev_run.hwm_value THEN 'PASS' ✓ — HWM ADVANCED'
        WHEN curr_run.hwm_value = prev_run.hwm_value THEN 'WARN' ⚠ — NO NEW DATA'
        WHEN curr_run.hwm_value < prev_run.hwm_value THEN 'FAIL' ✗ — HWM REGRESSED'
    END                          AS hwm_status
FROM etl_control.dbo.WATERMARK_LOG prev_run
JOIN etl_control.dbo.WATERMARK_LOG curr_run
    ON prev_run.table_name = curr_run.table_name
   AND prev_run.run_seq + 1 = curr_run.run_seq
WHERE curr_run.table_name = 'ORDERS'
ORDER BY curr_run.run_seq DESC;
-- — Rows inserted in current batch fall above previous HWM ———
SELECT
    COUNT(*)      AS out_of_window_rows
FROM dwh.dbo.FACT_ORDERS
WHERE LOAD_TIMESTAMP BETWEEN
    (SELECT MAX(hwm_value) FROM etl_control.dbo.WATERMARK_LOG
     WHERE table_name = 'ORDERS' AND run_seq =
       (SELECT MAX(run_seq)-1 FROM etl_control.dbo.WATERMARK_LOG
        WHERE table_name = 'ORDERS'))
AND
    (SELECT MAX(hwm_value) FROM etl_control.dbo.WATERMARK_LOG
     WHERE table_name = 'ORDERS')
AND SOURCE_LAST_MODIFIED_DT <=
    (SELECT MAX(hwm_value) FROM etl_control.dbo.WATERMARK_LOG
     WHERE table_name = 'ORDERS' AND run_seq =
       (SELECT MAX(run_seq)-1 FROM etl_control.dbo.WATERMARK_LOG
        WHERE table_name = 'ORDERS')));
-- Must return 0: no row sourced before the previous HWM should
-- have been loaded in the current batch
-- — Pre-existing rows unchanged (snapshot compare) —————
SELECT
    snap.ORDER_KEY,
    snap.AMOUNT      AS snap_amount,
    curr.AMOUNT      AS curr_amount,
    CASE
        WHEN curr.ORDER_KEY IS NULL      THEN 'DELETED — FAIL ✗'
        WHEN snap.AMOUNT <> curr.AMOUNT THEN 'MODIFIED — FAIL ✗'
        ELSE                               'INTACT — PASS ✓'
    END AS row_integrity_status
FROM etl_control.dbo.FACT_ORDERS_SNAPSHOT snap -- captured before this run
LEFT JOIN dwh.dbo.FACT_ORDERS curr
    ON snap.ORDER_KEY = curr.ORDER_KEY
WHERE snap.LOAD_TIMESTAMP < (
    SELECT MIN(LOAD_TIMESTAMP) FROM dwh.dbo.FACT_ORDERS
    WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE() AS DATE)
```

```

);
-- — New row count is additive (never shrinks) —————
SELECT
    run_date,
    cumulative_row_count,
    LAG(cumulative_row_count)
        OVER (ORDER BY run_date)      AS prev_cumulative,
    cumulative_row_count -
    LAG(cumulative_row_count)
        OVER (ORDER BY run_date)      AS batch_size,
    CASE
        WHEN cumulative_row_count >=
            LAG(cumulative_row_count) OVER (ORDER BY run_date)
        THEN 'PASS' ELSE 'FAIL - COUNT DECREASED'
    END
    AS append_only_check
FROM etl_control.dbo.LOAD_AUDIT_LOG
WHERE table_name = 'FACT_ORDERS'
ORDER BY run_date DESC;

```

2. Update Handling Validation

Purpose: When source records change, the target must reflect the latest

value exactly. Validate that UPDATES are applied (not inserted as duplicates), that the LAST_MODIFIED timestamp is refreshed, and that only changed fields are updated — not untouched columns.

```

-- — Updated rows in target match current source values —————
SELECT
    s.ORDER_ID,
    s.STATUS              AS source_status,
    t.STATUS              AS target_status,
    s.AMOUNT              AS source_amount,
    t.AMOUNT              AS target_amount,
    s.LAST_MODIFIED_DT    AS source_last_modified,
    t.LAST_MODIFIED_DT    AS target_last_modified,
    CASE
        WHEN s.STATUS = t.STATUS
            AND ABS(s.AMOUNT - t.AMOUNT) < 0.01
        THEN 'PASS ✓'
        ELSE 'FAIL ✗ - TARGET STALE'
    END AS update_status
FROM source_db.dbo.ORDERS s
JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE s.LAST_MODIFIED_DT > (
    SELECT MAX(hwm_value) - INTERVAL '1' HOUR
    FROM etl_control.dbo.WATERMARK_LOG
    WHERE table_name = 'ORDERS'
) -- only check recently changed rows
AND (s.STATUS <> t.STATUS OR ABS(s.AMOUNT - t.AMOUNT) >= 0.01);
-- — No duplicate rows from UPDATE mishandled as INSERT —————
SELECT
    SOURCE_ORDER_ID,
    COUNT(*)            AS row_count
FROM dwh.dbo.FACT_ORDERS
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(*) > 1;
-- must return 0 rows (upsert target)
-- — UPDATE_TIMESTAMP refreshed on changed rows —————
SELECT
    t.SOURCE_ORDER_ID,
    t.LAST_MODIFIED_DT    AS target_last_modified,
    s.LAST_MODIFIED_DT    AS source_last_modified,
    CASE
        WHEN t.LAST_MODIFIED_DT >= s.LAST_MODIFIED_DT THEN 'PASS ✓'
        ELSE 'FAIL ✗ - STALE TIMESTAMP'
    END AS ts_refresh_status
FROM source_db.dbo.ORDERS s
JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID

```



```

WHERE s.LAST_MODIFIED_DT > DATEADD(HOUR, -24, GETDATE())
AND t.LAST_MODIFIED_DT < s.LAST_MODIFIED_DT; -- must return 0 rows
-- — Only changed columns were updated (non-changed = intact) —
-- Compare a column that should NOT have changed (ORDER_DATE)
SELECT
    t.SOURCE_ORDER_ID,
    snap.ORDER_DATE      AS snap_order_date,
    t.ORDER_DATE         AS curr_order_date,
    CASE WHEN snap.ORDER_DATE = t.ORDER_DATE
        THEN 'INTACT ✓' ELSE 'CORRUPTED ✗'
    END AS immutable_col_check
FROM etl_control.dbo.FACT_ORDERS_SNAPSHOT snap
JOIN dwh.dbo.FACT_ORDERS t ON snap.ORDER_KEY = t.ORDER_KEY
WHERE snap.ORDER_DATE <> t.ORDER_DATE; -- must return 0 rows

```

3. Soft Delete vs Hard Delete

Purpose: Source systems signal deletes in two ways — soft (a flag/timestamp

is set, the row stays) or hard (the row is physically removed). Each requires a different ETL response. Validate that the pipeline correctly detects and propagates both delete types.

```

-- — SOFT DELETE: IS_DELETED flag propagated to target —————
SELECT
    s.ORDER_ID,
    s.IS_DELETED          AS source_is_deleted,
    t.IS_DELETED          AS target_is_deleted,
    s.DELETED_AT          AS source_deleted_at,
    t.DELETED_AT          AS target_deleted_at,
    CASE
        WHEN t.IS_DELETED = s.IS_DELETED THEN 'PASS ✓'
        ELSE 'FAIL ✗ — DELETE FLAG NOT PROPAGATED'
    END AS soft_delete_status
FROM source_db.dbo.ORDERS s
JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE s.IS_DELETED = 1
    AND t.IS_DELETED = 0; -- source says deleted, target doesn't
-- — SOFT DELETE: deleted rows excluded from active reports ———
SELECT COUNT(*) AS active_deleted_rows
FROM dwh.dbo.FACT_ORDERS
WHERE IS_DELETED = 1
    AND IS_ACTIVE = 1; -- must return 0: can't be both
-- — HARD DELETE: rows removed from source must be marked/removed
-- in target (compare source PKs vs target source FKs)
SELECT
    t.SOURCE_ORDER_ID,
    t.ORDER_KEY,
    t.IS_DELETED          AS target_delete_flag,
    t.LOAD_TIMESTAMP
FROM dwh.dbo.FACT_ORDERS t
LEFT JOIN source_db.dbo.ORDERS s ON t.SOURCE_ORDER_ID = s.ORDER_ID
WHERE s.ORDER_ID IS NULL -- row gone from source
    AND t.IS_DELETED = 0 -- but target still shows it as active
    AND t.IS_ACTIVE = 1;
-- — Volume: how many soft-deleted rows exist (trend) —————
SELECT
    CAST(DELETED_AT AS DATE) AS delete_date,
    COUNT(*)                AS soft_deleted_count,
    SUM(AMOUNT)             AS deleted_revenue_value
FROM dwh.dbo.FACT_ORDERS
WHERE IS_DELETED = 1
GROUP BY CAST(DELETED_AT AS DATE)
ORDER BY delete_date DESC;

```

4. Logical Delete Handling

Purpose: Logical deletes use business-domain fields to mark records as

inactive: STATUS='CANCELLED', ACTIVE_FLAG='N', END_DATE < TODAY, or VALID_TO = '9999-12-31' flipped. Validate the pipeline correctly reads these signals and routes records into the right active/inactive partitions.

```
-- — STATUS-based logical delete —————
SELECT
  s.ORDER_ID,
  s.STATUS          AS source_status,
  t.STATUS          AS target_status,
  t.IS_ACTIVE       AS target_is_active,
  CASE
    WHEN s.STATUS IN ('CANCELLED','CLOSED','VOID')
      AND t.IS_ACTIVE = 1 THEN 'FAIL ✗ — LOGICALLY DELETED BUT ACTIVE'
    WHEN s.STATUS IN ('CANCELLED','CLOSED','VOID')
      AND t.IS_ACTIVE = 0 THEN 'PASS ✔'
    WHEN s.STATUS = 'ACTIVE'
      AND t.IS_ACTIVE = 1 THEN 'PASS ✔'
    WHEN s.STATUS = 'ACTIVE'
      AND t.IS_ACTIVE = 0 THEN 'FAIL ✗ — ACTIVE BUT FLAGGED INACTIVE'
    ELSE 'UNKNOWN'
  END AS logical_delete_status
FROM source_db.dbo.ORDERS s
JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE (s.STATUS IN ('CANCELLED','CLOSED','VOID') AND t.IS_ACTIVE = 1)
      OR (s.STATUS = 'ACTIVE' AND t.IS_ACTIVE = 0);
-- — DATE-based logical delete: END_DATE passed —————
SELECT
  t.SOURCE_ORDER_ID,
  t.END_DATE,
  t.IS_ACTIVE,
  CASE
    WHEN t.END_DATE < CAST(GETDATE() AS DATE)
      AND t.IS_ACTIVE = 1 THEN 'FAIL ✗ — EXPIRED BUT STILL ACTIVE'
    WHEN t.END_DATE >= CAST(GETDATE() AS DATE)
      AND t.IS_ACTIVE = 0 THEN 'FAIL ✗ — VALID BUT MARKED INACTIVE'
    ELSE 'PASS ✔'
  END AS date_logical_delete_check
FROM dwh.dbo.FACT_ORDERS t
WHERE (t.END_DATE < CAST(GETDATE() AS DATE) AND t.IS_ACTIVE = 1)
      OR (t.END_DATE >= CAST(GETDATE() AS DATE) AND t.IS_ACTIVE = 0);
-- — Logical deletes not physically removed (audit requirement) —
SELECT COUNT(*) AS logically_deleted_rows_preserved
FROM dwh.dbo.FACT_ORDERS
WHERE IS_ACTIVE = 0;    -- must be > 0 if any deletes have occurred
                        -- ensures they're kept for history
-- — Count: active vs inactive distribution —————
SELECT
  IS_ACTIVE,
  STATUS,
  COUNT(*)      AS row_count,
  SUM(AMOUNT)   AS total_amount
FROM dwh.dbo.FACT_ORDERS
GROUP BY IS_ACTIVE, STATUS
ORDER BY IS_ACTIVE DESC, row_count DESC;
```

5. Reprocessing Validation

Purpose: When a failed batch is re-run or a historical date range is

reprocessed, the end state must be identical to if the data had loaded correctly the first time. Validate idempotency: running the pipeline twice on the same input produces exactly the same output.

```
-- — Idempotency: count must be equal after reprocess —————
-- Step 1: Record state before reprocess
SELECT COUNT(*) AS pre_reprocess_count,
       SUM(AMOUNT) AS pre_reprocess_total
```

```

FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_DATE BETWEEN '2024-11-01' AND '2024-11-30';
-- Step 2: Trigger the reprocess (re-run pipeline for Nov 2024)
-- Step 3: Validate post-reprocess state matches pre-reprocess
SELECT
    pre.pre_reprocess_count,
    post.post_reprocess_count,
    ABS(pre.pre_reprocess_count - post.post_reprocess_count) AS count_diff,
    ABS(pre.pre_reprocess_total - post.post_reprocess_total) AS total_diff,
    CASE
        WHEN pre.pre_reprocess_count = post.post_reprocess_count
            AND ABS(pre.pre_reprocess_total -
                post.post_reprocess_total) < 0.01
        THEN 'IDEMPOTENT ✓'
        ELSE 'NOT IDEMPOTENT ✗ - DOUBLE LOAD OR DATA LOSS'
    END AS idempotency_status
FROM (
    SELECT 50000 AS pre_reprocess_count, 1234567.89 AS pre_reprocess_total
) pre
CROSS JOIN (
    SELECT COUNT(*) AS post_reprocess_count,
           SUM(AMOUNT) AS post_reprocess_total
    FROM dwh.dbo.FACT_ORDERS
    WHERE ORDER_DATE BETWEEN '2024-11-01' AND '2024-11-30'
) post;
-- — No phantom rows: reprocess date range must not grow —————
SELECT
    CAST(LOAD_TIMESTAMP AS DATE) AS load_date,
    COUNT(*) AS row_count,
    MIN(LOAD_TIMESTAMP) AS first_load,
    MAX(LOAD_TIMESTAMP) AS last_load
FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_DATE BETWEEN '2024-11-01' AND '2024-11-30'
GROUP BY CAST(LOAD_TIMESTAMP AS DATE)
HAVING COUNT(*) >
    (SELECT MAX(expected_count)
     FROM etl_control.dbo.LOAD_AUDIT_LOG
     WHERE partition_date = CAST(LOAD_TIMESTAMP AS DATE)
           AND table_name = 'FACT_ORDERS');
-- — Reprocess log: all runs for a given batch —————
SELECT
    run_id,
    run_number,
    run_status,
    rows_inserted,
    rows_updated,
    rows_deleted,
    run_start,
    run_end,
    DATEDIFF(SECOND, run_start, run_end) AS duration_secs
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE batch_date = '2024-11-01'
      AND table_name = 'ORDERS'
ORDER BY run_number;

```

6. Duplicate Prevention in Incremental Runs

Purpose: Each incremental run must load each source record exactly once

across all batches. Records near the HWM boundary are especially risky — they can be pulled by two consecutive runs. Validate the deduplication mechanism is airtight across batches.

```

-- — Cross-batch duplicates: same source key in multiple batches —
SELECT
    SOURCE_ORDER_ID,
    COUNT(*) AS load_count,
    COUNT(DISTINCT CAST(LOAD_TIMESTAMP AS DATE)) AS batch_count,

```

```

        MIN(LOAD_TIMESTAMP)          AS first_loaded,
        MAX(LOAD_TIMESTAMP)          AS last_loaded
FROM dwh.dbo.FACT_ORDERS
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(*) > 1                -- must return 0 rows
ORDER BY load_count DESC;
-- — HWM boundary overlap: records loaded in consecutive batches —
SELECT
    f.SOURCE_ORDER_ID,
    f.SOURCE_LAST_MODIFIED_DT,
    f.LOAD_BATCH_ID,
    COUNT(*) OVER (PARTITION BY f.SOURCE_ORDER_ID) AS times_loaded
FROM dwh.dbo.FACT_ORDERS f
JOIN etl_control.dbo.WATERMARK_LOG w
    ON f.SOURCE_LAST_MODIFIED_DT = w.hwm_value -- loaded exactly at HWM boundary
WHERE w.table_name = 'ORDERS';
-- — Batch-level deduplication effectiveness —————
SELECT
    LOAD_BATCH_ID,
    COUNT(*)                AS rows_in_target,
    COUNT(DISTINCT SOURCE_ORDER_ID) AS unique_source_keys,
    COUNT(*) -
    COUNT(DISTINCT SOURCE_ORDER_ID) AS within_batch_dupes,
    CASE
        WHEN COUNT(*) = COUNT(DISTINCT SOURCE_ORDER_ID)
        THEN 'CLEAN ☒' ELSE 'DUPES ☐'
    END AS batch_dedup_status
FROM dwh.dbo.FACT_ORDERS
GROUP BY LOAD_BATCH_ID
ORDER BY LOAD_BATCH_ID DESC;
-- — Detect HWM overlap window (risky rows) —————
-- Rows whose LAST_MODIFIED_DT falls within 1 second of any HWM
SELECT
    f.SOURCE_ORDER_ID,
    f.SOURCE_LAST_MODIFIED_DT,
    w.hwm_value,
    ABS(DATEDIFF(MILLISECOND,
        f.SOURCE_LAST_MODIFIED_DT,
        w.hwm_value))                AS ms_from_hwm,
    COUNT(*) OVER (
        PARTITION BY f.SOURCE_ORDER_ID
    )                                AS times_loaded
FROM dwh.dbo.FACT_ORDERS f
CROSS JOIN etl_control.dbo.WATERMARK_LOG w
WHERE w.table_name = 'ORDERS'
    AND ABS(DATEDIFF(MILLISECOND,
        f.SOURCE_LAST_MODIFIED_DT,
        w.hwm_value)) < 1000
ORDER BY ms_from_hwm;

```

7. Restartability Testing

Purpose: When a pipeline fails mid-run — network drop, DB timeout,

OOM kill — it must be able to restart cleanly from a known checkpoint without re-processing already-committed records or losing any uncommitted records. Validate the checkpoint mechanism and clean-restart behaviour.

```

-- — Validate checkpoint state on failure —————
SELECT
    run_id,
    pipeline_name,
    checkpoint_stage,    -- e.g. 'EXTRACT_COMPLETE', 'TRANSFORM_COMPLETE'
    checkpoint_hwm,
    rows_committed,
    rows_pending,
    failure_reason,
    failed_at,

```

```

restart_eligible,
CASE
    WHEN restart_eligible = 1
        AND checkpoint_stage IS NOT NULL THEN 'RESTARTABLE ✓'
    WHEN restart_eligible = 0 THEN 'FULL RERUN REQUIRED ⚠'
    ELSE 'UNKNOWN STATE ✗'
END AS restart_status
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE run_status = 'FAILED'
ORDER BY failed_at DESC;
-- — After restart: no records in the pre-checkpoint range ———
-- were loaded twice (already committed before failure)
SELECT
    SOURCE_ORDER_ID,
    COUNT(*) AS load_count
FROM dwh.dbo.FACT_ORDERS
WHERE SOURCE_LAST_MODIFIED_DT <= (
    SELECT checkpoint_hwm
    FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE run_id = 'FAILED_RUN_ID'
)
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(*) > 1; -- must return 0: committed rows not re-inserted
-- — After restart: all records above checkpoint were loaded —
SELECT
    s.ORDER_ID,
    CASE
        WHEN t.SOURCE_ORDER_ID IS NULL THEN 'MISSING — NOT LOADED ✗'
        ELSE 'PRESENT — LOADED ✓'
    END AS post_checkpoint_load_status
FROM source_db.dbo.ORDERS s
LEFT JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE s.LAST_MODIFIED_DT > (
    SELECT checkpoint_hwm
    FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE run_id = 'FAILED_RUN_ID'
)
AND t.SOURCE_ORDER_ID IS NULL; -- must return 0 rows
-- — Restart audit trail —————
SELECT
    original_run_id,
    restart_run_id,
    restart_from_stage,
    restart_from_hwm,
    rows_skipped_pre_checkpoint,
    rows_loaded_post_checkpoint,
    restart_status,
    restart_completed_at
FROM etl_control.dbo.PIPELINE_RESTART_LOG
WHERE original_run_id = 'FAILED_RUN_ID';

```

Regression Testing Checks

ETL Regression Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Regression\_Validations\_Python.md`

 Regression testing is your safety net when code changes. A new feature

1. Previous Data Consistency

Purpose: After any pipeline change (new transformation logic, schema

migration, dependency upgrade), prove that every row already in the warehouse has exactly the same value it had before the change. Nothing old should move.

```
-- — Compare current data vs pre-deployment golden snapshot —
SELECT
  curr.ORDER_KEY,
  curr.CUSTOMER_KEY,
  snap.AMOUNT          AS snap_amount,
  curr.AMOUNT          AS curr_amount,
  snap.STATUS          AS snap_status,
  curr.STATUS          AS curr_status,
  snap.NET_REVENUE     AS snap_net_revenue,
  curr.NET_REVENUE     AS curr_net_revenue,
  CASE
    WHEN ABS(snap.AMOUNT - curr.AMOUNT) > 0.01 THEN 'AMOUNT CHANGED ✖'
    WHEN snap.STATUS <> curr.STATUS THEN 'STATUS CHANGED ✖'
    WHEN ABS(snap.NET_REVENUE - curr.NET_REVENUE) > 0.01 THEN 'REVENUE CHANGED ✖'
    ELSE 'CONSISTENT ✔'
  END AS consistency_status
FROM etl_regression.dbo.FACT_ORDERS_GOLDEN snap
JOIN dwh.dbo.FACT_ORDERS curr
  ON snap.ORDER_KEY = curr.ORDER_KEY
WHERE snap.AMOUNT <> curr.AMOUNT
   OR snap.STATUS <> curr.STATUS
   OR ABS(snap.NET_REVENUE - curr.NET_REVENUE) > 0.01;
-- — Aggregate-level consistency: grand totals must not drift —
SELECT
  'PRE-DEPLOY' AS snapshot_type,
  SUM(AMOUNT) AS total_revenue,
  COUNT(*) AS row_count,
  COUNT(DISTINCT CUSTOMER_KEY) AS distinct_customers,
  AVG(AMOUNT) AS avg_order_value
FROM etl_regression.dbo.FACT_ORDERS_GOLDEN
UNION ALL
SELECT
  'POST-DEPLOY',
  SUM(AMOUNT),
  COUNT(*),
  COUNT(DISTINCT CUSTOMER_KEY),
  AVG(AMOUNT)
FROM dwh.dbo.FACT_ORDERS;
-- — Row-level hash comparison: any change = hash mismatch —
SELECT
  snap.ORDER_KEY,
  snap.row_hash AS golden_hash,
  curr_hash.row_hash AS current_hash,
  CASE
    WHEN snap.row_hash = curr_hash.row_hash THEN 'PASS ✔'
    ELSE 'REGRESSION ✖'
  END AS hash_regression_status
FROM etl_regression.dbo.FACT_ORDERS_GOLDEN_HASHES snap
LEFT JOIN (
  SELECT
    ORDER_KEY,
    HASHBYTES('SHA2_256', CONCAT(
```

```

        CAST(ORDER_KEY AS VARCHAR), '|',
        CAST(CUSTOMER_KEY AS VARCHAR), '|',
        CAST(AMOUNT AS VARCHAR), '|',
        ISNULL(STATUS, 'NULL'), '|',
        CAST(NET_REVENUE AS VARCHAR)
    )) AS row_hash
FROM dwh.dbo.FACT_ORDERS
) curr_hash ON snap.ORDER_KEY = curr_hash.ORDER_KEY
WHERE snap.row_hash <> curr_hash.row_hash
OR curr_hash.ORDER_KEY IS NULL;
-- -- Summary: regression impact sizing -----
SELECT
    COUNT(*) AS total_compared,
    SUM(CASE WHEN snap.row_hash =
        curr_hash.row_hash THEN 1 ELSE 0 END) AS unchanged_rows,
    SUM(CASE WHEN snap.row_hash <>
        curr_hash.row_hash THEN 1 ELSE 0 END) AS changed_rows,
    SUM(CASE WHEN curr_hash.ORDER_KEY IS NULL
        THEN 1 ELSE 0 END) AS missing_rows,
    ROUND(100.0 *
        SUM(CASE WHEN snap.row_hash <>
            curr_hash.row_hash THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 4) AS pct_regressed
FROM etl_regression.dbo.FACT_ORDERS_GOLDEN_HASHES snap
LEFT JOIN (
    SELECT ORDER_KEY,
        HASHBYTES('SHA2_256', CONCAT(
            CAST(ORDER_KEY AS VARCHAR), '|', CAST(AMOUNT AS VARCHAR)
        )) AS row_hash
    FROM dwh.dbo.FACT_ORDERS
) curr_hash ON snap.ORDER_KEY = curr_hash.ORDER_KEY;

```

2. Historical Data Integrity

Purpose: Historical partitions (closed months, prior fiscal years) must

be frozen. Once a period is closed, no ETL run, schema change, or hotfix should alter data in that window. Validate that the archive is immutable.

```

-- -- Closed-period row count must be stable across runs -----
SELECT
    a.partition_month,
    a.row_count AS archived_count,
    curr.row_count AS current_count,
    a.row_count -
    curr.row_count AS delta,
    CASE
        WHEN a.row_count = curr.row_count THEN 'FROZEN ☒'
        ELSE 'MUTATED ☐ - INVESTIGATE'
    END AS historical_integrity_status
FROM etl_control.dbo.PARTITION_AUDIT_LOG a
JOIN (
    SELECT
        FORMAT(ORDER_DATE, 'yyyy-MM') AS partition_month,
        COUNT(*) AS row_count
    FROM dwh.dbo.FACT_ORDERS
    WHERE ORDER_DATE < DATEADD(MONTH, -1,
        DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE)))
    GROUP BY FORMAT(ORDER_DATE, 'yyyy-MM')
) curr ON a.partition_month = curr.partition_month
WHERE a.partition_month < FORMAT(
    DATEADD(MONTH, -1, CAST(GETDATE() AS DATE)), 'yyyy-MM'
) -- only audit closed (prior) months
ORDER BY a.partition_month;
-- -- Historical SUM totals must match the audit ledger -----
SELECT
    a.partition_month,
    a.total_revenue AS audited_revenue,
    curr.total_revenue AS current_revenue,

```

```

ABS(a.total_revenue - curr.total_revenue) AS revenue_drift,
CASE
    WHEN ABS(a.total_revenue - curr.total_revenue) < 0.01
    THEN 'PASS ✓' ELSE 'FAIL ✗ - HISTORICAL DATA CHANGED'
END AS revenue_integrity_status
FROM etl_control.dbo.PARTITION_AUDIT_LOG a
JOIN (
    SELECT
        FORMAT(ORDER_DATE, 'yyyy-MM') AS partition_month,
        SUM(AMOUNT) AS total_revenue
    FROM dwh.dbo.FACT_ORDERS
    WHERE ORDER_DATE < DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
    GROUP BY FORMAT(ORDER_DATE, 'yyyy-MM')
) curr ON a.partition_month = curr.partition_month
WHERE a.audited_revenue <> curr.total_revenue;
-- — No DML on closed partitions (audit log check) —————
SELECT
    operation_type,
    table_name,
    affected_partition,
    rows_affected,
    executed_by,
    executed_at,
    pipeline_run_id
FROM etl_control.dbo.DML_AUDIT_LOG
WHERE CAST(affected_partition AS DATE) <
    DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
    AND executed_at >= DATEADD(DAY, -7, GETDATE()) -- last 7 days activity
ORDER BY executed_at DESC;
-- — Row-level hash audit for a specific closed month —————
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN h.row_hash = curr.row_hash
        THEN 1 ELSE 0 END) AS hash_matches,
    SUM(CASE WHEN h.row_hash <> curr.row_hash
        THEN 1 ELSE 0 END) AS hash_mismatches,
    ROUND(100.0 *
        SUM(CASE WHEN h.row_hash <> curr.row_hash THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 6) AS pct_mutated
FROM etl_control.dbo.HISTORICAL_ROW_HASHES h -- hashes frozen at period close
JOIN (
    SELECT ORDER_KEY,
        HASHBYTES('SHA2_256',
            CONCAT(CAST(ORDER_KEY AS VARCHAR), '|',
                CAST(AMOUNT AS VARCHAR), '|', STATUS)
        ) AS row_hash
    FROM dwh.dbo.FACT_ORDERS
    WHERE FORMAT(ORDER_DATE, 'yyyy-MM') = '2024-10'
) curr ON h.ORDER_KEY = curr.ORDER_KEY
WHERE h.partition_month = '2024-10';

```

3. Backward Compatibility

Purpose: Schema and logic changes must not break downstream consumers

(BI tools, APIs, data marts, ML feature stores). Validate that all existing columns still exist with the same names, types, and nullability — and that no breaking changes were introduced silently.

```

-- — Schema: columns present before must still be present —————
SELECT
    baseline.COLUMN_NAME,
    baseline.DATA_TYPE AS baseline_type,
    baseline.IS_NULLABLE AS baseline_nullable,
    baseline.CHARACTER_MAXIMUM_LENGTH AS baseline_length,
    curr.DATA_TYPE AS current_type,
    curr.IS_NULLABLE AS current_nullable,
    curr.CHARACTER_MAXIMUM_LENGTH AS current_length,

```



```

CASE
    WHEN curr.COLUMN_NAME IS NULL
    THEN 'COLUMN DROPPED ✖'
    WHEN baseline.DATA_TYPE <> curr.DATA_TYPE
    THEN 'TYPE CHANGED ✖'
    WHEN baseline.IS_NULLABLE = 'NO'
    AND curr.IS_NULLABLE = 'YES'
    THEN 'NULLABILITY RELAXED ⚠'
    WHEN baseline.IS_NULLABLE = 'YES'
    AND curr.IS_NULLABLE = 'NO'
    THEN 'NULLABILITY TIGHTENED ✖'
    WHEN ISNULL(baseline.CHARACTER_MAXIMUM_LENGTH, 0) >
    ISNULL(curr.CHARACTER_MAXIMUM_LENGTH, 0)
    THEN 'LENGTH REDUCED ✖ - TRUNCATION RISK'
    ELSE 'COMPATIBLE ✔'
END AS compatibility_status
FROM etl_regression.dbo.SCHEMA_BASELINE baseline
LEFT JOIN INFORMATION_SCHEMA.COLUMNS curr
    ON curr.TABLE_NAME = baseline.TABLE_NAME
    AND curr.TABLE_SCHEMA = baseline.TABLE_SCHEMA
    AND curr.COLUMN_NAME = baseline.COLUMN_NAME
WHERE baseline.TABLE_NAME = 'FACT_ORDERS'
ORDER BY
    CASE
        WHEN curr.COLUMN_NAME IS NULL THEN 1
        WHEN baseline.DATA_TYPE <> curr.DATA_TYPE THEN 2
        WHEN baseline.IS_NULLABLE <> curr.IS_NULLABLE THEN 3
        ELSE 4
    END;
-- — New columns added (non-breaking, but worth auditing) —
SELECT
    curr.COLUMN_NAME,
    curr.DATA_TYPE,
    curr.IS_NULLABLE,
    curr.ORDINAL_POSITION,
    'NEW COLUMN ⚠' AS change_type
FROM INFORMATION_SCHEMA.COLUMNS curr
LEFT JOIN etl_regression.dbo.SCHEMA_BASELINE baseline
    ON curr.TABLE_NAME = baseline.TABLE_NAME
    AND curr.COLUMN_NAME = baseline.COLUMN_NAME
WHERE curr.TABLE_NAME = 'FACT_ORDERS'
    AND baseline.COLUMN_NAME IS NULL
ORDER BY curr.ORDINAL_POSITION;
-- — View / stored proc dependencies: still compilable? —
SELECT
    o.name AS dependent_object,
    o.type_desc AS object_type,
    m.definition AS object_definition,
    CASE
        WHEN TRY_CAST(m.object_id AS INT) IS NOT NULL THEN 'EXISTS ✔'
        ELSE 'BROKEN ✖'
    END AS dependency_status
FROM sys.sql_modules m
JOIN sys.objects o ON m.object_id = o.object_id
WHERE m.definition LIKE '%FACT_ORDERS%' -- objects that reference this table
ORDER BY o.type_desc, o.name;

```

4. Re-Run Comparison

Purpose: Run the pipeline twice on identical inputs. Output must be

byte-for-byte identical. If run 1 and run 2 produce different results, the pipeline has non-deterministic logic (random ordering, timestamp drift, ambiguous tie-breaking) that will cause unexplainable discrepancies in production.

```
-- — Compare two pipeline runs on the same input —
```

```

SELECT
    r1.ORDER_KEY,
    r1.AMOUNT          AS run1_amount,
    r2.AMOUNT          AS run2_amount,
    r1.NET_REVENUE     AS run1_revenue,
    r2.NET_REVENUE     AS run2_revenue,
    r1.STATUS          AS run1_status,
    r2.STATUS          AS run2_status,
    CASE
        WHEN r2.ORDER_KEY IS NULL                                THEN 'MISSING IN RUN2 ✗'
        WHEN ABS(r1.AMOUNT - r2.AMOUNT) > 0.01                  THEN 'AMOUNT DIFFERS ✗'
        WHEN ABS(r1.NET_REVENUE - r2.NET_REVENUE) > 0.01       THEN 'REVENUE DIFFERS ✗'
        WHEN r1.STATUS <> r2.STATUS                              THEN 'STATUS DIFFERS ✗'
        ELSE                                                    'MATCH ✔'
    END AS rerun_comparison_status
FROM etl_regression.dbo.FACT_ORDERS_RUN1 r1
LEFT JOIN etl_regression.dbo.FACT_ORDERS_RUN2 r2
    ON r1.ORDER_KEY = r2.ORDER_KEY
WHERE r2.ORDER_KEY IS NULL
    OR ABS(r1.AMOUNT - r2.AMOUNT) > 0.01
    OR ABS(r1.NET_REVENUE - r2.NET_REVENUE) > 0.01
    OR r1.STATUS <> r2.STATUS;
-- — Aggregate comparison: run1 vs run2 totals —————
SELECT
    'RUN1'              AS run_label,
    COUNT(*)            AS row_count,
    SUM(AMOUNT)         AS total_revenue,
    SUM(NET_REVENUE)    AS total_net_revenue,
    AVG(AMOUNT)         AS avg_order_value
FROM etl_regression.dbo.FACT_ORDERS_RUN1
UNION ALL
SELECT 'RUN2', COUNT(*), SUM(AMOUNT), SUM(NET_REVENUE), AVG(AMOUNT)
FROM etl_regression.dbo.FACT_ORDERS_RUN2;
-- — Hash-based determinism check —————
WITH run1_hashes AS (
    SELECT ORDER_KEY,
        HASHBYTES('SHA2_256', CONCAT(
            CAST(ORDER_KEY AS VARCHAR), '|',
            CAST(AMOUNT AS VARCHAR), '|', STATUS, '|',
            CAST(NET_REVENUE AS VARCHAR)
        )) AS row_hash
    FROM etl_regression.dbo.FACT_ORDERS_RUN1
),
run2_hashes AS (
    SELECT ORDER_KEY,
        HASHBYTES('SHA2_256', CONCAT(
            CAST(ORDER_KEY AS VARCHAR), '|',
            CAST(AMOUNT AS VARCHAR), '|', STATUS, '|',
            CAST(NET_REVENUE AS VARCHAR)
        )) AS row_hash
    FROM etl_regression.dbo.FACT_ORDERS_RUN2
)
SELECT
    COUNT(*)              AS total_rows,
    SUM(CASE WHEN r1.row_hash = r2.row_hash
        THEN 1 ELSE 0 END) AS deterministic_rows,
    SUM(CASE WHEN r1.row_hash <> r2.row_hash
        THEN 1 ELSE 0 END) AS non_deterministic_rows,
    CASE
        WHEN SUM(CASE WHEN r1.row_hash <> r2.row_hash
            THEN 1 ELSE 0 END) = 0
        THEN 'DETERMINISTIC ✔' ELSE 'NON-DETERMINISTIC ✗'
    END AS determinism_status
FROM run1_hashes r1
JOIN run2_hashes r2 ON r1.ORDER_KEY = r2.ORDER_KEY;

```

5. Data Drift Detection

Purpose: Data drift is a gradual, silent change in statistical properties

of the data — shifting distributions, new NULL rates, value range creep, new categorical values appearing, or cardinality explosions. Detect drift before it corrupts downstream ML models, dashboards, or aggregations.

```
-- — Statistical profile comparison: current vs baseline ———
WITH current_stats AS (
  SELECT
    'AMOUNT' AS column_name,
    COUNT(*) AS row_count,
    COUNT(AMOUNT) AS non_null_count,
    ROUND(AVG(AMOUNT), 4) AS mean_val,
    ROUND(STDEV(AMOUNT), 4) AS std_dev,
    MIN(AMOUNT) AS min_val,
    MAX(AMOUNT) AS max_val,
    ROUND(
      PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY AMOUNT)
      OVER (), 4) AS median_val
  FROM dwh.dbo.FACT_ORDERS
  WHERE ORDER_DATE >= DATEADD(DAY, -30, GETDATE())
)
SELECT
  b.column_name,
  b.mean_val AS baseline_mean,
  c.mean_val AS current_mean,
  ABS(b.mean_val - c.mean_val)
  / NULLIF(b.std_dev, 0) AS mean_z_score,
  b.std_dev AS baseline_std,
  c.std_dev AS current_std,
  b.min_val AS baseline_min,
  c.min_val AS current_min,
  b.max_val AS baseline_max,
  c.max_val AS current_max,
  ROUND(100.0 *
    (b.non_null_count - c.non_null_count)
    / NULLIF(b.row_count, 0), 4) AS null_rate_change_pct,
  CASE
    WHEN ABS(b.mean_val - c.mean_val)
      / NULLIF(b.std_dev, 0) > 3 THEN 'DRIFT — MEAN SHIFTED ✕'
    WHEN c.std_dev > b.std_dev * 2 THEN 'DRIFT — VARIANCE EXPLODED ✕'
    WHEN c.min_val < b.min_val * 0.5 THEN 'DRIFT — MIN ANOMALY ✕'
    WHEN c.max_val > b.max_val * 2 THEN 'DRIFT — MAX ANOMALY ✕'
    ELSE 'STABLE ✔'
  END AS drift_status
FROM etl_regression.dbo.COLUMN_STATS_BASELINE b
JOIN current_stats c ON b.column_name = c.column_name;
-- — Categorical drift: new values not seen in baseline ———
SELECT DISTINCT
  t.STATUS AS new_value,
  COUNT(*) OVER (
    PARTITION BY t.STATUS
  ) AS row_count,
  'NEW CATEGORY ⚠' AS drift_type
FROM dwh.dbo.FACT_ORDERS t
WHERE t.STATUS NOT IN (
  SELECT DISTINCT STATUS_VALUE
  FROM etl_regression.dbo.CATEGORICAL_BASELINE
  WHERE column_name = 'STATUS'
  AND table_name = 'FACT_ORDERS'
);
-- — NULL rate drift ———
SELECT
  b.column_name,
  b.null_rate_pct AS baseline_null_rate,
  curr.null_rate_pct AS current_null_rate,
```

```

curr.null_rate_pct -
b.null_rate_pct      AS null_rate_delta,
CASE
  WHEN ABS(curr.null_rate_pct - b.null_rate_pct) > 5
  THEN 'NULL RATE DRIFT ✖'
  WHEN ABS(curr.null_rate_pct - b.null_rate_pct) > 1
  THEN 'WARN ⚠'
  ELSE 'STABLE ✔'
END AS null_drift_status
FROM etl_regression.dbo.NULL_RATE_BASELINE b
JOIN (
  SELECT
    'DISCOUNT_AMOUNT' AS column_name,
    ROUND(100.0 * SUM(CASE WHEN DISCOUNT_AMOUNT IS NULL
                          THEN 1 ELSE 0 END)
          / NULLIF(COUNT(*), 0), 4) AS null_rate_pct
  FROM dwh.dbo.FACT_ORDERS
) curr ON b.column_name = curr.column_name
WHERE b.table_name = 'FACT_ORDERS';

```

Data Quality (DQ) Checks

ETL DQ Checks — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_DQ\_Checks\_Python.md`

 Data quality is not a one-time test — it is a continuous, automated

1. Completeness Check

Purpose: Completeness measures whether all expected data is present —

no missing rows, no NULL values in mandatory fields, no partially loaded partitions. A 99% complete dataset produces 100% wrong aggregations for the missing 1%.

```
-- — NULL completeness per column —————
SELECT
  'ORDER_ID'          AS column_name,
  COUNT(*)            AS total_rows,
  SUM(CASE WHEN ORDER_ID IS NULL THEN 1 ELSE 0 END) AS null_count,
  ROUND(100.0 * SUM(CASE WHEN ORDER_ID IS NULL
                      THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 4)                AS null_pct,
  CASE WHEN SUM(CASE WHEN ORDER_ID IS NULL
                      THEN 1 ELSE 0 END) = 0
        THEN 'COMPLETE ☒' ELSE 'INCOMPLETE ☐' END      AS status
FROM dwh.dbo.FACT_ORDERS
UNION ALL SELECT 'CUSTOMER_KEY', COUNT(*),
  SUM(CASE WHEN CUSTOMER_KEY IS NULL THEN 1 ELSE 0 END),
  ROUND(100.0 * SUM(CASE WHEN CUSTOMER_KEY IS NULL THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*),0), 4),
  CASE WHEN SUM(CASE WHEN CUSTOMER_KEY IS NULL THEN 1 ELSE 0 END) = 0
        THEN 'COMPLETE ☒' ELSE 'INCOMPLETE ☐' END
FROM dwh.dbo.FACT_ORDERS
UNION ALL SELECT 'AMOUNT', COUNT(*),
  SUM(CASE WHEN AMOUNT IS NULL THEN 1 ELSE 0 END),
  ROUND(100.0 * SUM(CASE WHEN AMOUNT IS NULL THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*),0), 4),
  CASE WHEN SUM(CASE WHEN AMOUNT IS NULL THEN 1 ELSE 0 END) = 0
        THEN 'COMPLETE ☒' ELSE 'INCOMPLETE ☐' END
FROM dwh.dbo.FACT_ORDERS;
-- — Row completeness: rows where ANY mandatory field is NULL —
SELECT
  COUNT(*)                                AS total_rows,
  SUM(CASE WHEN ORDER_ID IS NULL
            OR CUSTOMER_KEY IS NULL
            OR AMOUNT IS NULL
            OR ORDER_DATE IS NULL
            THEN 1 ELSE 0 END)              AS incomplete_rows,
  ROUND(100.0 *
        SUM(CASE WHEN ORDER_ID IS NULL
            OR CUSTOMER_KEY IS NULL
            OR AMOUNT IS NULL
            OR ORDER_DATE IS NULL
            THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 4)          AS incomplete_row_pct,
  CASE
    WHEN SUM(CASE WHEN ORDER_ID IS NULL
                  OR CUSTOMER_KEY IS NULL
                  OR AMOUNT IS NULL
                  OR ORDER_DATE IS NULL
                  THEN 1 ELSE 0 END) = 0
    THEN 'FULLY COMPLETE ☒'
    ELSE 'HAS INCOMPLETE ROWS ☐'
  END
  AS row_completeness_status
FROM dwh.dbo.FACT_ORDERS;
```

```
-- — Partition completeness: expected dates all loaded —————
WITH expected_dates AS (
  SELECT CAST(DATEADD(DAY, -n, CAST(GETDATE() AS DATE))) AS DATE) AS dt
  FROM (
    SELECT TOP 30
      ROW_NUMBER() OVER (ORDER BY object_id) - 1 AS n
    FROM sys.objects
  ) nums
)
SELECT
  e.dt                                AS expected_date,
  COUNT(f.ORDER_DATE)                AS loaded_rows,
  CASE
    WHEN COUNT(f.ORDER_DATE) = 0 THEN 'MISSING PARTITION ✖'
    WHEN COUNT(f.ORDER_DATE) < 100 THEN 'SPARSE PARTITION ⚠'
    ELSE 'COMPLETE ✔'
  END                                AS partition_status
FROM expected_dates e
LEFT JOIN dwh.dbo.FACT_ORDERS f ON CAST(f.ORDER_DATE AS DATE) = e.dt
GROUP BY e.dt
ORDER BY e.dt DESC;
-- — Empty string masquerading as non-NULL —————
SELECT
  COUNT(*)      AS total_rows,
  SUM(CASE WHEN LTRIM(RTRIM(STATUS)) = '' THEN 1 ELSE 0 END) AS empty_status,
  SUM(CASE WHEN LTRIM(RTRIM(CHANNEL)) = '' THEN 1 ELSE 0 END) AS empty_channel,
  SUM(CASE WHEN STATUS = 'N/A' THEN 1 ELSE 0 END) AS na_status,
  SUM(CASE WHEN STATUS = 'NULL' THEN 1 ELSE 0 END) AS string_null_status,
  SUM(CASE WHEN STATUS = 'UNKNOWN' THEN 1 ELSE 0 END) AS unknown_status
FROM dwh.dbo.FACT_ORDERS;
```

2. Accuracy Check

Purpose: Accuracy validates that data values correctly represent the

real-world entity they describe. Wrong values are worse than NULLs — they silently corrupt every calculation downstream without raising any error.

```
-- — Business rule accuracy: amount must be positive —————
SELECT
  COUNT(*)      AS total_rows,
  SUM(CASE WHEN AMOUNT <= 0 THEN 1 ELSE 0 END) AS non_positive_amt,
  SUM(CASE WHEN AMOUNT > 1000000 THEN 1 ELSE 0 END) AS suspiciously_large,
  SUM(CASE WHEN UNIT_PRICE <= 0 THEN 1 ELSE 0 END) AS non_positive_price,
  SUM(CASE WHEN QUANTITY <= 0 THEN 1 ELSE 0 END) AS non_positive_qty,
  SUM(CASE WHEN DISCOUNT_PCT < 0
    OR DISCOUNT_PCT > 100 THEN 1 ELSE 0 END) AS invalid_discount,
  SUM(CASE WHEN TAX_RATE < 0
    OR TAX_RATE > 1 THEN 1 ELSE 0 END) AS invalid_tax_rate
FROM dwh.dbo.FACT_ORDERS;
-- — Cross-field accuracy: derived value matches components ———
SELECT
  ORDER_KEY,
  UNIT_PRICE,
  QUANTITY,
  AMOUNT                                AS stored_amount,
  ROUND(UNIT_PRICE * QUANTITY, 2)      AS expected_amount,
  ABS(AMOUNT - ROUND(UNIT_PRICE * QUANTITY, 2)) AS diff,
  CASE
    WHEN ABS(AMOUNT -
      ROUND(UNIT_PRICE * QUANTITY, 2)) < 0.01
    THEN 'ACCURATE ✔'
    ELSE 'INACCURATE ✖'
  END AS calculation_accuracy
FROM dwh.dbo.FACT_ORDERS
WHERE ABS(AMOUNT - ROUND(UNIT_PRICE * QUANTITY, 2)) >= 0.01;
-- — Referential accuracy: FK points to active, valid dimension —
SELECT
```

```

f.ORDER_KEY,
f.CUSTOMER_KEY,
d.CUSTOMER_ID,
d.IS_ACTIVE,
d.CUSTOMER_TIER,
CASE
    WHEN d.CUSTOMER_KEY IS NULL THEN 'NO DIM ROW ✕'
    WHEN d.IS_ACTIVE = 0 THEN 'INACTIVE CUSTOMER △'
    WHEN d.CUSTOMER_TIER IS NULL THEN 'MISSING TIER △'
    ELSE 'ACCURATE ✓'
END AS referential_accuracy
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE d.CUSTOMER_KEY IS NULL
    OR d.IS_ACTIVE = 0
    OR d.CUSTOMER_TIER IS NULL;
-- — Source vs target accuracy: spot-check 1000 random rows —
SELECT
    s.ORDER_ID,
    s.AMOUNT AS source_amount,
    t.AMOUNT AS target_amount,
    s.STATUS AS source_status,
    t.STATUS AS target_status,
    CASE
        WHEN ABS(s.AMOUNT - t.AMOUNT) < 0.01
            AND s.STATUS = t.STATUS THEN 'ACCURATE ✓'
        WHEN ABS(s.AMOUNT - t.AMOUNT) >= 0.01 THEN 'AMOUNT ERROR ✕'
        ELSE 'STATUS ERROR ✕'
    END AS accuracy_status
FROM (
    SELECT TOP 1000 ORDER_ID, AMOUNT, STATUS
    FROM source_db.dbo.ORDERS
    ORDER BY NEWID() -- random sample
) s
JOIN dwh.dbo.FACT_ORDERS t ON s.ORDER_ID = t.SOURCE_ORDER_ID
WHERE ABS(s.AMOUNT - t.AMOUNT) >= 0.01
    OR s.STATUS <> t.STATUS;

```

3. Consistency Check

Purpose: Consistency ensures that the same fact is represented

identically across tables, systems, and time. Inconsistent data means two reports showing the same metric produce different numbers — the most credibility-destroying outcome in any data platform.

```

-- — Cross-table consistency: fact vs aggregate mart —
SELECT
    f.daily_revenue AS fact_revenue,
    m.daily_revenue AS mart_revenue,
    ABS(f.daily_revenue -
        m.daily_revenue) AS diff,
    f.order_date,
    CASE
        WHEN ABS(f.daily_revenue - m.daily_revenue) < 0.01
            THEN 'CONSISTENT ✓'
        ELSE 'INCONSISTENT ✕'
    END AS consistency_status
FROM (
    SELECT ORDER_DATE AS order_date, SUM(AMOUNT) AS daily_revenue
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY ORDER_DATE
) f
JOIN (
    SELECT ORDER_DATE, SUM(REVENUE) AS daily_revenue
    FROM dwh.dbo.DAILY_REVENUE_MART
    GROUP BY ORDER_DATE
) m ON f.order_date = m.order_date;

```

```

) m ON f.order_date = m.ORDER_DATE
WHERE ABS(f.daily_revenue - m.daily_revenue) >= 0.01
ORDER BY diff DESC;
-- — Cross-system consistency: DW vs operational report ———
SELECT
    'DW_TOTAL'      AS system,
    SUM(AMOUNT)     AS total_revenue,
    COUNT(*)        AS row_count
FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_DATE = CAST(GETDATE()-1 AS DATE)
UNION ALL
SELECT
    'OPS_REPORT',
    TOTAL_REVENUE,
    ORDER_COUNT
FROM ops_system.dbo.DAILY_SUMMARY
WHERE REPORT_DATE = CAST(GETDATE()-1 AS DATE);
-- — Temporal consistency: same metric across time windows ———
-- Revenue reported in rolling-7d view must match sum of daily rows
SELECT
    w.week_start,
    w.weekly_revenue      AS weekly_view_revenue,
    d.sum_daily_revenue   AS sum_of_daily_revenue,
    ABS(w.weekly_revenue -
        d.sum_daily_revenue) AS diff,
    CASE
        WHEN ABS(w.weekly_revenue - d.sum_daily_revenue) < 0.01
        THEN 'CONSISTENT ✓' ELSE 'INCONSISTENT ✗'
    END AS temporal_consistency
FROM dwh.dbo.WEEKLY_REVENUE_VIEW w
JOIN (
    SELECT DATEADD(DAY, -(DATEPART(WEEKDAY, ORDER_DATE)-1),
        ORDER_DATE) AS week_start,
        SUM(AMOUNT) AS sum_daily_revenue
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY DATEADD(DAY, -(DATEPART(WEEKDAY, ORDER_DATE)-1), ORDER_DATE)
) d ON w.week_start = d.week_start
WHERE ABS(w.weekly_revenue - d.sum_daily_revenue) >= 0.01;
-- — Intra-row consistency: derived field agrees with components —
SELECT
    ORDER_KEY,
    GROSS_AMOUNT,
    DISCOUNT_AMOUNT,
    TAX_AMOUNT,
    NET_REVENUE,
    ROUND(GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT, 2) AS expected_net,
    ABS(NET_REVENUE -
        ROUND(GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT, 2)) AS net_diff,
    CASE
        WHEN ABS(NET_REVENUE -
            ROUND(GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT, 2)) < 0.01
        THEN 'CONSISTENT ✓' ELSE 'INCONSISTENT ✗'
    END AS intra_row_consistency
FROM dwh.dbo.FACT_ORDERS
WHERE ABS(NET_REVENUE -
    ROUND(GROSS_AMOUNT - DISCOUNT_AMOUNT - TAX_AMOUNT, 2)) >= 0.01;

```

4. Timeliness Check

Purpose: Timely data is data that is available when it is needed.

Stale data, missed SLA windows, late-arriving records, and slow pipeline completion are all timeliness failures. Validate that data freshness meets business expectations at every layer.

```

-- — Data freshness: when was the latest record loaded? ———
SELECT
    MAX(LOAD_TIMESTAMP) AS latest_load_ts,

```



```

    GETDATE() AS current_ts,
    DATEDIFF(MINUTE, MAX(LOAD_TIMESTAMP),
             GETDATE()) AS minutes_since_last_load,
    CASE
        WHEN DATEDIFF(MINUTE, MAX(LOAD_TIMESTAMP), GETDATE()) <= 60
        THEN 'FRESH ✓'
        WHEN DATEDIFF(MINUTE, MAX(LOAD_TIMESTAMP), GETDATE()) <= 240
        THEN 'STALE △'
        ELSE 'VERY STALE ✗'
    END AS freshness_status
FROM dwh.dbo.FACT_ORDERS;
-- — Late-arriving records: events loaded after expected window —
SELECT
    ORDER_KEY,
    ORDER_DATE AS event_date,
    LOAD_TIMESTAMP AS loaded_at,
    DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) AS hours_late,
    CASE
        WHEN DATEDIFF(HOUR, ORDER_DATE,
                     LOAD_TIMESTAMP) <= 24 THEN 'ON TIME ✓'
        WHEN DATEDIFF(HOUR, ORDER_DATE,
                     LOAD_TIMESTAMP) <= 72 THEN 'LATE △'
        ELSE 'VERY LATE ✗'
    END AS timeliness_status
FROM dwh.dbo.FACT_ORDERS
WHERE DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) > 24
ORDER BY hours_late DESC;
-- — Pipeline SLA adherence: % of runs completing on time —
SELECT
    pipeline_name,
    COUNT(*) AS total_runs,
    SUM(CASE WHEN DATEDIFF(SECOND, run_start, run_end)
                <= sla_seconds THEN 1 ELSE 0 END) AS on_time_runs,
    ROUND(100.0 *
          SUM(CASE WHEN DATEDIFF(SECOND, run_start, run_end)
                        <= sla_seconds THEN 1 ELSE 0 END)
          / NULLIF(COUNT(*), 0), 2) AS sla_adherence_pct,
    AVG(DATEDIFF(SECOND, run_start, run_end)) AS avg_duration_secs,
    MAX(DATEDIFF(SECOND, run_start, run_end)) AS max_duration_secs
FROM etl_control.dbo.PIPELINE_RUN_LOG r
JOIN etl_control.dbo.PIPELINE_SLA s USING (pipeline_name)
WHERE r.batch_date >= DATEADD(DAY, -30, GETDATE())
    AND r.run_status = 'SUCCESS'
GROUP BY pipeline_name
ORDER BY sla_adherence_pct ASC;
-- — Event-to-load latency distribution —
SELECT
    CASE
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 1 THEN '< 1h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 4 THEN '1-4h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 24 THEN '4-24h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 72 THEN '1-3d'
        ELSE '> 3d'
    END AS latency_bucket,
    COUNT(*) AS row_count,
    ROUND(100.0 * COUNT(*)
          / NULLIF(SUM(COUNT(*)) OVER (), 0), 2) AS pct
FROM dwh.dbo.FACT_ORDERS
GROUP BY
    CASE
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 1 THEN '< 1h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 4 THEN '1-4h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 24 THEN '4-24h'
        WHEN DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP) < 72 THEN '1-3d'
        ELSE '> 3d'
    END
ORDER BY MIN(DATEDIFF(HOUR, ORDER_DATE, LOAD_TIMESTAMP));

```

5. Uniqueness Check

Purpose: Every entity that should be unique must appear exactly once.

Duplicate customers, orders, or products corrupt JOIN results, double-count metrics, and produce inconsistent headcounts. Uniqueness failures are the most common DQ defect in production pipelines.

```
-- — Primary key uniqueness —————
SELECT
  'ORDER_KEY (surrogate PK)' AS key_type,
  COUNT(*)                  AS total_rows,
  COUNT(DISTINCT ORDER_KEY) AS unique_values,
  COUNT(*) -
  COUNT(DISTINCT ORDER_KEY) AS duplicates,
  CASE
    WHEN COUNT(*) = COUNT(DISTINCT ORDER_KEY) THEN 'UNIQUE ✓'
    ELSE 'DUPLICATES ✗'
  END AS uniqueness_status
FROM dwh.dbo.FACT_ORDERS
UNION ALL SELECT
  'SOURCE_ORDER_ID (business key)',
  COUNT(*), COUNT(DISTINCT SOURCE_ORDER_ID),
  COUNT(*) - COUNT(DISTINCT SOURCE_ORDER_ID),
  CASE WHEN COUNT(*) = COUNT(DISTINCT SOURCE_ORDER_ID)
    THEN 'UNIQUE ✓' ELSE 'DUPLICATES ✗' END
FROM dwh.dbo.FACT_ORDERS;
-- — Find the actual duplicate rows —————
SELECT
  SOURCE_ORDER_ID,
  COUNT(*)          AS occurrence_count,
  MIN(ORDER_KEY)    AS first_key,
  MAX(ORDER_KEY)    AS last_key,
  MIN(LOAD_TIMESTAMP) AS first_loaded,
  MAX(LOAD_TIMESTAMP) AS last_loaded
FROM dwh.dbo.FACT_ORDERS
GROUP BY SOURCE_ORDER_ID
HAVING COUNT(*) > 1
ORDER BY occurrence_count DESC;
-- — Composite key uniqueness (grain check) —————
SELECT
  CUSTOMER_KEY,
  PRODUCT_KEY,
  CAST(ORDER_DATE AS DATE) AS order_date,
  COUNT(*)                AS row_count
FROM dwh.dbo.FACT_ORDER_LINES
GROUP BY CUSTOMER_KEY, PRODUCT_KEY, CAST(ORDER_DATE AS DATE)
HAVING COUNT(*) > 1
ORDER BY row_count DESC;
-- — Dimension uniqueness: one row per natural key —————
SELECT
  CUSTOMER_ID,
  COUNT(*) AS version_count
FROM dwh.dbo.DIM_CUSTOMERS
WHERE IS_CURRENT = 1      -- only active rows
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;      -- must return 0 rows
-- — Near-duplicate detection (fuzzy) —————
-- Customers with same name and different IDs
SELECT
  a.CUSTOMER_ID AS id_a,
  b.CUSTOMER_ID AS id_b,
  a.FULL_NAME,
  a.EMAIL       AS email_a,
  b.EMAIL       AS email_b,
  'POSSIBLE DUPLICATE ⚠' AS flag
FROM dwh.dbo.DIM_CUSTOMERS a
JOIN dwh.dbo.DIM_CUSTOMERS b
```

```

ON a.FULL_NAME = b.FULL_NAME
AND a.CUSTOMER_ID < b.CUSTOMER_ID -- avoid self-join and reverse dupes
WHERE a.IS_CURRENT = 1
AND b.IS_CURRENT = 1;

```

6. Validity Check

Purpose: Valid data conforms to defined formats, allowed values, domain

constraints, and referential rules. An email without '@', a date of '9999-99-99', a country code 'XX', or a STATUS value of 'PENDINGG' are all invalid — and all slip through if there is no validity gate.

```

-- — Format validation —————
SELECT
  COUNT(*)    AS total_rows,
  -- Email format
  SUM(CASE WHEN EMAIL NOT LIKE '%@%.%' THEN 1 ELSE 0 END) AS invalid_email,
  -- Phone: digits, spaces, +, -, ()
  SUM(CASE WHEN PHONE LIKE '%[^0-9 +\-\(\)]%' THEN 1 ELSE 0 END) AS invalid_phone,
  -- ZIP code: 5 digits or 5+4
  SUM(CASE WHEN POSTAL_CODE NOT LIKE '[0-9][0-9][0-9][0-9][0-9]'
        AND POSTAL_CODE NOT LIKE '[0-9][0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]'
        THEN 1 ELSE 0 END)
        AS invalid_zip,
  -- Date not in valid range
  SUM(CASE WHEN ORDER_DATE < '2000-01-01'
        OR ORDER_DATE > GETDATE() THEN 1 ELSE 0 END) AS invalid_date
FROM dwh.dbo.FACT_ORDERS;
-- — Allowed value (domain) validation —————
SELECT
  STATUS,
  COUNT(*)    AS row_count,
  CASE
    WHEN STATUS IN ('PENDING','PROCESSING','SHIPPED',
                   'DELIVERED','CANCELLED','RETURNED')
    THEN 'VALID ✓'
    ELSE 'INVALID ✗'
  END AS domain_validity
FROM dwh.dbo.FACT_ORDERS
GROUP BY STATUS
ORDER BY domain_validity, row_count DESC;
-- — Referential validity: all FK values exist in parent —————
SELECT COUNT(*) AS orphaned_orders
FROM dwh.dbo.FACT_ORDERS f
WHERE NOT EXISTS (
  SELECT 1 FROM dwh.dbo.DIM_CUSTOMERS d
  WHERE d.CUSTOMER_KEY = f.CUSTOMER_KEY
);
-- — Conditional validity: if DELIVERED then DELIVERY_DATE IS NOT NULL
SELECT
  COUNT(*)    AS total_delivered,
  SUM(CASE WHEN DELIVERY_DATE IS NULL THEN 1 ELSE 0 END) AS missing_delivery_date,
  SUM(CASE WHEN DELIVERY_DATE < ORDER_DATE THEN 1 ELSE 0 END) AS delivery_before_order
FROM dwh.dbo.FACT_ORDERS
WHERE STATUS = 'DELIVERED';
-- — Pattern validity: ORDER_ID must match ORD-YYYYMMDD-NNNNN —
SELECT ORDER_ID
FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_ID NOT LIKE
  'ORD-[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]';

```

ETL E2E Pipeline Tests — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_E2E\_Pipeline\_Tests\_Python.md`

🌐 Unit tests verify individual components. End-to-end tests verify that

1. Source-to-Stage Smoke Test

Purpose: Confirm the very first hop — source to staging — is clean.

Validates that extraction is complete, no schema drift occurred at the landing zone, and the raw staged data is fit for transformation.

```
-- — End-to-end row accountability: source → stage —————
SELECT
  src.source_count,
  stg.stage_count,
  src.source_count - stg.stage_count      AS gap,
  ROUND(100.0 * stg.stage_count
        / NULLIF(src.source_count, 0), 6)  AS load_pct,
  CASE
    WHEN src.source_count = stg.stage_count THEN 'PASS 🟢'
    WHEN src.source_count > stg.stage_count THEN 'UNDER-EXTRACTED ✖'
    ELSE                                     'OVER-EXTRACTED ✖'
  END
  AS e2e_src_stg_status
FROM
  (SELECT COUNT(*) AS source_count
   FROM source_db.dbo.ORDERS
   WHERE LAST_MODIFIED_DT >= (
     SELECT MAX(hwm_value) - INTERVAL '1' HOUR
     FROM etl_control.dbo.WATERMARK_LOG
     WHERE table_name = 'ORDERS'
   )) src,
  (SELECT COUNT(*) AS stage_count
   FROM staging.dbo.ORDERS_STAGE
   WHERE BATCH_ID = (
     SELECT MAX(BATCH_ID)
     FROM staging.dbo.ORDERS_STAGE
   )) stg;

-- — Schema conformance: staged columns match source —————
SELECT
  src_col.COLUMN_NAME,
  src_col.DATA_TYPE    AS source_type,
  stg_col.DATA_TYPE    AS stage_type,
  CASE
    WHEN stg_col.COLUMN_NAME IS NULL      THEN 'MISSING IN STAGE ✖'
    WHEN src_col.DATA_TYPE <> stg_col.DATA_TYPE THEN 'TYPE MISMATCH ✖'
    ELSE                                  'OK 🟢'
  END
  AS schema_status
FROM INFORMATION_SCHEMA.COLUMNS src_col
LEFT JOIN INFORMATION_SCHEMA.COLUMNS stg_col
  ON src_col.COLUMN_NAME = stg_col.COLUMN_NAME
  AND stg_col.TABLE_NAME = 'ORDERS_STAGE'
  AND stg_col.TABLE_SCHEMA = 'staging'
WHERE src_col.TABLE_NAME = 'ORDERS'
  AND src_col.TABLE_SCHEMA = 'source_db'
ORDER BY CASE WHEN stg_col.COLUMN_NAME IS NULL THEN 1 ELSE 2 END;

-- — Data sample spot-check: 10 source rows in stage —————
SELECT
  s.ORDER_ID,
  s.AMOUNT      AS src_amount,
  st.AMOUNT     AS stg_amount,
  s.STATUS      AS src_status,
  st.STATUS     AS stg_status,
  CASE
```

```

        WHEN ABS(s.AMOUNT - st.AMOUNT) < 0.01
        AND s.STATUS = st.STATUS      THEN 'MATCH ✓'
        ELSE                          'MISMATCH ✗'
    END
        AS spot_check_status
FROM (
    SELECT TOP 10 ORDER_ID, AMOUNT, STATUS
    FROM source_db.dbo.ORDERS
    ORDER BY NEWID()
) s
JOIN staging.dbo.ORDERS_STAGE st ON s.ORDER_ID = st.SOURCE_ORDER_ID;

```

2. Stage-to-Warehouse Smoke Test

Purpose: After transformation, every business rule must have fired

correctly. Validate that the warehouse layer holds correct values, all dimension FKs resolve, and the transformed measures match independently recalculated expectations.

```

-- — Stage → Warehouse row accountability —————
SELECT
    stg.stage_rows,
    wh.warehouse_rows,
    rej.rejected_rows,
    stg.stage_rows - wh.warehouse_rows
    - rej.rejected_rows                AS unaccounted,
    CASE
        WHEN stg.stage_rows = wh.warehouse_rows
        + rej.rejected_rows THEN 'PASS ✓'
        ELSE                  'FAIL ✗'
    END
        AS stg_wh_accountability
FROM
    (SELECT COUNT(*) AS stage_rows FROM staging.dbo.ORDERS_STAGE
     WHERE BATCH_ID = (SELECT MAX(BATCH_ID) FROM staging.dbo.ORDERS_STAGE)) stg,
    (SELECT COUNT(*) AS warehouse_rows FROM dwh.dbo.FACT_ORDERS
     WHERE LOAD_BATCH_ID = (SELECT MAX(LOAD_BATCH_ID)
                           FROM dwh.dbo.FACT_ORDERS)) wh,
    (SELECT COUNT(*) AS rejected_rows FROM etl_control.dbo.LOAD_REJECTS
     WHERE CAST(rejected_at AS DATE) = CAST(GETDATE() AS DATE)) rej;
-- — Transformation correctness: re-derive 5 key metrics —————
SELECT
    wh.ORDER_KEY,
    wh.NET_REVENUE                AS stored_net,
    ROUND(stg.GROSS_AMOUNT
    - stg.DISCOUNT_AMOUNT
    - stg.TAX_AMOUNT, 2)          AS expected_net,
    ABS(wh.NET_REVENUE - ROUND(stg.GROSS_AMOUNT
    - stg.DISCOUNT_AMOUNT
    - stg.TAX_AMOUNT, 2))          AS net_diff,
    CASE
        WHEN ABS(wh.NET_REVENUE - ROUND(stg.GROSS_AMOUNT
    - stg.DISCOUNT_AMOUNT
    - stg.TAX_AMOUNT, 2)) < 0.01   THEN 'CORRECT ✓'
        ELSE                      'WRONG ✗'
    END
        AS transform_accuracy
FROM staging.dbo.ORDERS_STAGE stg
JOIN dwh.dbo.FACT_ORDERS      wh
    ON stg.SOURCE_ORDER_ID = wh.SOURCE_ORDER_ID
WHERE stg.BATCH_ID = (SELECT MAX(BATCH_ID) FROM staging.dbo.ORDERS_STAGE)
AND ABS(wh.NET_REVENUE - ROUND(stg.GROSS_AMOUNT
    - stg.DISCOUNT_AMOUNT - stg.TAX_AMOUNT, 2)) >= 0.01;
-- — All dimension FKs resolve in latest batch —————
SELECT
    'CUSTOMER_KEY'      AS fk_name,
    COUNT(*)            AS total_rows,
    SUM(CASE WHEN d.CUSTOMER_KEY IS NULL
        THEN 1 ELSE 0 END) AS unresolved_fks,
    ROUND(100.0 * SUM(CASE WHEN d.CUSTOMER_KEY IS NULL

```

```

        THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0), 4) AS unresolved_pct
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_CUSTOMERS d ON f.CUSTOMER_KEY = d.CUSTOMER_KEY
WHERE f.LOAD_BATCH_ID = (SELECT MAX(LOAD_BATCH_ID) FROM dwh.dbo.FACT_ORDERS)
UNION ALL
SELECT 'PRODUCT_KEY', COUNT(*),
    SUM(CASE WHEN p.PRODUCT_KEY IS NULL THEN 1 ELSE 0 END),
    ROUND(100.0 * SUM(CASE WHEN p.PRODUCT_KEY IS NULL THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*),0), 4)
FROM dwh.dbo.FACT_ORDERS f
LEFT JOIN dwh.dbo.DIM_PRODUCTS p ON f.PRODUCT_KEY = p.PRODUCT_KEY
WHERE f.LOAD_BATCH_ID = (SELECT MAX(LOAD_BATCH_ID) FROM dwh.dbo.FACT_ORDERS);

```

3. Warehouse-to-Report Layer Smoke Test

Purpose: The final hop — the numbers that analysts and executives see.

Validate that BI views, aggregate marts, and report queries return correct figures, match the underlying fact table, and respond within SLA.

```

-- — Report layer vs fact table: daily revenue agrees —————
SELECT
    r.REPORT_DATE,
    r.DAILY_REVENUE          AS report_revenue,
    f.FACT_REVENUE          AS fact_revenue,
    ABS(r.DAILY_REVENUE -
        f.FACT_REVENUE)      AS diff,
    CASE
        WHEN ABS(r.DAILY_REVENUE - f.FACT_REVENUE) < 0.01 THEN 'MATCH 

```

```

        <= refresh_sla_minutes THEN 'FRESH ✓'
    ELSE
        'STALE ✗'
    END AS freshness_status
FROM etl_control.dbo.REPORT_REFRESH_LOG
ORDER BY minutes_stale DESC;
-- — NULL KPIs in report layer (aggregation errors produce NULLs)
SELECT
    REPORT_DATE,
    DAILY_REVENUE,
    ORDER_COUNT,
    AVG_ORDER_VALUE,
    CASE
        WHEN DAILY_REVENUE IS NULL THEN 'NULL REVENUE ✗'
        WHEN ORDER_COUNT IS NULL THEN 'NULL COUNT ✗'
        WHEN AVG_ORDER_VALUE IS NULL THEN 'NULL AVG ✗'
        ELSE
            'OK ✓'
    END AS kpi_null_check
FROM dwh.dbo.RPT_DAILY_REVENUE
WHERE REPORT_DATE >= DATEADD(DAY, -7, GETDATE())
AND (DAILY_REVENUE IS NULL
    OR ORDER_COUNT IS NULL
    OR AVG_ORDER_VALUE IS NULL);

```

4. Golden Record Validation

Purpose: A golden record is a single, authoritative test transaction

injected at the source and traced all the way to the report layer.

If you can follow one known order from source DB → stage → fact → dashboard

with exact values, the entire pipeline is verified end-to-end.

```

-- — Inject a golden test record at source —————
INSERT INTO source_db.dbo.ORDERS
    (ORDER_ID, CUSTOMER_ID, PRODUCT_ID, AMOUNT,
     DISCOUNT_AMOUNT, TAX_AMOUNT, STATUS, ORDER_DATE)
VALUES
    ('E2E-TEST-20241201-001', 'CUST-99999', 'PROD-88888',
     1000.00, 50.00, 95.00, 'DELIVERED', '2024-12-01');
-- Expected NET_REVENUE = 1000 - 50 - 95 = 855.00
-- — Step 1: Verify golden record reached staging —————
SELECT
    SOURCE_ORDER_ID,
    AMOUNT, DISCOUNT_AMOUNT, TAX_AMOUNT, STATUS,
    BATCH_ID, LOADED_AT,
    CASE WHEN SOURCE_ORDER_ID = 'E2E-TEST-20241201-001'
        THEN 'FOUND IN STAGE ✓' ELSE 'NOT FOUND ✗' END AS stage_status
FROM staging.dbo.ORDERS_STAGE
WHERE SOURCE_ORDER_ID = 'E2E-TEST-20241201-001';
-- — Step 2: Verify transformation in warehouse —————
SELECT
    SOURCE_ORDER_ID,
    AMOUNT,
    NET_REVENUE,
    CUSTOMER_KEY,
    PRODUCT_KEY,
    STATUS,
    CASE
        WHEN SOURCE_ORDER_ID = 'E2E-TEST-20241201-001'
            AND ABS(NET_REVENUE - 855.00) < 0.01
            AND CUSTOMER_KEY IS NOT NULL
            AND PRODUCT_KEY IS NOT NULL
            THEN 'CORRECT IN WH ✓'
        WHEN SOURCE_ORDER_ID = 'E2E-TEST-20241201-001'
            AND ABS(NET_REVENUE - 855.00) >= 0.01
            THEN 'WRONG NET_REVENUE ✗'
        WHEN SOURCE_ORDER_ID = 'E2E-TEST-20241201-001'
            AND CUSTOMER_KEY IS NULL
            THEN 'UNRESOLVED FK ✗'
        ELSE
            'NOT FOUND ✗'
    END AS warehouse_status

```

```

FROM dwh.dbo.FACT_ORDERS
WHERE SOURCE_ORDER_ID = 'E2E-TEST-20241201-001';
-- — Step 3: Verify golden record appears in report —————
SELECT
    REPORT_DATE,
    DAILY_REVENUE,
    ORDER_COUNT,
    CASE
        WHEN DAILY_REVENUE >= 855.00 THEN 'PRESENT IN REPORT ✓'
        ELSE 'MISSING FROM REPORT ✗'
    END AS report_presence
FROM dwh.dbo.RPT_DAILY_REVENUE
WHERE REPORT_DATE = '2024-12-01';
-- — Cleanup: remove golden test record post-validation —————
DELETE FROM source_db.dbo.ORDERS
WHERE ORDER_ID = 'E2E-TEST-20241201-001';
DELETE FROM staging.dbo.ORDERS_STAGE
WHERE SOURCE_ORDER_ID = 'E2E-TEST-20241201-001';
DELETE FROM dwh.dbo.FACT_ORDERS
WHERE SOURCE_ORDER_ID = 'E2E-TEST-20241201-001';

```

5. Pipeline Orchestration & Dependency Validation

Purpose: In production, pipelines have dependencies — DIM loads must

complete before FACT loads, staging must succeed before transformation, aggregates must refresh after facts. Validate that the DAG executes in the correct order, failed dependencies halt downstream jobs, and SLA windows are respected across the full DAG.

```

-- — DAG execution order validation —————
SELECT
    t.task_name,
    t.task_order,
    t.start_time,
    t.end_time,
    t.status,
    DATEDIFF(SECOND, t.start_time, t.end_time) AS duration_secs,
    -- Verify each task started AFTER its dependency completed
    dep.end_time AS dependency_end,
    CASE
        WHEN dep.task_name IS NULL THEN 'NO DEPENDENCY ✓'
        WHEN dep.status = 'SUCCESS'
            AND t.start_time >= dep.end_time THEN 'ORDER CORRECT ✓'
        WHEN dep.status = 'SUCCESS'
            AND t.start_time < dep.end_time THEN 'STARTED BEFORE DEP ✗'
        WHEN dep.status = 'FAILED'
            AND t.status = 'SUCCESS' THEN 'RAN DESPITE FAILED DEP ✗'
        ELSE 'DEP FAILED — SKIPPED ⚠'
    END AS dependency_status
FROM etl_control.dbo.DAG_TASK_LOG t
LEFT JOIN etl_control.dbo.DAG_TASK_LOG dep
    ON dep.run_id = t.run_id
    AND dep.task_name = t.depends_on
WHERE t.run_id = (
    SELECT MAX(run_id)
    FROM etl_control.dbo.DAG_TASK_LOG
)
ORDER BY t.task_order;
-- — Critical path: longest-running chain —————
WITH RECURSIVE dag_chain AS (
    SELECT
        task_name,
        depends_on,
        DATEDIFF(SECOND, start_time, end_time) AS chain_duration,
        task_name AS chain_path
    FROM etl_control.dbo.DAG_TASK_LOG

```



```

WHERE depends_on IS NULL
AND run_id = (SELECT MAX(run_id) FROM etl_control.dbo.DAG_TASK_LOG)
UNION ALL
SELECT
    t.task_name,
    t.depends_on,
    chain_duration + DATEDIFF(SECOND, t.start_time, t.end_time),
    chain_path || ' → ' || t.task_name
FROM etl_control.dbo.DAG_TASK_LOG t
JOIN dag_chain c ON t.depends_on = c.task_name
WHERE t.run_id = (SELECT MAX(run_id) FROM etl_control.dbo.DAG_TASK_LOG)
)
SELECT chain_path, chain_duration AS critical_path_secs
FROM dag_chain
ORDER BY critical_path_secs DESC
LIMIT 1;
-- — Dependency failure propagation —————
SELECT
    parent.task_name      AS failed_task,
    child.task_name       AS downstream_task,
    child.status          AS downstream_status,
    CASE
        WHEN parent.status = 'FAILED'
            AND child.status = 'SKIPPED' THEN 'CORRECTLY BLOCKED ✓'
        WHEN parent.status = 'FAILED'
            AND child.status = 'SUCCESS' THEN 'RAN DESPITE FAILURE ✗'
        ELSE                                'N/A'
    END
    AS propagation_check
FROM etl_control.dbo.DAG_TASK_LOG parent
JOIN etl_control.dbo.DAG_TASK_LOG child
    ON child.depends_on = parent.task_name
    AND child.run_id = parent.run_id
WHERE parent.status = 'FAILED'
    AND parent.run_id = (SELECT MAX(run_id) FROM etl_control.dbo.DAG_TASK_LOG);

```

6. Full E2E Regression after Deployment

Purpose: After any deployment — hotfix, schema change, new feature —

run a compressed end-to-end regression that compares today's pipeline output against yesterday's (baseline) across every layer. Any deviation is a regression until proven otherwise.

```

-- — Layer-by-layer regression: today vs yesterday —————
SELECT
    'SOURCE'      AS layer,
    'ROW_COUNT'   AS metric,
    yesterday.n   AS yesterday_val,
    today.n       AS today_val,
    today.n - yesterday.n AS delta,
    ROUND(100.0 * (today.n - yesterday.n)
        / NULLIF(yesterday.n, 0), 2) AS pct_delta,
    CASE
        WHEN ABS(100.0 * (today.n - yesterday.n)
            / NULLIF(yesterday.n, 0)) <= 20 THEN 'NORMAL ✓'
        WHEN today.n = 0                      THEN 'ZERO LOAD ✗'
        ELSE                                'REGRESSION ✗'
    END AS regression_status
FROM
    (SELECT COUNT(*) AS n FROM source_db.dbo.ORDERS
     WHERE CAST(LAST_MODIFIED_DT AS DATE) = CAST(GETDATE()-1 AS DATE)) yesterday,
    (SELECT COUNT(*) AS n FROM source_db.dbo.ORDERS
     WHERE CAST(LAST_MODIFIED_DT AS DATE) = CAST(GETDATE() AS DATE)) today
UNION ALL SELECT 'STAGING', 'ROW_COUNT',
    (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
     WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE()-1 AS DATE)),
    (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
     WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE() AS DATE)),

```

```

(SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
 WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE() AS DATE)) -
(SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
 WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE()-1 AS DATE)),
ROUND(100.0 * (
  (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
   WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE() AS DATE)) -
  (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
   WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE()-1 AS DATE))
) / NULLIF(
  (SELECT COUNT(*) FROM staging.dbo.ORDERS_STAGE
   WHERE CAST(LOADED_AT AS DATE) = CAST(GETDATE()-1 AS DATE)), 0), 2),
'SEE ABOVE'
UNION ALL SELECT 'WAREHOUSE', 'TOTAL_REVENUE',
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE()-1 AS DATE)),
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE() AS DATE)),
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE() AS DATE)) -
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE()-1 AS DATE)),
ROUND(100.0 * (
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE() AS DATE)) -
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE()-1 AS DATE))
) / NULLIF(
  (SELECT SUM(AMOUNT) FROM dwh.dbo.FACT_ORDERS
   WHERE CAST(LOAD_TIMESTAMP AS DATE) = CAST(GETDATE()-1 AS DATE)), 0), 2),
'SEE ABOVE';

```

ETL Metadata Lineage Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Metadata\_Lineage\_Validations\_Python.md`

 Metadata is data about your data. Lineage is the proof of where it came

1. Column-Level Lineage Validation

Purpose: Every column in the warehouse must be traceable back to a

source column through every transformation step. Orphaned columns (no lineage), broken lineage (source column renamed or dropped), and undocumented derivations are all lineage failures that block regulatory compliance and make debugging impossible.

```
-- — Column lineage completeness: every WH column has a source —
SELECT
  t.TABLE_NAME,
  t.COLUMN_NAME,
  l.source_table,
  l.source_column,
  l.transformation_type,    -- DIRECT, DERIVED, LOOKUP, CONSTANT
  l.transformation_logic,
  CASE
    WHEN l.target_column IS NULL
    THEN 'NO LINEAGE ✖'
    WHEN l.source_column IS NULL
      AND l.transformation_type NOT IN ('CONSTANT','DERIVED')
    THEN 'MISSING SOURCE ✖'
    ELSE 'DOCUMENTED ✔'
  END AS lineage_status
FROM INFORMATION_SCHEMA.COLUMNS t
LEFT JOIN etl_metadata.dbo.COLUMN_LINEAGE l
  ON l.target_table = t.TABLE_NAME
  AND l.target_column = t.COLUMN_NAME
WHERE t.TABLE_NAME = 'FACT_ORDERS'
  AND t.TABLE_SCHEMA = 'dbo'
ORDER BY
  CASE WHEN l.target_column IS NULL THEN 1 ELSE 2 END,
  t.ORDINAL_POSITION;

-- — Broken lineage: source column no longer exists —
SELECT
  l.target_table,
  l.target_column,
  l.source_table,
  l.source_column,
  l.transformation_type,
  CASE
    WHEN src.COLUMN_NAME IS NULL THEN 'SOURCE COLUMN DROPPED ✖'
    ELSE 'SOURCE INTACT ✔'
  END AS lineage_integrity
FROM etl_metadata.dbo.COLUMN_LINEAGE l
LEFT JOIN INFORMATION_SCHEMA.COLUMNS src
  ON src.TABLE_NAME = l.source_table
  AND src.COLUMN_NAME = l.source_column
WHERE l.target_table = 'FACT_ORDERS'
  AND l.source_column IS NOT NULL
  AND src.COLUMN_NAME IS NULL;    -- source column gone

-- — Multi-hop lineage depth: source → stage → transform → WH —
WITH lineage_hops AS (
  SELECT
    target_table,
    target_column,
    source_table,
```

```

        source_column,
        transformation_type,
        1 AS hop_level,
        CAST(source_table + '.' + source_column AS VARCHAR(MAX)) AS lineage_path
FROM etl_metadata.dbo.COLUMN_LINEAGE
WHERE target_table = 'FACT_ORDERS'
UNION ALL
SELECT
    l.target_table,
    l.target_column,
    l.source_table,
    l.source_column,
    l.transformation_type,
    h.hop_level + 1,
    h.lineage_path + ' → ' + l.source_table + '.' + l.source_column
FROM etl_metadata.dbo.COLUMN_LINEAGE l
JOIN lineage_hops h
    ON l.target_table = h.source_table
    AND l.target_column = h.source_column
WHERE h.hop_level < 10 -- prevent infinite loops
)
SELECT
    target_column,
    MAX(hop_level) AS lineage_depth,
    MIN(lineage_path) AS full_lineage_path
FROM lineage_hops
GROUP BY target_column
ORDER BY lineage_depth DESC;
-- — Coverage summary: % of columns with documented lineage —
SELECT
    t.TABLE_NAME,
    COUNT(t.COLUMN_NAME) AS total_columns,
    COUNT(l.target_column) AS documented_columns,
    COUNT(t.COLUMN_NAME) - COUNT(l.target_column) AS undocumented,
    ROUND(100.0 * COUNT(l.target_column)
        / NULLIF(COUNT(t.COLUMN_NAME), 0), 2) AS coverage_pct,
    CASE
        WHEN ROUND(100.0 * COUNT(l.target_column)
            / NULLIF(COUNT(t.COLUMN_NAME), 0), 2) = 100
        THEN 'FULL COVERAGE '
        WHEN ROUND(100.0 * COUNT(l.target_column)
            / NULLIF(COUNT(t.COLUMN_NAME), 0), 2) >= 80
        THEN 'PARTIAL '
        ELSE 'POOR COVERAGE '
    END AS coverage_status
FROM INFORMATION_SCHEMA.COLUMNS t
LEFT JOIN etl_metadata.dbo.COLUMN_LINEAGE l
    ON l.target_table = t.TABLE_NAME
    AND l.target_column = t.COLUMN_NAME
WHERE t.TABLE_SCHEMA = 'dbo'
    AND t.TABLE_NAME IN ('FACT_ORDERS', 'DIM_CUSTOMERS', 'DIM_PRODUCTS')
GROUP BY t.TABLE_NAME
ORDER BY coverage_pct ASC;

```

2. Catalog Completeness Validation

Purpose: The data catalog is the single source of truth for what exists,

what it means, and who owns it. An incomplete catalog means analysts query the wrong table, engineers duplicate datasets, and governance teams cannot map sensitive data. Validate that every table and column has the mandatory catalog attributes populated.

```

-- — Table-level catalog completeness —
SELECT
    t.TABLE_NAME,
    c.table_description,
    c.data_owner,

```

```

c.data_steward,
c.business_domain,
c.data_classification,    -- PUBLIC, INTERNAL, CONFIDENTIAL, PII
c.refresh_frequency,
c.last_updated_by,
c.last_updated_at,
-- Score each attribute
(CASE WHEN c.table_description IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_owner        IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_steward      IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.business_domain   IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_classification IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.refresh_frequency IS NOT NULL THEN 1 ELSE 0 END
)
)                                AS attributes_populated,
6                                AS total_attributes,
ROUND(100.0 *
(CASE WHEN c.table_description IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_owner        IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_steward      IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.business_domain   IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_classification IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.refresh_frequency IS NOT NULL THEN 1 ELSE 0 END
) / 6.0, 2)                      AS completeness_pct,
CASE
    WHEN (CASE WHEN c.table_description IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_owner        IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_steward      IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.business_domain   IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_classification IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.refresh_frequency IS NOT NULL THEN 1 ELSE 0 END
) = 6                        THEN 'COMPLETE ✓'
    WHEN (CASE WHEN c.table_description IS NOT NULL THEN 1 ELSE 0 END +
CASE WHEN c.data_owner        IS NOT NULL THEN 1 ELSE 0 END
) < 2                        THEN 'CRITICAL GAPS ✗'
    ELSE                      'PARTIAL △'
END                            AS catalog_status
FROM INFORMATION_SCHEMA.TABLES t
LEFT JOIN etl_metadata.dbo.TABLE_CATALOG c
    ON c.table_name = t.TABLE_NAME
WHERE t.TABLE_SCHEMA = 'dbo'
    AND t.TABLE_TYPE = 'BASE TABLE'
ORDER BY completeness_pct ASC;
-- — Column-level catalog: business definitions populated ———
SELECT
    c.TABLE_NAME,
    c.COLUMN_NAME,
    c.DATA_TYPE,
    cat.business_definition,
    cat.example_values,
    cat.is_pii,
    cat.is_nullable_expected,
    CASE
        WHEN cat.COLUMN_NAME IS NULL          THEN 'NOT IN CATALOG ✗'
        WHEN cat.business_definition IS NULL THEN 'NO DEFINITION ✗'
        WHEN cat.is_pii IS NULL                THEN 'PII FLAG MISSING △'
        ELSE                                  'DOCUMENTED ✓'
    END AS column_catalog_status
FROM INFORMATION_SCHEMA.COLUMNS c
LEFT JOIN etl_metadata.dbo.COLUMN_CATALOG cat
    ON cat.table_name = c.TABLE_NAME
    AND cat.column_name = c.COLUMN_NAME
WHERE c.TABLE_NAME = 'FACT_ORDERS'
    AND c.TABLE_SCHEMA = 'dbo'
ORDER BY
    CASE WHEN cat.COLUMN_NAME IS NULL THEN 1
        WHEN cat.business_definition IS NULL THEN 2
        ELSE 3 END,
    c.ORDINAL_POSITION;

```

```
-- — Stale catalog: last updated > 90 days ago —————
SELECT
    table_name,
    last_updated_at,
    last_updated_by,
    DATEDIFF(DAY, last_updated_at, GETDATE()) AS days_since_update,
    CASE
        WHEN DATEDIFF(DAY, last_updated_at, GETDATE()) > 180 THEN 'VERY STALE ✕'
        WHEN DATEDIFF(DAY, last_updated_at, GETDATE()) > 90 THEN 'STALE ⚠'
        ELSE 'CURRENT ✔'
    END AS staleness_status
FROM etl_metadata.dbo.TABLE_CATALOG
ORDER BY days_since_update DESC;
```

3. Audit Trail Validation

Purpose: Every data change — INSERT, UPDATE, DELETE — must be recorded

with who did it, when, what changed, and why. Audit trails are mandatory for SOX, GDPR, HIPAA, and any regulated industry. Validate that the audit log is complete, tamper-evident, and queryable.

```
-- — Audit log coverage: every table change has an audit entry —
SELECT
    dl.operation_type,
    dl.table_name,
    dl.affected_rows,
    dl.executed_by,
    dl.executed_at,
    dl.pipeline_run_id,
    dl.batch_id,
    r.run_status,
    CASE
        WHEN r.run_id IS NULL THEN 'ORPHAN AUDIT — NO RUN ✕'
        WHEN dl.affected_rows
            IS NULL THEN 'MISSING ROW COUNT ⚠'
        ELSE 'VALID AUDIT ENTRY ✔'
    END AS audit_validity
FROM etl_control.dbo.DML_AUDIT_LOG dl
LEFT JOIN etl_control.dbo.PIPELINE_RUN_LOG r
    ON dl.pipeline_run_id = r.run_id
WHERE dl.executed_at >= DATEADD(DAY, -7, GETDATE())
ORDER BY dl.executed_at DESC;

-- — Audit completeness: every pipeline run has audit entries —
SELECT
    r.run_id,
    r.pipeline_name,
    r.batch_date,
    r.run_status,
    COUNT(dl.audit_id) AS audit_entries,
    SUM(dl.affected_rows) AS total_rows_audited,
    CASE
        WHEN COUNT(dl.audit_id) = 0 THEN 'NO AUDIT TRAIL ✕'
        WHEN SUM(dl.affected_rows) IS NULL THEN 'NULL ROW COUNTS ⚠'
        ELSE 'COMPLETE ✔'
    END AS audit_completeness
FROM etl_control.dbo.PIPELINE_RUN_LOG r
LEFT JOIN etl_control.dbo.DML_AUDIT_LOG dl ON dl.pipeline_run_id = r.run_id
WHERE r.batch_date >= DATEADD(DAY, -7, GETDATE())
GROUP BY r.run_id, r.pipeline_name, r.batch_date, r.run_status
ORDER BY r.batch_date DESC;

-- — Tamper detection: audit log row hash chain —————
-- Each audit entry includes a hash of the previous entry
-- (blockchain-style) to detect tampering
SELECT
    a.audit_id,
    a.executed_at,
```

```

a.table_name,
a.operation_type,
a.affected_rows,
a.entry_hash,
a.prev_entry_hash,
HASHBYTES('SHA2_256', CONCAT(
    CAST(LAG(a.audit_id) OVER (ORDER BY a.audit_id) AS VARCHAR), '|',
    LAG(a.table_name) OVER (ORDER BY a.audit_id), '|',
    LAG(a.operation_type) OVER (ORDER BY a.audit_id), '|',
    CAST(LAG(a.affected_rows) OVER (ORDER BY a.audit_id) AS VARCHAR)
))
AS expected_prev_hash,
CASE
    WHEN a.prev_entry_hash =
        HASHBYTES('SHA2_256', CONCAT(
            CAST(LAG(a.audit_id) OVER (ORDER BY a.audit_id) AS VARCHAR), '|',
            LAG(a.table_name) OVER (ORDER BY a.audit_id), '|',
            LAG(a.operation_type) OVER (ORDER BY a.audit_id), '|',
            CAST(LAG(a.affected_rows) OVER (ORDER BY a.audit_id) AS VARCHAR)
        ))
    THEN 'CHAIN INTACT ☒'
    ELSE 'CHAIN BROKEN — POSSIBLE TAMPERING ✕'
END AS tamper_check
FROM etl_control.dbo.DML_AUDIT_LOG a
ORDER BY a.audit_id;
-- — Data change history: what changed for a specific record —
SELECT
    audit_id,
    table_name,
    record_pk,
    operation_type,
    column_name,
    old_value,
    new_value,
    changed_by,
    changed_at,
    pipeline_run_id
FROM etl_control.dbo.FIELD_CHANGE_LOG
WHERE table_name = 'FACT_ORDERS'
    AND record_pk = '12345'
ORDER BY changed_at;

```

4. Schema Metadata Validation

Purpose: Schema metadata — data types, constraints, default values,

indexes, primary keys — must match the design specification at all times.

Undetected schema drift causes silent type coercions, truncations, and incorrect query results at TB scale.

```

-- — Schema vs design specification: full diff —————
SELECT
    spec.table_name,
    spec.column_name,
    spec.expected_type AS spec_type,
    spec.expected_nullable AS spec_nullable,
    spec.expected_max_length AS spec_length,
    spec.expected_precision AS spec_precision,
    spec.expected_scale AS spec_scale,
    curr.DATA_TYPE AS actual_type,
    curr.IS_NULLABLE AS actual_nullable,
    curr.CHARACTER_MAXIMUM_LENGTH AS actual_length,
    curr.NUMERIC_PRECISION AS actual_precision,
    curr.NUMERIC_SCALE AS actual_scale,
    CASE
        WHEN curr.COLUMN_NAME IS NULL
        THEN 'COLUMN MISSING ✕'
        WHEN spec.expected_type <> curr.DATA_TYPE
        THEN 'TYPE MISMATCH ✕'
    END

```

```

        WHEN spec.expected_nullable <> curr.IS_NULLABLE
        THEN 'NULLABLE MISMATCH ⚠'
        WHEN ISNULL(spec.expected_max_length, 0) >
            ISNULL(curr.CHARACTER_MAXIMUM_LENGTH, 0)
            AND curr.CHARACTER_MAXIMUM_LENGTH IS NOT NULL
        THEN 'LENGTH REDUCED ✖'
        WHEN ISNULL(spec.expected_scale, 0) <>
            ISNULL(curr.NUMERIC_SCALE, 0)
        THEN 'PRECISION CHANGED ⚠'
        ELSE 'MATCHES SPEC ✔'
    END AS schema_drift_status
FROM etl_metadata.dbo.SCHEMA_SPECIFICATION spec
LEFT JOIN INFORMATION_SCHEMA.COLUMNS curr
    ON curr.TABLE_NAME = spec.table_name
    AND curr.COLUMN_NAME = spec.column_name
WHERE spec.table_name = 'FACT_ORDERS'
ORDER BY
    CASE
        WHEN curr.COLUMN_NAME IS NULL THEN 1
        WHEN spec.expected_type <> curr.DATA_TYPE THEN 2
        ELSE 3
    END;
-- — Index metadata: required indexes exist —————
SELECT
    spec.table_name,
    spec.index_name,
    spec.index_columns,
    spec.is_unique          AS spec_unique,
    idx.is_unique           AS actual_unique,
    idx.type_desc           AS index_type,
    CASE
        WHEN idx.name IS NULL THEN 'INDEX MISSING ✖'
        WHEN spec.is_unique = 1
            AND idx.is_unique = 0 THEN 'UNIQUENESS DROPPED ✖'
        ELSE 'INDEX OK ✔'
    END AS index_status
FROM etl_metadata.dbo.INDEX_SPECIFICATION spec
LEFT JOIN sys.indexes idx
    ON OBJECT_NAME(idx.object_id) = spec.table_name
    AND idx.name = spec.index_name
WHERE spec.table_name = 'FACT_ORDERS';
-- — Table statistics freshness —————
SELECT
    OBJECT_NAME(s.object_id) AS table_name,
    s.name                   AS stat_name,
    sp.last_updated,
    sp.rows,
    sp.rows_sampled,
    ROUND(100.0 * sp.rows_sampled / NULLIF(sp.rows, 0), 2) AS sample_pct,
    DATEDIFF(DAY, sp.last_updated, GETDATE())              AS days_stale,
    CASE
        WHEN DATEDIFF(DAY, sp.last_updated, GETDATE()) > 7
        THEN 'STALE STATS ✖ — QUERY PLANS UNRELIABLE'
        WHEN DATEDIFF(DAY, sp.last_updated, GETDATE()) > 3
        THEN 'AGING STATS ⚠'
        ELSE 'FRESH ✔'
    END AS stats_status
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE OBJECT_NAME(s.object_id) IN ('FACT_ORDERS', 'DIM_CUSTOMERS')
ORDER BY days_stale DESC;

```

5. Pipeline Run Metadata Validation

Purpose: Every pipeline execution must leave a complete, accurate

metadata footprint — start time, end time, row counts, status, batch ID,

watermarks. This metadata is the backbone of debugging, SLA reporting, reprocessing, and capacity planning.

```
-- — Run log completeness: all mandatory fields populated ———
SELECT
    run_id,
    pipeline_name,
    batch_date,
    run_start,
    run_end,
    run_status,
    rows_processed,
    rows_inserted,
    rows_updated,
    rows_rejected,
    bytes_processed,
    load_batch_id,
    watermark_start,
    watermark_end,
    -- Score completeness
    (CASE WHEN run_start      IS NOT NULL THEN 1 ELSE 0 END +
     CASE WHEN run_end       IS NOT NULL THEN 1 ELSE 0 END +
     CASE WHEN run_status    IS NOT NULL THEN 1 ELSE 0 END +
     CASE WHEN rows_processed IS NOT NULL THEN 1 ELSE 0 END +
     CASE WHEN load_batch_id IS NOT NULL THEN 1 ELSE 0 END +
     CASE WHEN watermark_end IS NOT NULL THEN 1 ELSE 0 END
    )
    AS fields_populated,
    CASE
        WHEN run_start      IS NULL THEN 'MISSING START TIME ✕'
        WHEN run_end        IS NULL
            AND run_status = 'SUCCESS' THEN 'MISSING END TIME ✕'
        WHEN rows_processed IS NULL THEN 'MISSING ROW COUNT ⚠'
        WHEN load_batch_id  IS NULL THEN 'MISSING BATCH ID ⚠'
        ELSE                      'COMPLETE ✔'
    END AS run_metadata_status
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE batch_date >= DATEADD(DAY, -7, GETDATE())
ORDER BY batch_date DESC, run_start DESC;
-- — Batch ID continuity: no gaps in batch sequence ———
SELECT
    load_batch_id,
    LAG(load_batch_id) OVER (ORDER BY load_batch_id) AS prev_batch_id,
    load_batch_id -
    LAG(load_batch_id) OVER (ORDER BY load_batch_id) AS gap,
    CASE
        WHEN load_batch_id -
            LAG(load_batch_id) OVER (ORDER BY load_batch_id) = 1
        THEN 'SEQUENTIAL ✔'
        WHEN LAG(load_batch_id) OVER (ORDER BY load_batch_id) IS NULL
        THEN 'FIRST BATCH'
        ELSE 'GAP DETECTED ✕'
    END AS batch_continuity
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE pipeline_name = 'ORDERS_ETL'
  AND run_status    = 'SUCCESS'
ORDER BY load_batch_id;
-- — Row count reconciliation across run log ———
SELECT
    r.run_id,
    r.rows_inserted + r.rows_updated AS run_log_total,
    COUNT(f.ORDER_KEY)              AS actual_rows_in_target,
    ABS(r.rows_inserted + r.rows_updated -
        COUNT(f.ORDER_KEY))        AS row_count_discrepancy,
    CASE
        WHEN ABS(r.rows_inserted + r.rows_updated -
            COUNT(f.ORDER_KEY)) = 0 THEN 'MATCHES ✔'
        ELSE                      'DISCREPANCY ✕'
    END AS row_count_accuracy
```

```
FROM etl_control.dbo.PIPELINE_RUN_LOG r
JOIN dwh.dbo.FACT_ORDERS f
  ON f.LOAD_BATCH_ID = r.load_batch_id
WHERE r.batch_date >= DATEADD(DAY, -7, GETDATE())
GROUP BY r.run_id, r.rows_inserted, r.rows_updated;
```

6. Data Classification & Sensitivity Metadata

Purpose: Every column containing sensitive data — PII, financial,

health, credentials — must be classified, and that classification must

be enforced through masking, access controls, and retention policies.

Unclassified sensitive columns are a GDPR/CCPA/HIPAA violation waiting

to happen. Validate that all sensitive columns are identified, tagged,

and properly governed.

```
-- — PII column classification completeness —————
SELECT
  c.TABLE_NAME,
  c.COLUMN_NAME,
  c.DATA_TYPE,
  cat.is_pii,
  cat.data_classification,    -- PII, SENSITIVE, INTERNAL, PUBLIC
  cat.pii_category,          -- NAME, EMAIL, PHONE, SSN, DOB, ADDRESS
  cat.masking_rule,
  cat.retention_days,
  CASE
    WHEN cat.COLUMN_NAME IS NULL      THEN 'NOT CLASSIFIED ✕'
    WHEN cat.is_pii = 1
      AND cat.masking_rule IS NULL    THEN 'PII WITHOUT MASKING ✕'
    WHEN cat.is_pii = 1
      AND cat.masking_rule IS NOT NULL THEN 'PII MASKED ✔'
    WHEN cat.is_pii = 0              THEN 'NON-PII ✔'
    ELSE                             'CLASSIFICATION INCOMPLETE ⚠'
  END AS classification_status
FROM INFORMATION_SCHEMA.COLUMNS c
LEFT JOIN etl_metadata.dbo.COLUMN_CATALOG cat
  ON cat.table_name = c.TABLE_NAME
  AND cat.column_name = c.COLUMN_NAME
WHERE c.TABLE_SCHEMA = 'dbo'
  AND c.TABLE_NAME IN ('FACT_ORDERS', 'DIM_CUSTOMERS', 'DIM_EMPLOYEES')
ORDER BY
  CASE WHEN cat.COLUMN_NAME IS NULL THEN 1
        WHEN cat.is_pii=1 AND cat.masking_rule IS NULL THEN 2
        ELSE 3 END,
  c.TABLE_NAME, c.ORDINAL_POSITION;

-- — Auto-detect potential PII by column name pattern —————
SELECT
  TABLE_NAME,
  COLUMN_NAME,
  DATA_TYPE,
  'POTENTIAL PII – REVIEW REQUIRED ⚠' AS flag
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
  AND (
    COLUMN_NAME LIKE '%EMAIL%'
  OR COLUMN_NAME LIKE '%PHONE%'
  OR COLUMN_NAME LIKE '%SSN%'
  OR COLUMN_NAME LIKE '%SOCIAL%'
  OR COLUMN_NAME LIKE '%PASSPORT%'
  OR COLUMN_NAME LIKE '%DOB%'
  OR COLUMN_NAME LIKE '%BIRTH%'
  OR COLUMN_NAME LIKE '%ADDRESS%'
  OR COLUMN_NAME LIKE '%SALARY%'
  OR COLUMN_NAME LIKE '%CREDIT%'
  OR COLUMN_NAME LIKE '%ACCOUNT%'
  OR COLUMN_NAME LIKE '%LICENSE%'
```

```

    OR COLUMN_NAME LIKE '%IP_ADDR%'
  )
  AND COLUMN_NAME NOT IN (
    SELECT column_name FROM etl_metadata.dbo.COLUMN_CATALOG
    WHERE is_pii IS NOT NULL
  )
ORDER BY TABLE_NAME, COLUMN_NAME;
-- — Retention policy: tables with data beyond retention window —
SELECT
  c.table_name,
  c.retention_days,
  DATEDIFF(DAY,
    MIN(f.ORDER_DATE),
    GETDATE()) AS oldest_data_days,
  CASE
    WHEN DATEDIFF(DAY, MIN(f.ORDER_DATE), GETDATE())
      > c.retention_days THEN 'RETENTION BREACH ✖'
    ELSE 'WITHIN RETENTION ✔'
  END AS retention_status
FROM etl_metadata.dbo.TABLE_CATALOG c
JOIN dwh.dbo.FACT_ORDERS f ON f.ORDER_DATE IS NOT NULL
WHERE c.table_name = 'FACT_ORDERS'
GROUP BY c.table_name, c.retention_days;
Reporting / BI Validation

```

ETL Reporting BI Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Reporting\_BI\_Validations\_Python.md`

📌 The report layer is the only layer that business users actually see.

1. Report vs Warehouse Data Match

Purpose: Every number on a report must be traceable to — and exactly

match — the underlying warehouse fact table. Rounding, currency conversion, fiscal calendar offsets, and BI tool aggregation settings all introduce silent discrepancies. Validate that report layer values match independently computed warehouse values for every key metric.

```
-- — Daily revenue: report vs warehouse —————
SELECT
  r.report_date,
  r.daily_revenue          AS report_revenue,
  f.warehouse_revenue      AS warehouse_revenue,
  ABS(r.daily_revenue -
    f.warehouse_revenue)   AS abs_diff,
  ROUND(100.0 * ABS(r.daily_revenue - f.warehouse_revenue)
    / NULLIF(f.warehouse_revenue, 0), 6) AS diff_pct,
  CASE
    WHEN ABS(r.daily_revenue -
      f.warehouse_revenue) < 0.01 THEN 'MATCH ✅'
    WHEN ABS(r.daily_revenue -
      f.warehouse_revenue) < 1.00 THEN 'ROUNDING ⚠️'
    ELSE                               'MISMATCH ❌'
  END AS match_status
FROM bi_layer.dbo.RPT_DAILY_REVENUE r
JOIN (
  SELECT
    CAST(order_date AS DATE) AS report_date,
    SUM(net_revenue)        AS warehouse_revenue
  FROM dwh.dbo.FACT_ORDERS
  GROUP BY CAST(order_date AS DATE)
) f ON r.report_date = f.report_date
WHERE r.report_date >= DATEADD(DAY, -30, GETDATE())
  AND ABS(r.daily_revenue - f.warehouse_revenue) >= 0.01
ORDER BY abs_diff DESC;

-- — Multi-metric report vs warehouse: MTD snapshot —————
SELECT
  metric,
  report_value,
  warehouse_value,
  ABS(report_value - warehouse_value) AS abs_diff,
  CASE
    WHEN ABS(report_value - warehouse_value) < 0.01 THEN 'MATCH ✅'
    ELSE                               'MISMATCH ❌'
  END AS status
FROM (
  SELECT 'MTD Revenue' AS metric,
    (SELECT SUM(daily_revenue) FROM bi_layer.dbo.RPT_DAILY_REVENUE
     WHERE report_date BETWEEN
       DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
       AND CAST(GETDATE()-1 AS DATE)) AS report_value,
    (SELECT SUM(net_revenue) FROM dwh.dbo.FACT_ORDERS
     WHERE order_date BETWEEN
       DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
       AND CAST(GETDATE()-1 AS DATE)) AS warehouse_value
  UNION ALL
  SELECT 'MTD Orders',
```

```

        (SELECT SUM(order_count)
         FROM bi_layer.dbo.RPT_DAILY_REVENUE
         WHERE report_date BETWEEN
              DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
              AND CAST(GETDATE()-1 AS DATE)),
        (SELECT COUNT(*)
         FROM dwh.dbo.FACT_ORDERS
         WHERE order_date BETWEEN
              DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
              AND CAST(GETDATE()-1 AS DATE))
UNION ALL
SELECT 'MTD Avg Order Value',
        (SELECT AVG(avg_order_value)
         FROM bi_layer.dbo.RPT_DAILY_REVENUE
         WHERE report_date BETWEEN
              DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
              AND CAST(GETDATE()-1 AS DATE)),
        (SELECT AVG(net_revenue)
         FROM dwh.dbo.FACT_ORDERS
         WHERE order_date BETWEEN
              DATEADD(DAY, 1-DAY(GETDATE()), CAST(GETDATE() AS DATE))
              AND CAST(GETDATE()-1 AS DATE))
) metrics;
-- — Row-level report vs warehouse: 100 random record spot check —
SELECT
    r.order_id,
    r.displayed_revenue      AS report_revenue,
    f.net_revenue            AS warehouse_revenue,
    r.displayed_status       AS report_status,
    f.status                 AS warehouse_status,
    r.displayed_customer     AS report_customer,
    d.customer_name          AS warehouse_customer,
    CASE
        WHEN ABS(r.displayed_revenue - f.net_revenue) < 0.01
         AND r.displayed_status = f.status
         AND r.displayed_customer = d.customer_name THEN 'MATCH' ✓
        WHEN ABS(r.displayed_revenue - f.net_revenue) >= 0.01 THEN 'REVENUE MISMATCH ✗'
        WHEN r.displayed_status <> f.status THEN 'STATUS MISMATCH ✗'
        ELSE 'CUSTOMER MISMATCH ✗'
    END AS row_match_status
FROM (SELECT TOP 100 * FROM bi_layer.dbo.RPT_ORDER_DETAIL ORDER BY NEWID()) r
JOIN dwh.dbo.FACT_ORDERS f ON r.order_id = f.source_order_id
JOIN dwh.dbo.DIM_CUSTOMERS d ON f.customer_key = d.customer_key
WHERE ABS(r.displayed_revenue - f.net_revenue) >= 0.01
    OR r.displayed_status <> f.status
    OR r.displayed_customer <> d.customer_name;

```

2. Filter Logic Validation

Purpose: BI report filters — date ranges, status dropdowns, region

selectors — must apply correctly to the underlying data. A filter that includes extra rows, excludes valid rows, or silently fails (returning all data unfiltered) produces wrong numbers without any error message.

Validate every filter type used in production reports.

```

-- — Date filter accuracy: only selected period rows returned —
SELECT
    COUNT(*)                AS report_rows,
    MIN(order_date)          AS earliest_date,
    MAX(order_date)          AS latest_date,
    SUM(CASE WHEN order_date < '2024-12-01'
              OR order_date > '2024-12-31'
              THEN 1 ELSE 0 END) AS out_of_range_rows,
    CASE
        WHEN SUM(CASE WHEN order_date < '2024-12-01'
                      OR order_date > '2024-12-31'

```

```

        THEN 1 ELSE 0 END) = 0 THEN 'FILTER CORRECT ✓'
    ELSE
        'FILTER LEAKS DATA ✗'
    END AS date_filter_status
FROM bi_layer.dbo.RPT_ORDER_DETAIL
WHERE report_period = '2024-12';    -- simulates user's Dec 2024 filter
-- — Status filter: only selected statuses in result —————
SELECT
    status,
    COUNT(*) AS row_count
FROM bi_layer.dbo.RPT_ORDER_DETAIL
WHERE status_filter = 'DELIVERED' -- user selected DELIVERED
GROUP BY status;
-- Must return exactly 1 row: DELIVERED
-- Any other status in result = filter bug
-- — Multi-select filter: multiple values all included —————
-- User selects: status IN ('DELIVERED', 'SHIPPED')
SELECT
    status,
    COUNT(*) AS row_count
FROM bi_layer.dbo.RPT_ORDER_DETAIL
WHERE status IN ('DELIVERED', 'SHIPPED')
GROUP BY status;
-- Must return exactly 2 rows: DELIVERED and SHIPPED only
-- — NULL filter: "All" selection includes NULL values —————
SELECT
    COUNT(*) AS all_orders,
    SUM(CASE WHEN region IS NULL
        THEN 1 ELSE 0 END) AS null_region_orders,
    COUNT(DISTINCT region) AS distinct_regions
FROM bi_layer.dbo.RPT_ORDER_DETAIL
WHERE region_filter = 'ALL';    -- ALL filter must include NULLs
-- null_region_orders must be > 0 if source has NULL regions
-- — Cross-filter interaction: date AND region applied —————
SELECT
    COUNT(*) AS filtered_rows,
    MIN(order_date) AS min_date,
    MAX(order_date) AS max_date,
    COUNT(DISTINCT region) AS distinct_regions,
    SUM(CASE WHEN region <> 'US'
        THEN 1 ELSE 0 END) AS non_us_rows,
    SUM(CASE WHEN order_date
        NOT BETWEEN '2024-12-01' AND '2024-12-31'
        THEN 1 ELSE 0 END) AS out_of_date_rows
FROM bi_layer.dbo.RPT_ORDER_DETAIL
WHERE report_period = '2024-12'
    AND region_filter = 'US';
-- non_us_rows = 0 AND out_of_date_rows = 0

```

3. Aggregation Accuracy

Purpose: BI tools aggregate data at query time — SUM, COUNT, AVG,

DISTINCT COUNT, running totals, YoY comparisons. Every aggregation is an opportunity for error: double-counting from fan-out joins, AVG of AVGs, COUNT DISTINCT over pre-aggregated tables, and fiscal-year offset errors. Validate that every aggregate on every report matches independently computed values.

```

-- — SUM accuracy: revenue cannot double-count —————
SELECT
    'Report SUM' AS source,
    SUM(daily_revenue) AS total_revenue,
    COUNT(*) AS row_count
FROM bi_layer.dbo.RPT_DAILY_REVENUE
WHERE report_date BETWEEN '2024-01-01' AND '2024-12-31'
UNION ALL
SELECT

```

```

    'Warehouse SUM',
    SUM(net_revenue),
    COUNT(*)
FROM dwh.dbo.FACT_ORDERS
WHERE YEAR(order_date) = 2024;
-- — AVG accuracy: weighted vs simple average —————
-- WRONG: AVG of daily AVGs ≠ true period average
SELECT
    AVG(avg_order_value)    AS wrong_avg    -- AVG of AVGs
FROM bi_layer.dbo.RPT_DAILY_REVENUE
WHERE report_date BETWEEN '2024-12-01' AND '2024-12-31';
-- CORRECT: true period average
SELECT
    SUM(net_revenue) /
    NULLIF(COUNT(*), 0)    AS correct_avg
FROM dwh.dbo.FACT_ORDERS
WHERE order_date BETWEEN '2024-12-01' AND '2024-12-31';
-- — COUNT DISTINCT: unique customers —————
SELECT
    'Report'                AS source,
    unique_customers        AS distinct_customers
FROM bi_layer.dbo.RPT_MONTHLY_SUMMARY
WHERE month_key = '2024-12'
UNION ALL
SELECT
    'Warehouse',
    COUNT(DISTINCT customer_key)
FROM dwh.dbo.FACT_ORDERS
WHERE FORMAT(order_date, 'yyyy-MM') = '2024-12';
-- — Running total accuracy: cumulative revenue —————
SELECT
    r.report_date,
    r.cumulative_revenue    AS report_running_total,
    SUM(f.daily_revenue)
        OVER (ORDER BY f.order_date
              ROWS BETWEEN UNBOUNDED PRECEDING
              AND CURRENT ROW) AS warehouse_running_total,
    ABS(r.cumulative_revenue -
        SUM(f.daily_revenue)
            OVER (ORDER BY f.order_date
                  ROWS BETWEEN UNBOUNDED PRECEDING
                  AND CURRENT ROW)) AS running_total_diff
FROM bi_layer.dbo.RPT_DAILY_REVENUE r
JOIN (
    SELECT CAST(order_date AS DATE) AS order_date,
           SUM(net_revenue)        AS daily_revenue
    FROM dwh.dbo.FACT_ORDERS
    GROUP BY CAST(order_date AS DATE)
) f ON r.report_date = f.order_date
WHERE r.report_date BETWEEN '2024-01-01' AND '2024-12-31'
    AND ABS(r.cumulative_revenue -
        SUM(f.daily_revenue) OVER (ORDER BY f.order_date
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)) > 0.01
ORDER BY running_total_diff DESC;
-- — YoY comparison: same period last year alignment —————
SELECT
    r.current_year_revenue,
    r.prior_year_revenue,
    r.yoy_growth_pct        AS report_yoy_pct,
    -- Independently compute YoY
    ROUND(100.0 *
        (cy.revenue - py.revenue)
        / NULLIF(py.revenue, 0), 2) AS warehouse_yoy_pct,
    ABS(r.yoy_growth_pct -
        ROUND(100.0 * (cy.revenue - py.revenue)
            / NULLIF(py.revenue, 0), 2)) AS yoy_diff
FROM bi_layer.dbo.RPT_YOY_SUMMARY r,
(SELECT SUM(net_revenue) AS revenue FROM dwh.dbo.FACT_ORDERS
 WHERE YEAR(order_date) = YEAR(GETDATE())) cy,

```

```
(SELECT SUM(net_revenue) AS revenue FROM dwh.dbo.FACT_ORDERS
WHERE YEAR(order_date) = YEAR(GETDATE()) - 1) py
WHERE ABS(r.yoy_growth_pct -
ROUND(100.0 * (cy.revenue - py.revenue)
/ NULLIF(py.revenue, 0), 2)) > 0.01;
```

4. Drill-Down Validation

Purpose: Drill-down is the most-used BI interaction: click "December

Revenue" → see by Region → see by Product → see individual orders.

At every level, the sum of children must equal the parent, no records

must appear at a level they don't belong to, and the leaf-level detail

must match the underlying fact table row by row.

```
-- — Level 1→2: YTD total = sum of monthly totals —————
SELECT
    ytd.ytd_revenue                AS level1_ytd,
    SUM(m.monthly_revenue)         AS level2_sum_of_months,
    ABS(ytd.ytd_revenue -
        SUM(m.monthly_revenue))    AS drill_diff,
    CASE
        WHEN ABS(ytd.ytd_revenue -
            SUM(m.monthly_revenue)) < 0.01 THEN 'CONSISTENT ✓'
        ELSE 'DRILL LEAK ✗'
    END AS drill_down_status
FROM bi_layer.dbo.RPT_YTD_SUMMARY ytd,
     bi_layer.dbo.RPT_MONTHLY_REVENUE m
WHERE m.year_key = ytd.year_key
GROUP BY ytd.ytd_revenue, ytd.year_key;
-- — Level 2→3: month total = sum of daily totals —————
SELECT
    m.month_key,
    m.monthly_revenue             AS monthly_total,
    SUM(d.daily_revenue)          AS sum_of_daily,
    ABS(m.monthly_revenue -
        SUM(d.daily_revenue))     AS diff,
    CASE
        WHEN ABS(m.monthly_revenue -
            SUM(d.daily_revenue)) < 0.01 THEN 'CONSISTENT ✓'
        ELSE 'DRILL LEAK ✗'
    END AS month_day_drill_status
FROM bi_layer.dbo.RPT_MONTHLY_REVENUE m
JOIN bi_layer.dbo.RPT_DAILY_REVENUE d
    ON FORMAT(d.report_date, 'yyyy-MM') = m.month_key
WHERE m.year_key = '2024'
GROUP BY m.month_key, m.monthly_revenue
HAVING ABS(m.monthly_revenue - SUM(d.daily_revenue)) >= 0.01;
-- — Level 3→4: daily total = sum of order-level rows —————
SELECT
    d.report_date,
    d.daily_revenue               AS daily_total,
    SUM(o.displayed_revenue)      AS sum_of_orders,
    ABS(d.daily_revenue -
        SUM(o.displayed_revenue)) AS diff,
    COUNT(o.order_id)             AS order_count,
    d.order_count                 AS reported_order_count,
    CASE
        WHEN ABS(d.daily_revenue -
            SUM(o.displayed_revenue)) < 0.01
            AND d.order_count = COUNT(o.order_id) THEN 'CONSISTENT ✓'
        WHEN d.order_count <> COUNT(o.order_id) THEN 'COUNT MISMATCH ✗'
        ELSE 'REVENUE MISMATCH ✗'
    END AS daily_order_drill_status
FROM bi_layer.dbo.RPT_DAILY_REVENUE d
JOIN bi_layer.dbo.RPT_ORDER_DETAIL o
    ON CAST(o.order_date AS DATE) = d.report_date
```



```

WHERE d.report_date >= DATEADD(DAY, -7, GETDATE())
GROUP BY d.report_date, d.daily_revenue, d.order_count
HAVING ABS(d.daily_revenue - SUM(o.displayed_revenue)) >= 0.01
    OR d.order_count <> COUNT(o.order_id);
-- — Leaf level: order detail matches fact table —————
SELECT
    o.order_id,
    o.displayed_revenue      AS report_revenue,
    f.net_revenue            AS fact_revenue,
    ABS(o.displayed_revenue - f.net_revenue) AS diff
FROM bi_layer.dbo.RPT_ORDER_DETAIL o
JOIN dwh.dbo.FACT_ORDERS f ON o.order_id = f.source_order_id
WHERE ABS(o.displayed_revenue - f.net_revenue) >= 0.01
    AND o.order_date >= DATEADD(DAY, -7, GETDATE())
ORDER BY diff DESC;

```

5. Dashboard Refresh Validation

Purpose: A dashboard that looks current but serves stale data is

the most dangerous BI failure — it is invisible. Validate that scheduled refreshes completed on time, cache invalidation worked, the data visible to users reflects the latest pipeline run, and that refresh failures are detected and surfaced before users notice.

```

-- — Dashboard refresh log: all dashboards refreshed on schedule —
SELECT
    d.dashboard_name,
    d.refresh_schedule,      -- 'DAILY_06:00', 'HOURLY', 'REALTIME'
    r.last_refresh_at,
    r.refresh_status,
    r.refresh_duration_secs,
    r.rows_refreshed,
    DATEDIFF(MINUTE, r.last_refresh_at, GETDATE()) AS minutes_stale,
    d.max_stale_minutes      AS stale_sla_minutes,
    CASE
        WHEN r.dashboard_id IS NULL
        THEN 'NEVER REFRESHED ✖'
        WHEN r.refresh_status = 'FAILED'
        THEN 'LAST REFRESH FAILED ✖'
        WHEN DATEDIFF(MINUTE, r.last_refresh_at, GETDATE())
            > d.max_stale_minutes
        THEN 'STALE ✖'
        WHEN DATEDIFF(MINUTE, r.last_refresh_at, GETDATE())
            > d.max_stale_minutes * 0.8
        THEN 'APPROACHING STALE ⚠'
        ELSE 'FRESH ✔'
    END AS freshness_status
FROM bi_layer.dbo.DASHBOARD_REGISTRY d
LEFT JOIN (
    SELECT dashboard_id,
        MAX(refresh_at) AS last_refresh_at,
        MAX(CASE WHEN refresh_at = MAX(refresh_at) OVER (PARTITION BY dashboard_id)
            THEN refresh_status END) AS refresh_status,
        MAX(CASE WHEN refresh_at = MAX(refresh_at) OVER (PARTITION BY dashboard_id)
            THEN refresh_duration_secs END) AS refresh_duration_secs,
        MAX(CASE WHEN refresh_at = MAX(refresh_at) OVER (PARTITION BY dashboard_id)
            THEN rows_refreshed END) AS rows_refreshed
    FROM bi_layer.dbo.DASHBOARD_REFRESH_LOG
    GROUP BY dashboard_id
) r ON r.dashboard_id = d.dashboard_id
ORDER BY minutes_stale DESC;
-- — Data visible on dashboard = latest pipeline batch —————
SELECT
    d.dashboard_name,
    d.data_as_of_date        AS dashboard_data_date,
    p.latest_batch_date      AS pipeline_latest_date,

```

```

DATEDIFF(DAY,
  d.data_as_of_date,
  p.latest_batch_date) AS data_lag_days,
CASE
  WHEN DATEDIFF(DAY, d.data_as_of_date,
    p.latest_batch_date) = 0 THEN 'CURRENT ✓'
  WHEN DATEDIFF(DAY, d.data_as_of_date,
    p.latest_batch_date) = 1 THEN 'ONE DAY LAG △'
  ELSE 'STALE DATA ✗'
END AS data_currency_status
FROM bi_layer.dbo.DASHBOARD_METADATA d,
  (SELECT MAX(batch_date) AS latest_batch_date
   FROM etl_control.dbo.PIPELINE_RUN_LOG
   WHERE pipeline_name = 'ORDERS_ETL'
   AND run_status = 'SUCCESS') p;
-- — Cache invalidation: post-refresh query returns new data —
-- Capture revenue BEFORE pipeline + refresh, then AFTER
-- Both queries identical – result must differ after refresh
SELECT
  SUM(daily_revenue) AS total_revenue,
  MAX(report_date) AS latest_date
FROM bi_layer.dbo.RPT_DAILY_REVENUE;
-- Run this BEFORE refresh → revenue_before
-- Run this AFTER refresh → revenue_after
-- PASS: revenue_after > revenue_before (new day data visible)
-- — Refresh failure rate: last 30 days —
SELECT
  dashboard_name,
  COUNT(*) AS total_refreshes,
  SUM(CASE WHEN refresh_status = 'SUCCESS'
    THEN 1 ELSE 0 END) AS successful,
  SUM(CASE WHEN refresh_status = 'FAILED'
    THEN 1 ELSE 0 END) AS failed,
  ROUND(100.0 *
    SUM(CASE WHEN refresh_status = 'FAILED'
      THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0), 2) AS failure_rate_pct,
  AVG(refresh_duration_secs) AS avg_refresh_secs,
  MAX(refresh_duration_secs) AS max_refresh_secs
FROM bi_layer.dbo.DASHBOARD_REFRESH_LOG r
JOIN bi_layer.dbo.DASHBOARD_REGISTRY d
  ON d.dashboard_id = r.dashboard_id
WHERE r.refresh_at >= DATEADD(DAY, -30, GETDATE())
GROUP BY d.dashboard_name
ORDER BY failure_rate_pct DESC;

```

Big Data-Specific Testing Checks (Hadoop / Hive / Spark)

ETL BigData Hadoop Hive Spark Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_BigData\_Hadoop\_Hive\_Spark\_Validations\_Python.md`

 Big Data testing is not just "does the count match?" — it is understanding

1. HDFS File Availability

Purpose: Before any Hive or Spark job runs, confirm that the source

files exist in HDFS, are non-zero in size, were written completely (no partial files from failed jobs), and arrived within the expected SLA window.

A missing HDFS file produces a silent empty result — not an error.

```
-- — Check expected partitions exist via Hive SHOW PARTITIONS —
SHOW PARTITIONS orders_external;
-- Expected output must include today's partition:
-- dt=2024-12-01/region=US
-- dt=2024-12-01/region=EU
-- — Validate partition has data (not empty) —————
SELECT
    COUNT(*)                AS row_count,
    SUM(amount)              AS total_amount,
    MAX(load_timestamp)      AS latest_record
FROM orders_external
WHERE dt = '2024-12-01'
    AND region = 'US';
-- row_count = 0 means the partition exists but the file is empty
-- — HDFS file metadata via Hive SerDe —————
DESCRIBE FORMATTED orders_external PARTITION (dt='2024-12-01');
-- Returns: Location, InputFormat, OutputFormat, NumFiles, TotalSize
-- — Cross-check: file count matches expected batch count —————
SELECT
    dt,
    region,
    COUNT(*)    AS row_count,
    COUNT(*)    = 0 AS is_empty_partition
FROM orders_external
GROUP BY dt, region
HAVING COUNT(*) = 0;    -- must return 0 rows: no empty partitions
-- — Spark SQL: list files in a path —————
-- (Run in PySpark / Spark SQL context)
SELECT path, size, modificationTime, isDirectory
FROM json.`hdfs://warehouse/orders/dt=2024-12-01/`
LIMIT 20;
```

2. HDFS Block Size Validation

Purpose: HDFS stores files in blocks (default 128 MB or 256 MB). Files

significantly smaller than a block waste NameNode memory and produce excessive map tasks. Files larger than one block are fine — but a misconfigured block size causes suboptimal splits, wrong parallelism, and poor I/O throughput.

Validate that block size matches cluster configuration and data volume.

```
-- — Check table storage properties including block size —————
DESCRIBE FORMATTED orders_orc;
-- Look for:
-- Storage Desc Params: orc.block.size = 134217728 (128MB)
-- — HDFS dfs -stat to check actual block size —————
-- (Run from shell or via spark.sparkContext._jvm)
-- hdfs dfs -stat "%o %b %n" /warehouse/orders/dt=2024-12-01/*
-- %o = block size, %b = total bytes, %n = filename
-- — Hive: blocks per file via InputSplits —————
-- Estimated splits = CEIL(file_size / block_size)
```

```
-- Optimal: 1 mapper per block
SELECT
    SUM(total_size)                AS total_bytes,
    ROUND(SUM(total_size) / 1073741824.0, 2) AS total_gb,
    SUM(num_files)                 AS total_files,
    ROUND(SUM(total_size)
          / NULLIF(SUM(num_files), 0)
          / 1048576.0, 2)          AS avg_file_size_mb,
    CEIL(SUM(total_size)
         / 134217728.0)           AS expected_map_tasks_128mb,
    CEIL(SUM(total_size)
         / 268435456.0)           AS expected_map_tasks_256mb
FROM (
    DESCRIBE FORMATTED orders_orc
) t;
-- Use this to tune mapreduce.input.fileinputformat.split.maxsize
```

3. Partition Pruning Validation

Purpose: A partitioned Hive/Spark table that does NOT prune partitions

on a filter query is the single most expensive Big Data anti-pattern.

Without pruning, a 5-year fact table scans every partition on every query.

Validate that the query engine eliminates irrelevant partitions and that the partition filter is pushed down to the scan level.

```
-- — EXPLAIN: confirm partition pruning in the query plan ———
EXPLAIN EXTENDED
SELECT order_id, amount
FROM   orders_partitioned
WHERE  dt      = '2024-12-01'    -- partition column
      AND region = 'US';        -- second partition column
-- Look for: "Partition Filters: (dt = 2024-12-01 and region = US)"
-- BAD: "TableScan ... (all partitions)" = no pruning
-- — Count partitions scanned vs total ———
-- Before query: SET hive.exec.dynamic.partition=true;
-- After query check:
SELECT
    COUNT(DISTINCT dt)                AS total_partitions,
    COUNT(DISTINCT CASE WHEN dt = '2024-12-01' THEN dt END) AS scanned,
    COUNT(DISTINCT dt) -
    COUNT(DISTINCT CASE WHEN dt = '2024-12-01' THEN dt END) AS pruned
FROM orders_partitioned;
-- — Anti-pattern: partition column in function = no pruning ———
-- BAD (no pruning – function wraps partition column):
SELECT * FROM orders_partitioned
WHERE YEAR(dt) = 2024 AND MONTH(dt) = 12;
-- GOOD (pruning works – literal comparison on partition column):
SELECT * FROM orders_partitioned
WHERE dt BETWEEN '2024-12-01' AND '2024-12-31';
-- — Validate stats: ANALYZE TABLE for accurate partition stats —
ANALYZE TABLE orders_partitioned
PARTITION (dt='2024-12-01') COMPUTE STATISTICS;
ANALYZE TABLE orders_partitioned
PARTITION (dt='2024-12-01')
COMPUTE STATISTICS FOR COLUMNS order_id, amount, status;
-- — Check partition stats are populated ———
DESCRIBE FORMATTED orders_partitioned PARTITION (dt='2024-12-01');
-- numRows, rawDataSize must NOT be -1 (means stats are missing)
```

4. Bucketing Validation

Purpose: Hive/Spark bucketing co-locates data with the same hash value

into the same file — enabling bucket-aware joins that skip the shuffle

entirely. Validate that bucket count matches the spec, data is distributed

evenly across buckets, and bucket joins are actually converting to

SortMergeJoin without shuffle when they should.

```
-- — Verify table is bucketed as expected —————
DESCRIBE FORMATTED orders_bucketed;
-- Must show:
-- Num Buckets: 256
-- Bucket Columns: [customer_id]
-- Sort Columns: [customer_id ASC]
-- — Bucket distribution: rows per bucket should be uniform ———
SELECT
  FLOOR(HASH(customer_id) % 256) AS bucket_id,
  COUNT(*) AS rows_in_bucket,
  AVG(COUNT(*)) OVER () AS avg_rows_per_bucket,
  ROUND((COUNT(*) - AVG(COUNT(*)) OVER ())
    / NULLIF(STDDEV(COUNT(*)) OVER (), 0), 2) AS z_score
FROM orders_bucketed
GROUP BY FLOOR(HASH(customer_id) % 256)
HAVING ABS(ROUND((COUNT(*) - AVG(COUNT(*)) OVER ())
  / NULLIF(STDDEV(COUNT(*)) OVER (), 0), 2)) > 3
ORDER BY rows_in_bucket DESC;
-- Must return 0 rows: no severely skewed buckets (|Z| > 3)
-- — Verify bucket join converts to SMJ without shuffle —————
EXPLAIN
SELECT o.order_id, c.customer_name, o.amount
FROM orders_bucketed o
JOIN customers_bucketed c
  ON o.customer_id = c.customer_id;
-- Look for: SortMergeJoin (NOT ShuffleHashJoin)
-- GOOD: SortMergeJoin [customer_id = customer_id]
-- BAD: Exchange hashpartitioning(customer_id) — means shuffle happening
-- — Bucket count match: both tables must have same bucket count —
SELECT
  'orders_bucketed' AS table_name,
  256 AS expected_buckets,
  COUNT(DISTINCT FLOOR(HASH(customer_id) % 256)) AS actual_bucket_ids
FROM orders_bucketed
UNION ALL
SELECT
  'customers_bucketed',
  256,
  COUNT(DISTINCT FLOOR(HASH(customer_id) % 256))
FROM customers_bucketed;
```

5. Hive External vs Managed Table Checks

Purpose: External tables point to data that is managed outside Hive —

dropping the table does NOT delete the data. Managed tables own their data — dropping them DELETES the HDFS files. Miscategorising a table can cause catastrophic, unrecoverable data loss or surprise data orphans. Validate every table's type matches its intended lifecycle.

```
-- — List all tables with their type —————
SHOW TABLES IN orders_db;
SELECT
  t.table_name,
  t.table_type,          -- 'MANAGED_TABLE' or 'EXTERNAL_TABLE'
  t.location,
  p.partition_count,
  CASE
    WHEN t.table_type = 'EXTERNAL_TABLE'
      AND t.location LIKE '/warehouse/managed/%'
    THEN 'WRONG LOCATION ⚠'
    WHEN t.table_type = 'MANAGED_TABLE'
      AND t.location LIKE '/warehouse/external/%'
    THEN 'WRONG LOCATION ⚠'
    WHEN t.table_type = 'EXTERNAL_TABLE'
      AND spec.expected_type = 'MANAGED'
```

```

        THEN 'SHOULD BE MANAGED ✖'
        WHEN t.table_type = 'MANAGED_TABLE'
            AND spec.expected_type = 'EXTERNAL'
        THEN 'SHOULD BE EXTERNAL ✖'
        ELSE 'CORRECT ✔'
    END AS table_type_status
FROM information_schema.tables t
LEFT JOIN etl_metadata.table_spec spec
    ON t.table_name = spec.table_name
LEFT JOIN (
    SELECT table_name, COUNT(*) AS partition_count
    FROM information_schema.partitions
    GROUP BY table_name
) p ON t.table_name = p.table_name
WHERE t.table_schema = 'orders_db'
ORDER BY t.table_type, t.table_name;
-- — External table: verify HDFS location exists & is accessible —
DESCRIBE FORMATTED orders_external;
-- Location: hdfs:///data/external/orders ← must be valid HDFS path
-- — Managed table: data must be under Hive warehouse dir —
DESCRIBE FORMATTED orders_managed;
-- Location: hdfs:///user/hive/warehouse/orders_db.db/orders_managed
-- — External table has EXTERNAL keyword —
SHOW CREATE TABLE orders_external;
-- Must contain: CREATE EXTERNAL TABLE
-- — Managed table: confirm no EXTERNAL keyword —
SHOW CREATE TABLE orders_managed;
-- Must NOT contain: EXTERNAL

```

6. ORC / Parquet Format Validation

Purpose: ORC and Parquet are columnar formats with built-in compression,

statistics, and predicate pushdown. A table stored as plain CSV or uncompressed text runs 10–100× slower and uses 5–10× more disk. Validate file format, compression codec, stripe/row-group count, and that column statistics enable predicate pushdown.

```

-- — Verify table is stored in ORC or Parquet —
DESCRIBE FORMATTED orders_orc;
-- InputFormat: org.apache.hadoop.hive ql.io.orc.OrcInputFormat
-- OutputFormat: org.apache.hadoop.hive ql.io.orc.OrcOutputFormat
-- SerDe Library: org.apache.hadoop.hive ql.io.orc.OrcSerde
DESCRIBE FORMATTED orders_parquet;
-- InputFormat: org.apache.hadoop.mapred.SequenceFileInputFormat ← WRONG
-- InputFormat: org.apache.parquet.hadoop.ParquetInputFormat ← CORRECT
-- — ORC: verify compression is not NONE —
DESCRIBE FORMATTED orders_orc;
-- Storage Desc Params:
--   orc.compress = SNAPPY ← OK
--   orc.compress = ZLIB ← OK (better compression ratio)
--   orc.compress = NONE ← BAD: no compression
-- — ORC file stats via Hive (predicate pushdown needs stats) —
-- orc.include.columns.pushdown works only with statistics
SELECT
    COUNT(*) AS total_rows,
    MIN(amount) AS min_amount,
    MAX(amount) AS max_amount
FROM orders_orc
WHERE dt = '2024-12-01';
-- If predicate pushdown works: only reads 'amount' column from ORC stripe
-- — Parquet: check row group size —
-- spark.sql.parquet.block.size default = 128MB (matches HDFS block)
-- Validate via:
-- parquet-tools meta hdfs:///warehouse/orders/part-00000.parquet
-- — Wrong format detection: text files in a columnar table —
SELECT INPUT_FILE_NAME, COUNT(*)
FROM orders_orc

```

```
GROUP BY INPUT__FILE__NAME
HAVING INPUT__FILE__NAME NOT LIKE '%.orc'
AND INPUT__FILE__NAME NOT LIKE '%.parquet';
-- Must return 0 rows
```

7. Schema-on-Read Validation

Purpose: Hive and Spark use schema-on-read — the schema is applied when

data is queried, not when it is written. This means wrong data is written silently and only fails at read time. Validate that the physical data matches the declared schema: data types coerce correctly, struct fields exist, and no fields produce mass-NULL reads due to schema drift.

```
-- — Read validation: check for type coercion failures —————
SELECT
    COUNT(*) AS total_rows,
    -- If CAST fails, returns NULL – count NULLs to find failures
    SUM(CASE WHEN CAST(amount AS DECIMAL(18,2))
        IS NULL AND amount IS NOT NULL
        THEN 1 ELSE 0 END) AS amount_cast_failures,
    SUM(CASE WHEN CAST(order_date AS DATE)
        IS NULL AND order_date IS NOT NULL
        THEN 1 ELSE 0 END) AS date_cast_failures,
    SUM(CASE WHEN CAST(customer_id AS BIGINT)
        IS NULL AND customer_id IS NOT NULL
        THEN 1 ELSE 0 END) AS id_cast_failures
FROM orders_raw; -- raw text/CSV table with declared schema
-- — Schema drift: NULL rows where new column was added —————
-- When a new column is added to schema but old files don't have it,
-- all old-partition reads will return NULL for that column
SELECT
    dt,
    COUNT(*) AS total_rows,
    SUM(CASE WHEN new_column IS NULL
        THEN 1 ELSE 0 END) AS null_new_col,
    ROUND(100.0 * SUM(CASE WHEN new_column IS NULL THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 2) AS null_pct
FROM orders_with_new_col
GROUP BY dt
ORDER BY dt;
-- Old partitions will show 100% NULL for new_column
-- — JSON/complex type: nested field exists —————
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN get_json_object(
        metadata_json, '$.source_system') IS NULL
        THEN 1 ELSE 0 END) AS missing_source_system,
    SUM(CASE WHEN get_json_object(
        metadata_json, '$.event_type') IS NULL
        THEN 1 ELSE 0 END) AS missing_event_type
FROM orders_json_raw;
```

8. Spark Job Output Validation

Purpose: A Spark job that completes with status SUCCESS does not guarantee

correct output. Validate row counts, measure totals, output file count, schema correctness, and that no tasks silently produced empty partitions or corrupt records.

```
-- — Output row count matches expected —————
SELECT
    COUNT(*) AS output_rows,
    COUNT(DISTINCT order_id) AS distinct_orders,
    SUM(amount) AS total_revenue,
    MIN(order_date) AS earliest_date,
    MAX(order_date) AS latest_date,
```

```

COUNT(DISTINCT dt)          AS partitions_written
FROM orders_spark_output
WHERE spark_job_id = 'job_20241201_001';
-- — Empty output partitions —————
SELECT
    dt,
    COUNT(*)          AS rows
FROM orders_spark_output
WHERE spark_job_id = 'job_20241201_001'
GROUP BY dt
HAVING COUNT(*) = 0;  -- must return 0 rows
-- — Schema validation: all expected columns present —————
DESCRIBE orders_spark_output;
-- Verify: order_id, customer_id, amount, status, dt all present
-- and no unexpected columns like '_corrupt_record'
-- — Corrupt record detection (JSON/CSV bad parse) —————
SELECT COUNT(*) AS corrupt_records
FROM orders_spark_output
WHERE _corrupt_record IS NOT NULL;
-- Must return 0: any value means malformed source rows

```

9. Spark Shuffle & Skew Detection

Purpose: Shuffle is Spark's most expensive operation — data crosses

executor boundaries over the network. Data skew during shuffle means 1 executor handles 90% of the data while 31 sit idle — the job runs at the speed of the slowest key. Validate shuffle volume, detect skewed partition keys, and identify straggler tasks.

```

-- — Partition size distribution after shuffle —————
-- Run in Spark: df.groupBy("customer_id").count().summary()
-- — Key skew: find the hot keys driving imbalance —————
SELECT
    customer_id,
    COUNT(*)          AS row_count,
    ROUND(100.0 * COUNT(*)
          / SUM(COUNT(*) OVER (), 4) AS pct_of_total
FROM orders_fact
GROUP BY customer_id
ORDER BY row_count DESC
LIMIT 20;
-- Any single key > 5% of total is a skew risk in large joins
-- — Shuffle bytes per stage (from Spark History Server) —————
SELECT
    stage_id,
    stage_name,
    shuffle_read_bytes / 1073741824.0 AS shuffle_read_gb,
    shuffle_write_bytes / 1073741824.0 AS shuffle_write_gb,
    num_tasks,
    max_task_duration_ms,
    median_task_duration_ms,
    max_task_duration_ms /
    NULLIF(median_task_duration_ms, 0) AS straggler_ratio,
    CASE
        WHEN max_task_duration_ms /
             NULLIF(median_task_duration_ms, 0) > 5
        THEN 'SEVERE SKEW ✖'
        WHEN max_task_duration_ms /
             NULLIF(median_task_duration_ms, 0) > 2
        THEN 'MODERATE SKEW ⚠'
        ELSE 'BALANCED ✔'
    END AS skew_status
FROM spark_history.stage_metrics
WHERE app_id = 'application_20241201_001'
ORDER BY shuffle_read_gb DESC;

```


10. Small File Problem Detection

Purpose: The small file problem is Hadoop's silent killer. Thousands of

files smaller than an HDFS block waste NameNode heap (each file = ~150 bytes of NameNode memory), produce excessive map tasks, and make every read inefficient. Validate file counts, detect small file accumulation, and validate compaction has run where needed.

```
-- — File count per partition —————
-- (Run via Hive InputSplit analysis or HDFS dfs -count)
-- hdfs dfs -count -q /warehouse/orders/*
-- Format: DIR_COUNT FILE_COUNT CONTENT_SIZE PATHNAME
-- — Hive: estimate file count from storage stats —————
DESCRIBE FORMATTED orders_partitioned PARTITION (dt='2024-12-01');
-- numFiles = 10000 ← PROBLEM: 10K small files
-- totalSize = 1073741824 ← 1GB / 10000 files = 100KB avg = SMALL
-- — Estimated NameNode memory usage —————
-- Each file + block = ~300 bytes of NameNode heap
-- 1M files = ~300MB NameNode heap consumed just for this table!
SELECT
    SUM(num_files)                AS total_files,
    ROUND(SUM(total_size)
          / 1073741824.0, 2)      AS total_gb,
    ROUND(SUM(total_size)
          / NULLIF(SUM(num_files),0)
          / 1048576.0, 2)        AS avg_file_size_mb,
    ROUND(SUM(num_files) * 300
          / 1048576.0, 2)        AS est_namenode_heap_mb,
    CASE
        WHEN ROUND(SUM(total_size)
                  / NULLIF(SUM(num_files),0)
                  / 1048576.0, 2) < 1
        THEN 'CRITICAL – TINY FILES ✖'
        WHEN ROUND(SUM(total_size)
                  / NULLIF(SUM(num_files),0)
                  / 1048576.0, 2) < 64
        THEN 'SMALL FILES ⚠'
        ELSE 'HEALTHY ✔'
    END AS small_file_status
FROM (
    SELECT SUM(num_files) AS num_files, SUM(total_size) AS total_size
    FROM PARTITIONS
    WHERE table_name = 'orders_partitioned'
) t;
```

Performance & Scalability Testing

ETL Performance Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Performance\_Validations\_Python.md`

⚡ A pipeline that works on 10 GB will silently break at 10 TB.

1. Load Time Validation

Purpose: Every batch must complete within its SLA window. Validate

actual load duration against SLA thresholds, detect deteriorating trends before they breach SLAs, and flag any batch that is slowing down across runs.

```
-- — Current run duration vs SLA —————
SELECT
    r.run_id,
    r.pipeline_name,
    r.batch_date,
    r.run_start,
    r.run_end,
    DATEDIFF(SECOND, r.run_start, r.run_end)      AS duration_secs,
    DATEDIFF(MINUTE, r.run_start, r.run_end)      AS duration_mins,
    s.sla_seconds,
    DATEDIFF(SECOND, r.run_start, r.run_end)      AS sla_breach_secs,
    - s.sla_seconds
    ROUND(100.0 *
        DATEDIFF(SECOND, r.run_start, r.run_end)
        / NULLIF(s.sla_seconds, 0), 2)          AS pct_of_sla,
    CASE
        WHEN DATEDIFF(SECOND, r.run_start, r.run_end)
            <= s.sla_seconds * 0.80              THEN 'HEALTHY ✓'
        WHEN DATEDIFF(SECOND, r.run_start, r.run_end)
            <= s.sla_seconds                    THEN 'WITHIN SLA ⚠'
        ELSE                                     'SLA BREACH ✗'
    END                                          AS sla_status
FROM etl_control.dbo.PIPELINE_RUN_LOG r
JOIN etl_control.dbo.PIPELINE_SLAs s
    ON r.pipeline_name = s.pipeline_name
WHERE r.batch_date = CAST(GETDATE() AS DATE)
ORDER BY r.run_start DESC;
-- — 30-day duration trend with moving average —————
WITH run_durations AS (
    SELECT
        batch_date,
        pipeline_name,
        DATEDIFF(SECOND, run_start, run_end)      AS duration_secs,
        rows_processed
    FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE run_status = 'SUCCESS'
        AND pipeline_name = 'ORDERS_ETL'
        AND batch_date >= DATEADD(DAY, -30, GETDATE())
)
SELECT
    batch_date,
    duration_secs,
    rows_processed,
    ROUND(duration_secs * 1.0 / NULLIF(rows_processed, 0)
        * 1000, 4)                                AS ms_per_1k_rows,
    AVG(duration_secs) OVER (
        ORDER BY batch_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    )                                              AS ma_7d_secs,
    ROUND(100.0 * (duration_secs -
        AVG(duration_secs) OVER (
            ORDER BY batch_date
```

```

        ROWS BETWEEN 6 PRECEDING AND 1 PRECEDING
    ))
    / NULLIF(AVG(duration_secs) OVER (
        ORDER BY batch_date
        ROWS BETWEEN 6 PRECEDING AND 1 PRECEDING
    ), 0), 2) AS pct_vs_ma,
CASE
    WHEN duration_secs > AVG(duration_secs) OVER (
        ORDER BY batch_date
        ROWS BETWEEN 6 PRECEDING AND 1 PRECEDING) * 1.5
    THEN 'DEGRADED ✖'
    WHEN duration_secs > AVG(duration_secs) OVER (
        ORDER BY batch_date
        ROWS BETWEEN 6 PRECEDING AND 1 PRECEDING) * 1.2
    THEN 'SLOW ⚠'
    ELSE 'NORMAL ✔'
END AS trend_status
FROM run_durations
ORDER BY batch_date DESC;
-- — Stage-level timing breakdown (which stage is the bottleneck?)
SELECT
    stage_name,
    stage_start,
    stage_end,
    DATEDIFF(SECOND, stage_start, stage_end) AS stage_secs,
    ROUND(100.0 *
        DATEDIFF(SECOND, stage_start, stage_end)
        / NULLIF(SUM(DATEDIFF(SECOND, stage_start, stage_end))
            OVER (), 0), 2) AS pct_of_total,
    rows_in,
    rows_out,
    ROUND(rows_out * 1.0
        / NULLIF(DATEDIFF(SECOND, stage_start, stage_end), 0),
        0) AS rows_per_sec
FROM etl_control.dbo.STAGE_TIMING_LOG
WHERE run_id = (
    SELECT MAX(run_id) FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE pipeline_name = 'ORDERS_ETL'
)
ORDER BY stage_start;
-- — Throughput: rows/sec and MB/sec per run —————
SELECT
    run_id,
    batch_date,
    rows_processed,
    bytes_processed,
    DATEDIFF(SECOND, run_start, run_end) AS duration_secs,
    ROUND(rows_processed * 1.0
        / NULLIF(DATEDIFF(SECOND, run_start, run_end), 0), 0) AS rows_per_sec,
    ROUND(bytes_processed / 1048576.0
        / NULLIF(DATEDIFF(SECOND, run_start, run_end), 0), 2) AS mb_per_sec
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE pipeline_name = 'ORDERS_ETL'
    AND run_status = 'SUCCESS'
ORDER BY batch_date DESC;

```

2. Query Response Time

Purpose: Critical warehouse queries (dashboard loads, report exports,

ad-hoc joins) must respond within SLA. Detect slow queries, query plan regressions, missing index usage, and statistics staleness — before analysts start complaining.

```

-- — Identify slow queries against the fact/dim tables —————
SELECT TOP 25
    qs.total_elapsed_time / qs.execution_count / 1000.0 AS avg_elapsed_ms,
    qs.total_elapsed_time / 1000.0 AS total_elapsed_ms,

```

```

qs.execution_count,
qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
qs.total_worker_time / qs.execution_count / 1000.0 AS avg_cpu_ms,
SUBSTRING(qt.TEXT,
  (qs.statement_start_offset / 2) + 1,
  ((CASE qs.statement_end_offset
    WHEN -1 THEN DATALENGTH(qt.TEXT)
    ELSE qs.statement_end_offset
  END - qs.statement_start_offset) / 2) + 1
) AS query_text,
qp.query_plan,
qs.last_execution_time
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE qt.TEXT LIKE '%FACT_ORDERS%'
AND qs.last_execution_time >= DATEADD(HOUR, -24, GETDATE())
ORDER BY avg_elapsed_ms DESC;
-- — Table scan detection (missing index = full scan at TB scale) —
SELECT
  OBJECT_NAME(i.object_id) AS table_name,
  i.name AS index_name,
  ios.user_scans,
  ios.user_seeks,
  ios.user_lookups,
  ios.user_updates,
  CASE
    WHEN ios.user_scans > 1000
      AND ios.user_seeks < ios.user_scans
    THEN 'TABLE SCAN HOT — ADD INDEX ✕'
    WHEN ios.user_scans > 100
    THEN 'WARN — FREQUENT SCANS ⚠'
    ELSE 'OK ✓'
  END AS scan_assessment,
  ios.last_user_scan
FROM sys.indexes i
JOIN sys.dm_db_index_usage_stats ios
  ON i.object_id = ios.object_id
  AND i.index_id = ios.index_id
  AND ios.database_id = DB_ID()
WHERE OBJECT_NAME(i.object_id) IN ('FACT_ORDERS', 'DIM_CUSTOMERS', 'DIM_PRODUCTS')
ORDER BY ios.user_scans DESC;
-- — Missing index recommendations —————
SELECT TOP 10
  ROUND(mid.avg_total_user_cost *
    mid.avg_user_impact *
    (mid.user_seeks + mid.user_scans), 0) AS improvement_measure,
  OBJECT_NAME(mid.object_id) AS table_name,
  mid.equality_columns,
  mid.inequality_columns,
  mid.included_columns,
  ROUND(mid.avg_user_impact, 0) AS avg_pct_benefit
FROM sys.dm_db_missing_index_details mid
JOIN sys.dm_db_missing_index_groups mig
  ON mid.index_handle = mig.index_handle
JOIN sys.dm_db_missing_index_group_stats migs
  ON mig.index_group_handle = migs.group_handle
WHERE mid.database_id = DB_ID()
ORDER BY improvement_measure DESC;
-- — Query SLA check: all critical reports under threshold ———
SELECT
  q.report_name,
  q.query_sla_ms,
  AVG(ql.elapsed_ms) AS avg_elapsed_ms,
  MAX(ql.elapsed_ms) AS max_elapsed_ms,
  COUNT(*) AS executions,
  SUM(CASE WHEN ql.elapsed_ms > q.query_sla_ms
    THEN 1 ELSE 0 END) AS sla_breaches,

```

```

ROUND(100.0 *
      SUM(CASE WHEN ql.elapsed_ms > q.query_sla_ms THEN 1 ELSE 0 END)
      / NULLIF(COUNT(*), 0), 2)          AS breach_pct
FROM etl_control.dbo.QUERY_SLA_CONFIG q
JOIN etl_control.dbo.QUERY_EXECUTION_LOG ql
      ON q.report_name = ql.report_name
WHERE ql.executed_at >= DATEADD(HOUR, -24, GETDATE())
GROUP BY q.report_name, q.query_sla_ms
ORDER BY breach_pct DESC;

```

3. Partition-Wise Performance

Purpose: At TB/PB scale, queries must prune partitions. A full-table scan

on a 5-TB fact table when only one partition is needed is a catastrophic performance defect. Validate partition elimination works, partition sizes are balanced, and skewed partitions are flagged before they kill parallelism.

```

-- — Partition elimination check: only relevant partitions scanned
-- Use SET STATISTICS IO ON to see pages read per partition
SELECT
    partition_number,
    rows,
    total_pages,
    used_pages,
    ROUND(total_pages * 8 / 1024.0, 2)          AS size_mb,
    ROUND(rows * 100.0
          / NULLIF(SUM(rows) OVER (), 0), 4)    AS pct_of_rows
FROM (
    SELECT
        p.partition_number,
        p.rows,
        a.total_pages,
        a.used_pages
    FROM sys.partitions p
    JOIN sys.allocation_units a
        ON (a.container_id = p.partition_id
            AND a.type IN (1, 3))
        OR (a.container_id = p.hobt_id
            AND a.type = 2)
    WHERE OBJECT_NAME(p.object_id) = 'FACT_ORDERS'
        AND p.index_id IN (0, 1)
) partition_data
ORDER BY partition_number;
-- — Partition size balance: detect skewed partitions —————
WITH partition_stats AS (
    SELECT
        p.partition_number,
        p.rows,
        AVG(p.rows) OVER () AS avg_rows,
        STDEV(p.rows) OVER () AS std_rows
    FROM sys.partitions p
    WHERE OBJECT_NAME(p.object_id) = 'FACT_ORDERS'
        AND p.index_id IN (0, 1)
)
SELECT
    partition_number,
    rows,
    avg_rows,
    ROUND((rows - avg_rows) / NULLIF(std_rows, 0), 2) AS z_score,
    CASE
        WHEN (rows - avg_rows) / NULLIF(std_rows, 0) > 3
        THEN 'HOT PARTITION ✖ — SKEWED'
        WHEN rows = 0
        THEN 'EMPTY PARTITION ⚠'
        ELSE 'BALANCED ✔'
    END AS balance_status
FROM partition_stats

```

```

ORDER BY rows DESC;
-- — Query with and without partition filter (Hive/Spark style) —
-- Baseline: no partition filter (full scan — expect high cost)
EXPLAIN SELECT COUNT(*), SUM(AMOUNT)
FROM dwh.dbo.FACT_ORDERS;
-- Optimised: with partition filter (expect partition pruning)
EXPLAIN SELECT COUNT(*), SUM(AMOUNT)
FROM dwh.dbo.FACT_ORDERS
WHERE ORDER_DATE BETWEEN '2024-11-01' AND '2024-11-30';
-- — Missing partition: expected partitions not present —————
WITH expected_months AS (
    SELECT DISTINCT
        FORMAT DATEADD(MONTH, -n, GETDATE()), 'yyyy-MM' AS expected_month
    FROM (
        SELECT TOP 24 ROW_NUMBER() OVER (ORDER BY object_id) - 1 AS n
        FROM sys.objects
    ) nums
)
SELECT
    e.expected_month,
    p.partition_label,
    CASE WHEN p.partition_label IS NULL
        THEN 'MISSING ✕' ELSE 'PRESENT ✔' END AS partition_status
FROM expected_months e
LEFT JOIN etl_control.dbo.PARTITION_REGISTRY p
    ON e.expected_month = p.partition_label
   AND p.table_name = 'FACT_ORDERS'
ORDER BY e.expected_month;

```

4. Parallelism Validation

Purpose: TB-scale pipelines must exploit parallelism — distributed

workers, thread pools, parallel query plans. If your 32-core cluster runs a pipeline single-threaded, you're 32× slower than you should be. Validate actual parallelism against expected concurrency, detect serial bottlenecks, and confirm no thread starvation.

```

-- — Active parallel workers at peak load —————
SELECT
    r.session_id,
    r.request_id,
    r.status,
    r.command,
    r.cpu_time,
    r.logical_reads,
    r.granted_query_memory AS memory_grant_kb,
    r.dop AS degree_of_parallelism,
    SUBSTRING(qt.TEXT, 1, 200) AS query_snippet
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) qt
WHERE r.database_id = DB_ID()
    AND r.dop > 1 -- only parallel queries
ORDER BY r.dop DESC, r.cpu_time DESC;
-- — DOP distribution: are we actually going parallel? —————
SELECT
    dop,
    COUNT(*) AS query_count,
    ROUND(100.0 * COUNT(*)
        / NULLIF(SUM(COUNT(*)) OVER (), 0), 2) AS pct_of_queries
FROM etl_control.dbo.QUERY_EXECUTION_LOG
WHERE executed_at >= DATEADD(HOUR, -24, GETDATE())
GROUP BY dop
ORDER BY dop;
-- — MAXDOP and parallelism configuration —————
SELECT
    name,
    value_in_use,

```

```

description
FROM sys.configurations
WHERE name IN (
    'max degree of parallelism',
    'cost threshold for parallelism',
    'max worker threads'
);
-- — Thread wait statistics: what are workers waiting on? ———
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    ROUND(100.0 * wait_time_ms
        / NULLIF(SUM(wait_time_ms) OVER (), 0), 2) AS pct_total_wait,
    max_wait_time_ms,
    CASE
        WHEN wait_type LIKE 'CXPACKET%' THEN 'PARALLELISM SKEW ⚠'
        WHEN wait_type LIKE 'LCK_%' THEN 'LOCKING ✖'
        WHEN wait_type LIKE 'IO_%' THEN 'I/O BOUND ⚠'
        WHEN wait_type LIKE 'RESOURCE_%' THEN 'RESOURCE PRESSURE ✖'
        WHEN wait_type = 'ASYNC_NETWORK_IO' THEN 'CLIENT TOO SLOW ⚠'
        ELSE 'OTHER'
    END AS wait_category
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
    'SLEEP_TASK', 'BROKER_TO_FLUSH', 'BROKER_EVENTHANDLER',
    'REQUEST_FOR_DEADLOCK_SEARCH', 'XE_TIMER_EVENT', 'WAITFOR',
    'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE'
)
ORDER BY wait_time_ms DESC;

```

5. Resource Utilization (CPU, Memory, I/O, Disk)

Purpose: A pipeline that finishes on time but consumes 95% CPU leaves

nothing for concurrent workloads. Memory pressure causes spills to disk that are 100–1000× slower than RAM operations. Validate that resource usage during ETL stays within safe operational bands.

```

-- — Memory grant efficiency ———
SELECT
    qs.total_grant_kb / qs.execution_count AS avg_grant_kb,
    qs.total_used_grant_kb / qs.execution_count AS avg_used_kb,
    ROUND(100.0 *
        (qs.total_grant_kb - qs.total_used_grant_kb)
        / NULLIF(qs.total_grant_kb, 0), 2) AS wasted_pct,
    qs.total_spills / qs.execution_count AS avg_spills,
    SUBSTRING(qt.TEXT, 1, 200) AS query_snippet
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
WHERE qs.total_grant_kb > 0
    AND qt.TEXT LIKE '%FACT_ORDERS%'
ORDER BY avg_spills DESC, wasted_pct DESC;
-- — Disk spill detection (sort/hash spills = memory pressure) —
SELECT
    session_id,
    request_id,
    spills,
    granted_query_memory AS memory_grant_kb,
    SUBSTRING(qt.TEXT, 1, 200) AS query_text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) qt
WHERE r.spills > 0
ORDER BY r.spills DESC;
-- — I/O pressure: most-read tables during ETL window ———
SELECT
    OBJECT_NAME(ios.object_id) AS table_name,
    ios.leaf_read_count,

```

```

    ios.leaf_write_count,
    ios.range_scan_count,
    ios.singleton_lookup_count,
    ios.forwarded_fetch_count
FROM sys.dm_db_index_operational_stats(DB_ID(), NULL, NULL, NULL) ios
WHERE OBJECT_NAME(ios.object_id) IN (
    'FACT_ORDERS', 'DIM_CUSTOMERS', 'DIM_PRODUCTS', 'ORDERS_STAGE'
)
ORDER BY ios.leaf_read_count DESC;
-- — TempDB usage (spills go here – monitor closely) —————
SELECT
    SUM(internal_object_reserved_page_count) AS internal_pages,
    SUM(user_object_reserved_page_count) AS user_pages,
    SUM(version_store_reserved_page_count) AS version_pages,
    SUM(internal_object_reserved_page_count +
        user_object_reserved_page_count +
        version_store_reserved_page_count) * 8
        / 1024 AS total_mb,
    (SELECT SUM(total_page_count) * 8 / 1024
     FROM sys.dm_db_file_space_usage
     WHERE database_id = 2) AS tempdb_size_mb
FROM sys.dm_db_session_space_usage
WHERE session_id > 50;

```

6. Job Failure Recovery Time

Purpose: In production, pipelines fail. The SLA is not just for the happy

path — it includes recovery time. Validate that checkpoint recovery starts quickly, re-runs complete faster than a cold start (checkpoint advantage), and the total recovery time (detection + restart + completion) fits within the operational recovery window.

```

-- — Failure detection latency —————
SELECT
    run_id,
    pipeline_name,
    run_start,
    failure_time,
    DATEDIFF(SECOND, failure_time,
        alert_sent_time) AS detection_latency_secs,
    DATEDIFF(SECOND, failure_time,
        restart_triggered_time) AS time_to_restart_secs,
    DATEDIFF(SECOND, run_start,
        run_end) AS total_wall_clock_secs,
    failure_stage,
    failure_reason,
    checkpoint_hwm,
    CASE
        WHEN DATEDIFF(SECOND, failure_time, alert_sent_time) <= 60
        THEN 'DETECTION OK ☒'
        ELSE 'SLOW DETECTION ☐'
    END AS detection_status,
    CASE
        WHEN DATEDIFF(SECOND, failure_time,
            restart_triggered_time) <= 300
        THEN 'RESTART FAST ☒'
        ELSE 'RESTART SLOW ☐'
    END AS restart_status
FROM etl_control.dbo.PIPELINE_FAILURE_LOG
WHERE pipeline_name = 'ORDERS_ETL'
ORDER BY failure_time DESC;
-- — Cold start vs warm restart duration comparison —————
SELECT
    'COLD START' AS run_type,
    AVG(DATEDIFF(SECOND, run_start, run_end)) AS avg_duration_secs,
    COUNT(*) AS run_count
FROM etl_control.dbo.PIPELINE_RUN_LOG

```



```

WHERE is_restart = 0 AND run_status = 'SUCCESS'
  AND pipeline_name = 'ORDERS_ETL'
UNION ALL
SELECT
  'WARM RESTART',
  AVG(DATEDIFF(SECOND, run_start, run_end)),
  COUNT(*)
FROM etl_control.dbo.PIPELINE_RUN_LOG
WHERE is_restart = 1 AND run_status = 'SUCCESS'
  AND pipeline_name = 'ORDERS_ETL';
-- — Mean time to recover (MTTR) —————
SELECT
  pipeline_name,
  COUNT(*) AS total_failures,
  AVG(DATEDIFF(MINUTE,
    failure_time, recovery_complete_time)) AS avg_mttr_mins,
  MAX(DATEDIFF(MINUTE,
    failure_time, recovery_complete_time)) AS max_mttr_mins,
  ROUND(100.0 *
    SUM(CASE WHEN DATEDIFF(MINUTE,
      failure_time,
      recovery_complete_time) <= 60
    THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0), 2) AS pct_within_1hr_sla
FROM etl_control.dbo.PIPELINE_FAILURE_LOG
WHERE failure_time >= DATEADD(DAY, -90, GETDATE())
GROUP BY pipeline_name
ORDER BY avg_mttr_mins DESC;

```

Job & Workflow Validation

ETL Job Workflow Validations — SQL Queries

SQL VALIDATION REFERENCE

SQL-only reference. Python scripts: `ETL\_Job\_Workflow\_Validations\_Python.md`

 A pipeline that produces correct data but runs at the wrong time,

1. Job Dependency Validation

Purpose: Every job in a DAG has upstream dependencies that must

complete successfully before it runs. Validate that no job starts before its dependencies finish, that dependency failures correctly block downstream tasks, and that no circular dependencies exist in the DAG definition.

```
-- — Dependency execution order: child started after parent ended —
SELECT
  parent.task_name          AS dependency_task,
  parent.status             AS dependency_status,
  parent.end_time           AS dependency_end,
  child.task_name           AS dependent_task,
  child.status              AS dependent_status,
  child.start_time          AS dependent_start,
  DATEDIFF(SECOND,
    parent.end_time,
    child.start_time)       AS gap_seconds,
  CASE
    WHEN parent.status <> 'SUCCESS'
      AND child.status = 'SUCCESS'
    THEN 'VIOLATED — RAN DESPITE FAILED DEP ✖'
    WHEN child.start_time < parent.end_time
    THEN 'VIOLATED — STARTED BEFORE DEP FINISHED ✖'
    WHEN parent.status = 'SUCCESS'
      AND child.status = 'SKIPPED'
    THEN 'VIOLATED — BLOCKED DESPITE SUCCESS ✖'
    WHEN parent.status = 'FAILED'
      AND child.status = 'SKIPPED'
    THEN 'CORRECTLY BLOCKED ✔'
    WHEN parent.status = 'SUCCESS'
      AND child.start_time >= parent.end_time
    THEN 'CORRECT ORDER ✔'
    ELSE 'REVIEW ⚠'
  END
  AS dependency_validation
FROM etl_control.dbo.DAG_TASK_LOG child
JOIN etl_control.dbo.DAG_TASK_LOG parent
  ON child.depends_on = parent.task_name
  AND child.run_id = parent.run_id
WHERE child.run_id = (
  SELECT MAX(run_id)
  FROM etl_control.dbo.DAG_TASK_LOG
)
ORDER BY child.start_time;
-- — All dependencies for a given task (full ancestry) —————
WITH RECURSIVE task_ancestry AS (
  SELECT
    task_name,
    depends_on,
    1 AS depth,
    CAST(task_name AS VARCHAR(MAX)) AS ancestry_path
  FROM etl_control.dbo.DAG_DEFINITION
  WHERE task_name = 'LOAD_FACT_ORDERS'
  UNION ALL
  SELECT
    d.task_name,
    d.depends_on,
    a.depth + 1,
```

```

        a.ancestry_path + ' ← ' + d.task_name
    FROM etl_control.dbo.DAG_DEFINITION d
    JOIN task_ancestry a ON d.task_name = a.depends_on
    WHERE a.depth < 20
)
SELECT
    task_name,
    depends_on,
    depth          AS dependency_level,
    ancestry_path
FROM task_ancestry
ORDER BY depth;
-- — Circular dependency detection —————
WITH RECURSIVE cycle_check AS (
    SELECT task_name, depends_on,
           CAST(task_name AS VARCHAR(MAX)) AS visited,
           0 AS has_cycle
    FROM etl_control.dbo.DAG_DEFINITION
    WHERE depends_on IS NOT NULL
    UNION ALL
    SELECT d.task_name, d.depends_on,
           c.visited + ',' + d.task_name,
           CASE WHEN c.visited LIKE '%' + d.task_name + '%'
                THEN 1 ELSE 0 END
    FROM etl_control.dbo.DAG_DEFINITION d
    JOIN cycle_check c ON d.depends_on = c.task_name
    WHERE c.has_cycle = 0
)
SELECT task_name, visited AS cycle_path
FROM cycle_check
WHERE has_cycle = 1; -- Must return 0 rows
-- — Dependency SLA: max wait time between tasks —————
SELECT
    child.task_name,
    parent.task_name AS upstream_task,
    AVG(DATEDIFF(SECOND, parent.end_time,
                 child.start_time)) AS avg_wait_secs,
    MAX(DATEDIFF(SECOND, parent.end_time,
                 child.start_time)) AS max_wait_secs,
    dep_sla.max_wait_seconds AS sla_wait_secs,
    CASE
        WHEN MAX(DATEDIFF(SECOND, parent.end_time,
                          child.start_time))
            <= dep_sla.max_wait_seconds THEN 'WITHIN SLA ✓'
        ELSE 'SLA BREACH ✗'
    END AS wait_sla_status
FROM etl_control.dbo.DAG_TASK_LOG child
JOIN etl_control.dbo.DAG_TASK_LOG parent
    ON child.depends_on = parent.task_name
   AND child.run_id = parent.run_id
JOIN etl_control.dbo.DEPENDENCY_SLA dep_sla
    ON dep_sla.child_task = child.task_name
   AND dep_sla.parent_task = parent.task_name
WHERE child.run_id >= (
    SELECT MAX(run_id) - 29
    FROM etl_control.dbo.DAG_TASK_LOG
)
GROUP BY child.task_name, parent.task_name, dep_sla.max_wait_seconds;

```

2. Workflow Sequence Validation

Purpose: A workflow is not just a dependency graph — it is a contract

that tasks execute in the correct logical sequence, that no tasks are skipped due to misconfigured triggers, and that parallel branches converge at the right synchronisation point before downstream tasks begin.

```

-- — Expected vs actual task execution sequence —————

```

```

WITH expected_sequence AS (
    SELECT task_name, expected_order
    FROM (VALUES
        ('EXTRACT_ORDERS', 1),
        ('VALIDATE_SOURCE', 2),
        ('LOAD_STAGE', 3),
        ('TRANSFORM_DIM', 4),
        ('TRANSFORM_FACT', 5),
        ('LOAD_DIM_CUSTOMERS', 6),
        ('LOAD_DIM_PRODUCTS', 6), -- parallel with DIM_CUSTOMERS
        ('LOAD_FACT_ORDERS', 7),
        ('REFRESH_REPORTS', 8),
        ('SEND_ALERTS', 9)
    ) t(task_name, expected_order)
),
actual_sequence AS (
    SELECT
        task_name,
        ROW_NUMBER() OVER (ORDER BY start_time) AS actual_order,
        status,
        start_time,
        end_time
    FROM etl_control.dbo.DAG_TASK_LOG
    WHERE run_id = (SELECT MAX(run_id)
        FROM etl_control.dbo.DAG_TASK_LOG)
)
SELECT
    e.task_name,
    e.expected_order,
    a.actual_order,
    a.status,
    a.start_time,
    CASE
        WHEN a.task_name IS NULL THEN 'TASK SKIPPED ✖'
        WHEN a.actual_order
            > e.expected_order + 1 THEN 'RUNS TOO LATE ⚠'
        WHEN a.actual_order
            < e.expected_order - 1 THEN 'RUNS TOO EARLY ⚠'
        WHEN a.status = 'SKIPPED' THEN 'WRONGLY SKIPPED ✖'
        ELSE 'CORRECT ✔'
    END AS sequence_status
FROM expected_sequence e
LEFT JOIN actual_sequence a ON e.task_name = a.task_name
ORDER BY e.expected_order;
-- — Parallel branch convergence: both parallel tasks done —
SELECT
    'PARALLEL CONVERGENCE CHECK' AS check_name,
    dim_cust.status AS dim_customers_status,
    dim_prod.status AS dim_products_status,
    fact.start_time AS fact_start,
    GREATEST(dim_cust.end_time,
        dim_prod.end_time) AS both_dims_done,
    CASE
        WHEN dim_cust.status = 'SUCCESS'
            AND dim_prod.status = 'SUCCESS'
            AND fact.start_time >= GREATEST(dim_cust.end_time,
                dim_prod.end_time)
        THEN 'CONVERGENCE CORRECT ✔'
        WHEN fact.start_time < GREATEST(dim_cust.end_time,
            dim_prod.end_time)
        THEN 'FACT STARTED BEFORE ALL DIMS DONE ✖'
        WHEN dim_cust.status <> 'SUCCESS'
            OR dim_prod.status <> 'SUCCESS'
        THEN 'FACT SHOULD NOT HAVE STARTED ✖'
        ELSE 'REVIEW ⚠'
    END AS convergence_status
FROM etl_control.dbo.DAG_TASK_LOG dim_cust
JOIN etl_control.dbo.DAG_TASK_LOG dim_prod

```

```

    ON dim_prod.task_name = 'LOAD_DIM_PRODUCTS'
    AND dim_prod.run_id = dim_cust.run_id
JOIN etl_control.dbo.DAG_TASK_LOG fact
    ON fact.task_name = 'LOAD_FACT_ORDERS'
    AND fact.run_id = dim_cust.run_id
WHERE dim_cust.task_name = 'LOAD_DIM_CUSTOMERS'
    AND dim_cust.run_id = (
        SELECT MAX(run_id) FROM etl_control.dbo.DAG_TASK_LOG
    );
-- — Missing tasks: tasks in definition not in run log —————
SELECT
    d.task_name          AS expected_task,
    l.task_name          AS actual_task,
    l.status,
    CASE
        WHEN l.task_name IS NULL      THEN 'NEVER EXECUTED ✕'
        WHEN l.status = 'SKIPPED'     THEN 'SKIPPED ⚠'
        ELSE                          'RAN ✔'
    END AS execution_status
FROM etl_control.dbo.DAG_DEFINITION d
LEFT JOIN etl_control.dbo.DAG_TASK_LOG l
    ON l.task_name = d.task_name
    AND l.run_id = (
        SELECT MAX(run_id) FROM etl_control.dbo.DAG_TASK_LOG
    )
WHERE d.dag_name = 'ORDERS_PIPELINE'
ORDER BY d.task_order;

```

3. Failure Handling Validation

Purpose: Production pipelines fail. The question is not if but when.

Validate that failures are caught, logged with actionable context, do not corrupt downstream state, trigger the correct fallback path, and leave the system in a deterministic, recoverable state.

```

-- — Failure log completeness: every failed run has error detail —
SELECT
    r.run_id,
    r.pipeline_name,
    r.batch_date,
    r.run_status,
    r.run_start,
    r.run_end,
    e.error_code,
    e.error_message,
    e.failed_task,
    e.stack_trace_available,
    CASE
        WHEN r.run_status = 'FAILED'
            AND e.run_id IS NULL      THEN 'FAILED WITH NO ERROR LOG ✕'
        WHEN r.run_status = 'FAILED'
            AND e.error_message IS NULL THEN 'FAILED WITH NULL MESSAGE ✕'
        WHEN r.run_status = 'FAILED'
            AND e.error_code IS NULL   THEN 'FAILED WITH NULL ERROR CODE ⚠'
        WHEN r.run_status = 'FAILED'
            AND e.error_message IS NOT NULL THEN 'FAILURE LOGGED ✔'
        ELSE 'N/A'
    END AS failure_log_status
FROM etl_control.dbo.PIPELINE_RUN_LOG r
LEFT JOIN etl_control.dbo.PIPELINE_ERROR_LOG e
    ON e.run_id = r.run_id
WHERE r.run_status = 'FAILED'
    AND r.batch_date >= DATEADD(DAY, -30, GETDATE())
ORDER BY r.run_start DESC;
-- — Downstream isolation: failed task blocked all dependents —
SELECT
    failed_task.task_name      AS failed_task,

```

```

downstream.task_name      AS downstream_task,
downstream.status         AS downstream_status,
CASE
    WHEN downstream.status = 'SKIPPED' THEN 'CORRECTLY BLOCKED ✓'
    WHEN downstream.status = 'SUCCESS' THEN 'RAN DESPITE FAILURE ✗'
    WHEN downstream.status = 'FAILED'  THEN 'CASCADED FAILURE ⚠'
    ELSE                               'REVIEW ⚠'
END AS isolation_status
FROM etl_control.dbo.DAG_TASK_LOG failed_task
JOIN etl_control.dbo.DAG_TASK_LOG downstream
    ON downstream.run_id = failed_task.run_id
    AND downstream.depends_on = failed_task.task_name
WHERE failed_task.status = 'FAILED'
AND failed_task.run_id = (
    SELECT run_id FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE run_status = 'FAILED'
    ORDER BY run_start DESC
    OFFSET 0 ROWS FETCH NEXT 1 ROWS ONLY
);
-- — Partial write detection: failed jobs must not leave orphans —
SELECT
    f.ORDER_KEY,
    f.LOAD_BATCH_ID,
    f.LOAD_TIMESTAMP,
    r.run_status,
    CASE
        WHEN r.run_status = 'FAILED'
        THEN 'ORPHAN ROWS FROM FAILED RUN ✗'
        ELSE 'CLEAN ✓'
    END AS orphan_status
FROM dwh.dbo.FACT_ORDERS f
JOIN etl_control.dbo.PIPELINE_RUN_LOG r
    ON r.load_batch_id = f.LOAD_BATCH_ID
WHERE r.run_status = 'FAILED';
-- Must return 0 rows: failed runs must not commit partial data
-- — Error categorisation: known vs unknown failures —
SELECT
    error_code,
    error_category,
    COUNT(*) AS occurrence_count,
    COUNT(DISTINCT batch_date) AS affected_days,
    MAX(failed_at) AS last_occurrence,
    CASE error_category
        WHEN 'SOURCE_UNAVAILABLE' THEN 'INFRA ⚡'
        WHEN 'DATA_QUALITY_FAILURE' THEN 'DATA 📊'
        WHEN 'MEMORY_OOM' THEN 'RESOURCE 📁'
        WHEN 'SCHEMA_MISMATCH' THEN 'SCHEMA 📁'
        WHEN 'TIMEOUT' THEN 'PERF ⌚'
        ELSE 'UNKNOWN ?'
    END
    AS failure_class
FROM etl_control.dbo.PIPELINE_ERROR_LOG
WHERE failed_at >= DATEADD(DAY, -30, GETDATE())
GROUP BY error_code, error_category
ORDER BY occurrence_count DESC;

```

4. Restart from Failure Point

Purpose: When a pipeline fails at step 7 of 10, it must be able to

restart from step 7 — not from step 1. Full reruns of multi-hour pipelines waste cluster resources and violate SLAs. Validate that checkpointing is implemented, restarts resume from the correct task, and no data is duplicated by a restart.

```

-- — Checkpoint log: each task records its completion state —
SELECT
    checkpoint_id,

```

```

    pipeline_name,
    batch_date,
    task_name,
    checkpoint_status,
    checkpoint_ts,
    rows_processed,
    watermark_value,
    CASE
        WHEN checkpoint_status = 'COMPLETE' THEN 'SAFE TO SKIP ON RESTART ✓'
        WHEN checkpoint_status = 'PARTIAL' THEN 'MUST REPROCESS ⚠'
        WHEN checkpoint_status IS NULL THEN 'NO CHECKPOINT – FULL RERUN ✗'
        ELSE checkpoint_status
    END AS restart_eligibility
FROM etl_control.dbo.PIPELINE_CHECKPOINT_LOG
WHERE batch_date = CAST(GETDATE()-1 AS DATE)
ORDER BY checkpoint_ts;
-- — Restart detection: which tasks were skipped on last restart –
WITH run_a AS (
    SELECT task_name, status, start_time, run_id
    FROM etl_control.dbo.DAG_TASK_LOG
    WHERE run_id = (
        SELECT MIN(run_id) FROM etl_control.dbo.PIPELINE_RUN_LOG
        WHERE batch_date = CAST(GETDATE()-1 AS DATE)
    )
),
run_b AS (
    SELECT task_name, status, start_time, run_id
    FROM etl_control.dbo.DAG_TASK_LOG
    WHERE run_id = (
        SELECT MAX(run_id) FROM etl_control.dbo.PIPELINE_RUN_LOG
        WHERE batch_date = CAST(GETDATE()-1 AS DATE)
    )
)
SELECT
    a.task_name,
    a.status AS first_run_status,
    b.status AS restart_status,
    CASE
        WHEN a.status = 'SUCCESS'
            AND b.status = 'SKIPPED' THEN 'CORRECTLY SKIPPED ✓'
        WHEN a.status = 'SUCCESS'
            AND b.status = 'SUCCESS' THEN 'RE-EXECUTED (DUPLICATE RISK ⚠)'
        WHEN a.status = 'FAILED'
            AND b.status = 'SUCCESS' THEN 'RECOVERED ✓'
        WHEN a.status = 'FAILED'
            AND b.status = 'FAILED' THEN 'STILL FAILING ✗'
        WHEN b.task_name IS NULL THEN 'NOT IN RESTART ⚠'
        ELSE 'REVIEW'
    END AS restart_behaviour
FROM run_a a
LEFT JOIN run_b b ON a.task_name = b.task_name
ORDER BY a.start_time;
-- — Idempotency: restart must not duplicate rows —————
SELECT
    LOAD_BATCH_ID,
    COUNT(*) AS rows_in_batch,
    COUNT(DISTINCT ORDER_KEY) AS distinct_keys,
    COUNT(*) -
    COUNT(DISTINCT ORDER_KEY) AS duplicate_rows,
    CASE
        WHEN COUNT(*) = COUNT(DISTINCT ORDER_KEY)
        THEN 'IDEMPOTENT ✓'
        ELSE 'DUPLICATED BY RESTART ✗'
    END AS idempotency_status
FROM dwh.dbo.FACT_ORDERS
WHERE LOAD_BATCH_ID IN (
    SELECT load_batch_id FROM etl_control.dbo.PIPELINE_RUN_LOG

```

```

WHERE batch_date = CAST(GETDATE()-1 AS DATE)
)
GROUP BY LOAD_BATCH_ID;

```

5. Alert & Notification Validation

Purpose: An alert that fires too late, goes to the wrong team, or

contains no actionable information is worse than no alert — it trains engineers to ignore notifications. Validate that the alerting system triggers on the right conditions, reaches the right recipients within SLA, and includes enough context to act without needing a dashboard login.

```

-- — Alert trigger validation: correct conditions fired alerts —
SELECT
    e.run_id,
    e.pipeline_name,
    e.error_code,
    e.failed_at,
    a.alert_id,
    a.alert_type,
    a.recipient,
    a.sent_at,
    DATEDIFF(SECOND, e.failed_at, a.sent_at) AS seconds_to_alert,
    a.alert_sla_seconds,
    CASE
        WHEN a.alert_id IS NULL
        THEN 'NO ALERT SENT ✖'
        WHEN DATEDIFF(SECOND, e.failed_at, a.sent_at)
            > a.alert_sla_seconds
        THEN 'ALERT LATE ✖'
        WHEN a.recipient IS NULL
        THEN 'NO RECIPIENT ✖'
        ELSE 'ALERT OK ✔'
    END AS alert_status
FROM etl_control.dbo.PIPELINE_ERROR_LOG e
LEFT JOIN etl_control.dbo.ALERT_LOG a
    ON a.run_id = e.run_id
    AND a.alert_type = 'PIPELINE_FAILURE'
WHERE e.failed_at >= DATEADD(DAY, -7, GETDATE())
ORDER BY e.failed_at DESC;

-- — False positive alerts: alerts fired when pipeline succeeded —
SELECT
    a.alert_id,
    a.alert_type,
    a.run_id,
    a.sent_at,
    r.run_status,
    CASE
        WHEN r.run_status = 'SUCCESS'
        THEN 'FALSE POSITIVE ✖'
        ELSE 'VALID ALERT ✔'
    END AS false_positive_check
FROM etl_control.dbo.ALERT_LOG a
JOIN etl_control.dbo.PIPELINE_RUN_LOG r ON a.run_id = r.run_id
WHERE a.alert_type = 'PIPELINE_FAILURE'
    AND a.sent_at >= DATEADD(DAY, -7, GETDATE())
    AND r.run_status = 'SUCCESS'; -- must return 0 rows

-- — Alert content completeness —
SELECT
    alert_id,
    alert_type,
    subject,
    CASE WHEN body LIKE '%pipeline_name%' THEN 1 ELSE 0 END AS has_pipeline,
    CASE WHEN body LIKE '%batch_date%' THEN 1 ELSE 0 END AS has_date,
    CASE WHEN body LIKE '%error_message%' THEN 1 ELSE 0 END AS has_error,
    CASE WHEN body LIKE '%run_id%' THEN 1 ELSE 0 END AS has_run_id,

```



```

CASE WHEN body LIKE '%steps_to_fix%'
      OR body LIKE '%runbook%'
      OR body LIKE '%wiki%' THEN 1 ELSE 0 END AS has_runbook,
(
CASE WHEN body LIKE '%pipeline_name%' THEN 1 ELSE 0 END +
CASE WHEN body LIKE '%batch_date%' THEN 1 ELSE 0 END +
CASE WHEN body LIKE '%error_message%' THEN 1 ELSE 0 END +
CASE WHEN body LIKE '%run_id%' THEN 1 ELSE 0 END +
CASE WHEN body LIKE '%runbook%' THEN 1 ELSE 0 END
) AS content_score,
CASE
  WHEN (CASE WHEN body LIKE '%pipeline_name%' THEN 1 ELSE 0 END +
        CASE WHEN body LIKE '%batch_date%' THEN 1 ELSE 0 END +
        CASE WHEN body LIKE '%error_message%' THEN 1 ELSE 0 END +
        CASE WHEN body LIKE '%run_id%' THEN 1 ELSE 0 END
       ) >= 4 THEN 'ACTIONABLE 🟢'
  WHEN (CASE WHEN body LIKE '%pipeline_name%' THEN 1 ELSE 0 END +
        CASE WHEN body LIKE '%error_message%' THEN 1 ELSE 0 END
       ) >= 2 THEN 'PARTIAL 🟡'
  ELSE 'VAGUE 🟠'
END AS alert_quality
FROM etl_control.dbo.ALERT_LOG
WHERE sent_at >= DATEADD(DAY, -7, GETDATE());

```

6. SLA Compliance Validation

Purpose: SLA compliance is the ultimate business contract for a data

pipeline. Business users need data by 8 AM. If the pipeline finishes at 8:47 AM, the SLA is breached — even if the data is perfect. Validate SLA adherence historically, detect trends toward breach, and identify which pipeline stages are the consistent bottlenecks.

```

-- — SLA compliance per pipeline over 30 days —————
SELECT
  pipeline_name,
  COUNT(*) AS total_runs,
  SUM(CASE WHEN DATEDIFF(SECOND, run_start,
                        run_end)
           <= sla.sla_seconds THEN 1 ELSE 0 END) AS on_time,
  SUM(CASE WHEN DATEDIFF(SECOND, run_start,
                        run_end)
           > sla.sla_seconds THEN 1 ELSE 0 END) AS breached,
  ROUND(100.0 * SUM(CASE WHEN DATEDIFF(SECOND, run_start,
                        run_end) <= sla.sla_seconds
                    THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 2) AS sla_compliance_pct,
  sla.sla_seconds AS sla_budget_secs,
  AVG(DATEDIFF(SECOND, run_start, run_end)) AS avg_duration_secs,
  MAX(DATEDIFF(SECOND, run_start, run_end)) AS max_duration_secs,
  PERCENTILE_CONT(0.95) WITHIN GROUP (
    ORDER BY DATEDIFF(SECOND, run_start, run_end)
  ) AS p95_duration_secs,
CASE
  WHEN ROUND(100.0 * SUM(CASE WHEN DATEDIFF(SECOND,
    run_start, run_end) <= sla.sla_seconds
    THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0), 2) >= 99
  THEN 'EXCELLENT 🟢'
  WHEN ROUND(100.0 * SUM(CASE WHEN DATEDIFF(SECOND,
    run_start, run_end) <= sla.sla_seconds
    THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0), 2) >= 95
  THEN 'ACCEPTABLE 🟡'
  ELSE 'AT RISK 🟠'
END AS sla_health
FROM etl_control.dbo.PIPELINE_RUN_LOG r
JOIN etl_control.dbo.PIPELINE_SLA sla
ON sla.pipeline_name = r.pipeline_name

```

```

WHERE r.run_status = 'SUCCESS'
AND r.batch_date >= DATEADD(DAY, -30, GETDATE())
GROUP BY r.pipeline_name, sla.sla_seconds
ORDER BY sla_compliance_pct ASC;
-- — Trend: is performance degrading toward SLA breach? —————
SELECT
    batch_date,
    DATEDIFF(SECOND, run_start, run_end) AS duration_secs,
    sla.sla_seconds AS sla_budget,
    ROUND(100.0 * DATEDIFF(SECOND, run_start, run_end)
        / NULLIF(sla.sla_seconds, 0), 1) AS pct_of_sla_used,
    AVG(DATEDIFF(SECOND, run_start, run_end))
        OVER (ORDER BY batch_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d_avg,
    CASE
        WHEN AVG(DATEDIFF(SECOND, run_start, run_end))
            OVER (ORDER BY batch_date
                ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
            > sla.sla_seconds * 0.9
        THEN 'TRENDING TO BREACH ⚠'
        ELSE 'HEALTHY ✅'
    END AS trend_status
FROM etl_control.dbo.PIPELINE_RUN_LOG r
JOIN etl_control.dbo.PIPELINE_SLA sla
    ON sla.pipeline_name = r.pipeline_name
WHERE r.pipeline_name = 'ORDERS_ETL'
AND r.run_status = 'SUCCESS'
AND r.batch_date >= DATEADD(DAY, -30, GETDATE())
ORDER BY batch_date;
-- — Stage-level bottleneck: which task consumes most SLA —————
SELECT
    task_name,
    AVG(DATEDIFF(SECOND, start_time, end_time)) AS avg_secs,
    MAX(DATEDIFF(SECOND, start_time, end_time)) AS max_secs,
    SUM(DATEDIFF(SECOND, start_time, end_time)) AS total_secs,
    ROUND(100.0 * AVG(DATEDIFF(SECOND, start_time, end_time))
        / SUM(AVG(DATEDIFF(SECOND, start_time, end_time))
            OVER (), 2) AS pct_of_pipeline,
    CASE
        WHEN ROUND(100.0 * AVG(DATEDIFF(SECOND, start_time, end_time))
            / SUM(AVG(DATEDIFF(SECOND, start_time, end_time))
                OVER (), 2) > 50
        THEN 'BOTTLENECK ❌'
        WHEN ROUND(100.0 * AVG(DATEDIFF(SECOND, start_time, end_time))
            / SUM(AVG(DATEDIFF(SECOND, start_time, end_time))
                OVER (), 2) > 30
        THEN 'HIGH CONSUMER ⚠'
        ELSE 'NORMAL ✅'
    END AS sla_impact
FROM etl_control.dbo.DAG_TASK_LOG
WHERE run_id IN (
    SELECT run_id FROM etl_control.dbo.PIPELINE_RUN_LOG
    WHERE pipeline_name = 'ORDERS_ETL'
    AND run_status = 'SUCCESS'
    AND batch_date >= DATEADD(DAY, -30, GETDATE())
)
GROUP BY task_name
ORDER BY avg_secs DESC;

```

Closing Perspective

SQL becomes a powerful ETL testing tool when every query is tied to a quality question: What can be wrong, how can I detect it, and what evidence proves the data is correct?

The supplied reference demonstrates that validation can be performed across the complete data lifecycle—from source readiness and extraction through transformation, warehouse modeling, reconciliation, incremental processing, data quality, metadata, reporting, Big Data platforms, performance and workflow controls.

Use the playbook as a working reference—not as a list to memorize.